5 TEAM
FINAL PROJECT

All - Care

Medi- Core

AI 신체 질환 진단 의료 보조 시스템

ML+DL+LLM

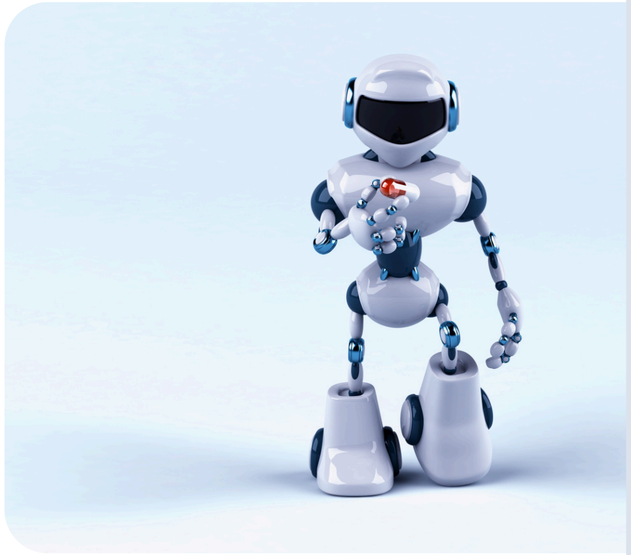
AI HEALTH CARE SYSTEM

질병의 불확실성을 '0'으로 통제하는 궁극의 메디컬 지능망

국소적인 단일 질환 진단의 한계를 파괴하고, 인체의 뇌, 척추, 장기, 피부, 안과를 아우르는 전방위적 AI 진단/관리 생태계 구축.

다양한 딥러닝 모델을 LangGraph 기반의 메인 두뇌(Supervisor)가 수족처럼 지휘하여, 오진율 0%에 도전하는 초거대 맞춤형 헬스케어 에이전트 시스템

김남준(PM), 김민수(PL), 변운조, 김담현, 박기완, 홍승현,김용



INDEX

1.개요

- 추진 배경 및 목적
- 기존 서비스 및 차별점
- 팀원 소개
- 개발 환경
- 일정계획

2. 기획

- 메뉴 구조도
- 기능정의 명세서
- 정책정의서

3. 설계

- 시스템 아키텍쳐
- 시스템 흐름도
- 네트워크 구조
- ERD

4. 구현

- 화면 구현

5. 모델 구현

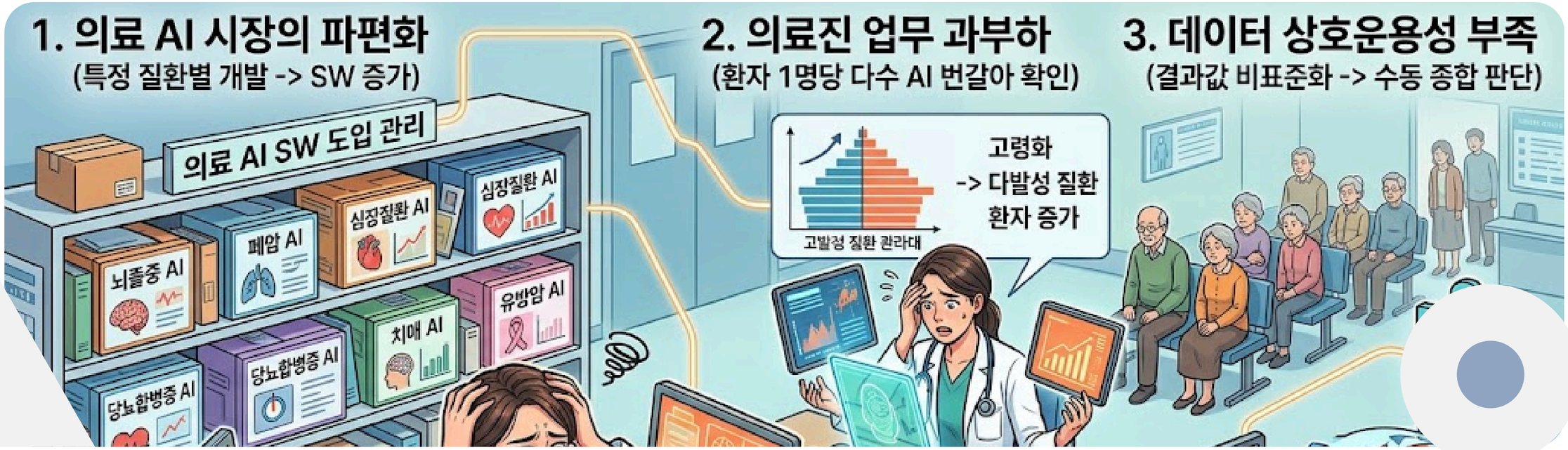
- 척추협착증 진단
- 대장암 예측
- 뇌종양 진단
- 만성 신부전 관리
- 피부 질환 분류
- 안과 진단

6. 테스트

- 단위 테스트 시나리오
- 단위 테스트 케이스
- 통합 테스트 시나리오
- 통합 테스트 케이스

프로젝트 개요

순서 추진 배경 및 목적, 기존 서비스 및 차별점, 팀원 소개, 개발 환경, 일정계획

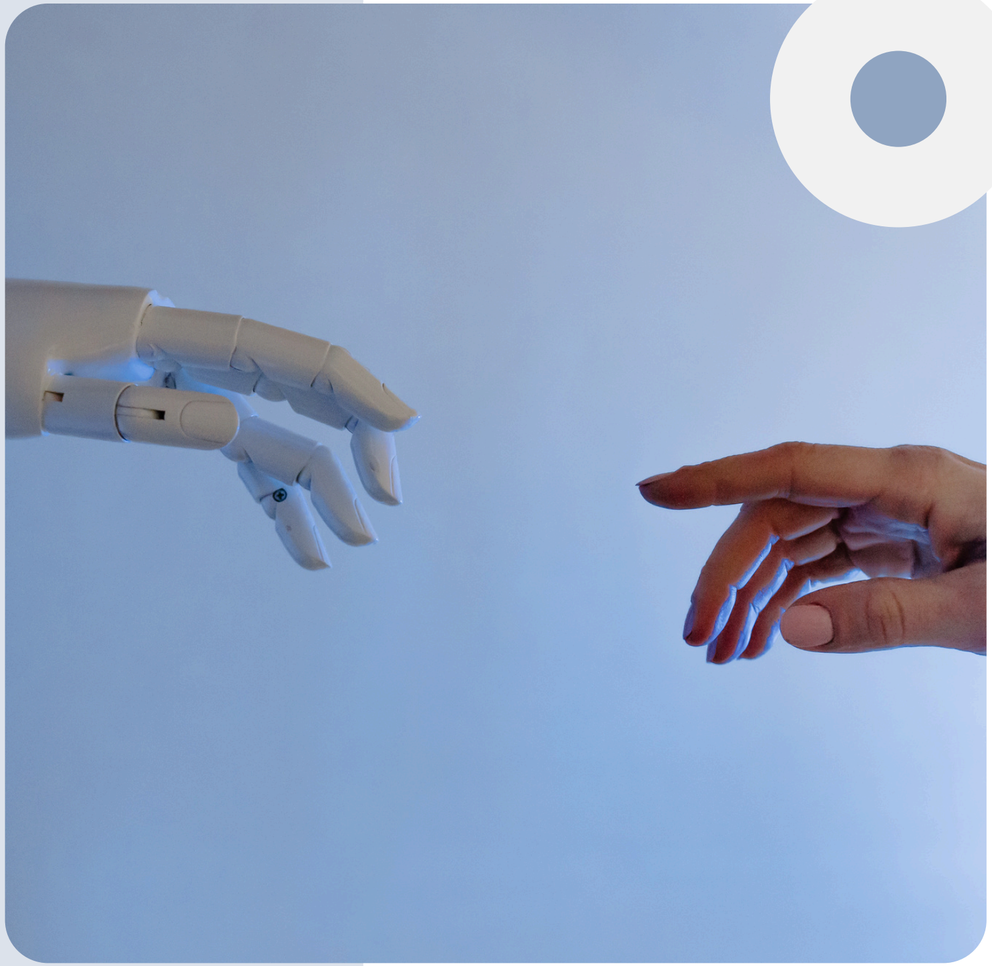


프로젝트 추진배경

1. 의료 AI 시장의 파편화
현재 의료 AI는 특정 질환별로 개발되어 있어, 병원 입장에서는 도입할수록 관리해야 할 소프트웨어만 늘어남.

2. 의료진의 업무 과부하
고령화로 인해 다발성 질환(복합 질환) 환자가 늘고 있지만, 의사는 1:N 구조로 다수의 환자를 케어해야 하는 상황이므로, 시간이 부족하며 일의 과정이 점점 늘어남.

3. 데이터 상호운용성 부족
각기 다른 AI 모델들이 내놓는 결과값이 표준화되지 않아, 이를 종합적으로 판단하기 위한 수동 작업이 발생함.

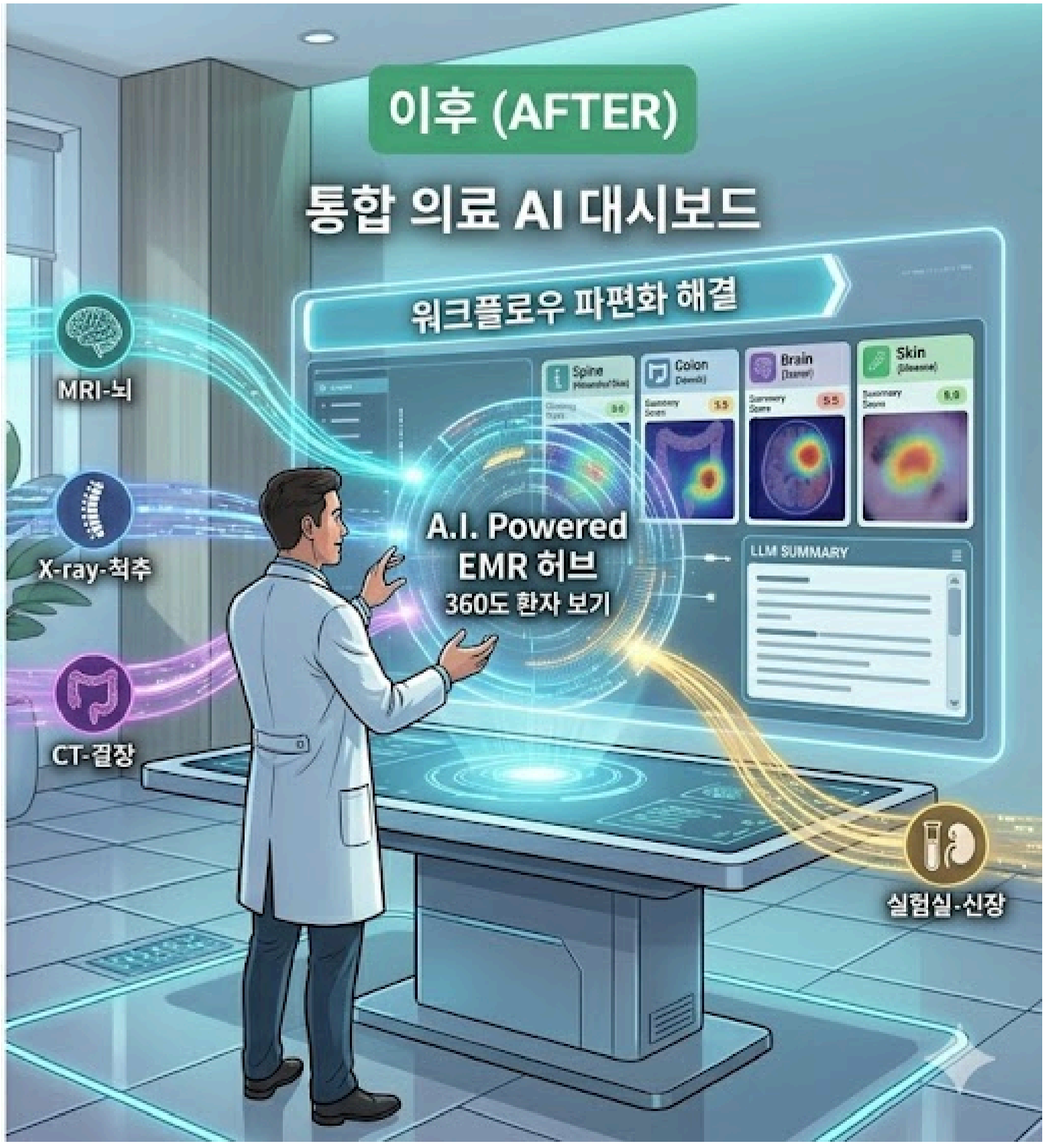


프로젝트 목적

질병의 불확실성을 '0'으로 수렴시키는 LANGGRAPH 기반 초거대 통합 헬스케어 생태계 구축

본 프로젝트는 특정 단일 질환의 진단을 넘어, LangGraph의 멀티 에이전트(Multi-Agent) 아키텍처를 활용한 중앙 통제식 의료 AI 플랫폼을 구축하는 것을 목표로 합니다. 환자의 단일 인터페이스(창구)를 통해 입력된 데이터는, 메인 두뇌 역할을 하는 라우터(Supervisor)에 의해 각 질환별 특화 에이전트로 분배되며, 최종적으로 환각(Hallucination) 없는 무결점의 통합 진단 리포트를 제공

이전 (BEFORE)
워크플로우 파편화
데이터 사일로
결장 CT
뇌 MRI
척추 MRI
피부 생검


이후 (AFTER)
통합 의료 AI 대시보드
워크플로우 파편화 해결
MRI-뇌
X-ray-척추
CT-결장
A.I. Powered
EMR 허브
360도 환자 보기
LLM SUMMARY
실험실-신장

작성자 : 김담현

프로세스

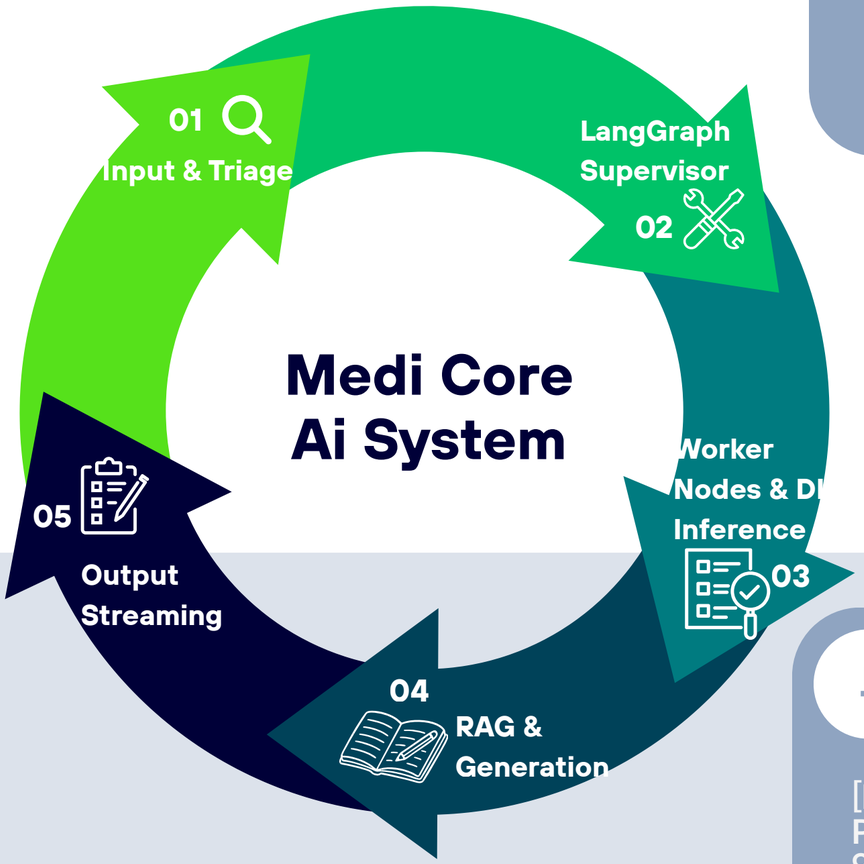
데이터 입력

[CLIENT - REACT]
사진을 업로드합니다.

[BACKEND - SPRING BOOT]
로드된 이미지를 정규화하여 임시 폴더에 적재한 뒤, LANGGRAPH 엔진을 호출

중앙 통제 및 하달

[SUPERVISOR NODE]
LANGGRAPH의 메인 라우터(두뇌)가 환자의 입력을 분석



에이전트 분석

각 에이전트(노드)는 분리된 딥러닝 추론 서버(PYTHON/FLASK 또는 FASTAPI)로 API 요청을 보냄

결과 송출

[BACKEND TO CLIENT]
생성된 리포트와 병변이 표시된 이미지를 화면(REACT)에 실시간으로 노출

무결점 검증 및 판결

[RAG NODE] 도출된 DL 분석 결과를 바탕으로, 에이전트가 PGVECTOR에 구축된 최신 의학 논문 및 보건복지부 가이드라인 DB를 뒤져 정확한 치료 근거를 가져옴

[GENERATOR NODE] 모든 데이터를 취합하여, 환각(HALLUCINATION)이 완전히 배제된 하나의 완벽한 진단 리포트 생성

5조
팀원소개

김담현
척추협착증

변운조
뇌종양 진단 모델

김남준
만성 신부전 관리 진단 모델

김민수
피부질환 분류 모델

박기완
시스템 관리

홍승현
안과 진단 모델

김용
데이터셋 수집

작성자 : 김민수

일정

추진내용			프로젝트 기간 (21일)																			
			17일	20일	21일	22일	23일	24일	27일	28일	29일	30일	1일	4일	5일	6일	7일	8일	11일	12일	13일	14일	15일
요구분석	기획	기획안작성																					
		개발환경설정																					
		역할분담																					
	분석	요구사항 수집/정의																					
		시스템 아키텍처 정의																					
설계	DB 설계	ERD																					
	화면설계	스토리보드																					
개발	프론트엔드	동적 웹페이지 개발																					
		사용자입력/ 결과페이지 구현																					
	백엔드	시스템 공통																					
		AI 예측모델 구현																					
		실시간 데이터 연동																					
	관리	시스템 관리																					
	머신러닝/딥러닝	모델학습 및 성능 개선																					
	LLM	LLM 통합 및 튜닝																					
테스트	단위테스트																						
	통합테스트																						
최종보고서 작성 및 수정 보안																							
프로젝트 수행 착수/중간/최종보고																							

프로젝트 기획

순서	기술 스택, 요구사항 정의서, 기능 명세서, 정책 정의서,메뉴 구조도

기술스텍

기술스텍

번호	분류	기술 항목	상세 역할 및 비고
1	Frontend	React, TailwindCSS	UI 컴포넌트 개발 및 유틸리티 기반 신속한 스타일링
2	Backend	Spring Boot	비즈니스 로직 제어, 데이터 전처리, 통합 관리 서버
3	AI Server	FastAPI	고성능 비동기 AI API 서빙 및 전용 추론 엔진
4	AI Engine	Gemini	RAG 기반 진단 소견 생성 및 치료 권고안 추론
5	AI Framework	Langchain	LLM 오케스트레이션 및 RAG 파이프라인 관리
6	Database	PostgreSQL (+pgvector)	관계형 데이터 저장 및 가이드 라인 벡터 데이터 통합 관리
7	ML/Vision	Scikit-Learn, PyTorch, YOLO	임상 데이터 전처리 및 시각 자료(초음파 등) 확장 분석 대비
8	Authentication	Spring Security, JWT	의료진 면허 기반 인증 및 토큰 기반 보안 세션 관리
9	Infra / DevOps	AWS(EC2), Docker	확장 가능한 서버 인스턴스 및 컨테이너 기반 환경 격리
10	CI / CD	Github Actions	지속적 통합 및 자동 배포 파이프라인 구축
11	Networking	Axios, REST API	프론트엔드 - 백엔드 간 JSON 데이터 통신
12	협업 도구	Github, Jira, Slack	코드 형상 관리, 업무 일정 관리 및 팀 실시간 소통

요구사항정의서

페이지명	로그인 페이지	작성자	변운조	
ID	**기능명**	**요구사항 상세**	**사용자**	**우선순위**	**비고**
COM-01	로그인 페이지(최초 화면)	서비스 접속 시 가장 먼저 노출되며, 의사 면허 번호(ID)와 비밀번호를 입력받는다.	의사	상	Unauthorized 접근 시 자동 리다이렉트
COM-02	의사 회원가입	의사 기본 정보(이름, 면허번호, 소속 과)를 입력받아 계정을 생성한다.	의사	상	중복 가입 방지 및 유효성 검사
COM-03	JWT 인증 처리	로그인 성공 시 서버로부터 JWT 토큰을 발급받아 로컬 스토리지/쿠키에 저장한다.	시스템	상	보안 및 API 호출 시 필수
COM-04	메인 대시보드 이동	인증 완료 시 헤더, 사이드바가 포함된 메인 페이지로 자동 이동한다.	의사	상	React Router 활용
COM-05	통합 환자 검색	메인 페이지 상단에서 이름/환자번호로 통합 검색 기능을 제공한다.	의사	상	전체 질환 모델 연동의 시작점
COM-06	실시간 모델 상태 확인	현재 가동 중인 5개 AI 서버의 상태(정상/지연/점검)를 메인에 표시한다.	의사	중	운영 효율성 증대
COM-07	로그아웃	토큰을 파기하고 다시 로그인 초기 화면으로 이동시킨다.	의사	상	세션 만료 및 보안 처리

페이지명	회원가입 페이지	작성자	김민수	
ID	**기능명**	**요구사항 상세 (상세 로직)**	**사용자**	**우선순위**	**비고**
AUTH-01	의사 회원가입	이름, 의사 면허 번호(ID로 활용), 비밀번호, 소속 과(내과/피부과 등), 이메일을 입력받아 저장한다.	의사	상	비밀번호는 반드시 암호화 처리
AUTH-02	면허 번호 중복 체크	가입 시 이미 등록된 면허 번호인지 DB에서 실시간으로 확인하여 중복 가입을 방지한다.	의사	상	Unique Constraint 활용
AUTH-03	로그인 및 JWT 발급	ID/PW 일치 시 서버에서 **JWT(AccessToken)**를 생성하여 반환하고, 클라이언트는 이를 로컬 스토리지에 저장한다.	의사	상	유효기간 설정 (예: 2시간)
AUTH-04	로그인 유효성 검사	아이디나 비밀번호가 틀릴 경우 구체적인 사유 대신 "정보가 일치하지 않습니다"라는 통합 메시지를 출력한다.	의사	중	보안(계정 탈취 방지) 목적
AUTH-05	인증 가드 (Auth Guard)	로그인하지 않은 사용자가 URL(예: /skin, /brain)로 직접 접근 시 자동으로 로그인 페이지로 리다이렉트한다.	시스템	상	React Router Protected Route
AUTH-06	자동 로그아웃	토큰 만료 시 세션을 종료하고 로그인 화면으로 이동시키며, 로그아웃 버튼 클릭 시 로컬 토큰을 즉시 파기한다.	의사	상	보안 강화

요구사항정의서

페이지명	환자 상세 페이지	작성자	김민수	
ID	기능명	요구사항 상세	사용자	우선순위	비고
DBD-01	환자 프로필 카드	환자의 이름, UID, 나이, 성별, 혈액형, 최근검사일, 담당의, 복용약물, 의료팀 정보를 카드로 표시한다. 필드 값이 없으면 '-'로 처리한다.	의사	상	GET /api/patients/:uid, EMR DB 연동
DBD-02	진단 분석 안내 패널	6개 질환 모듈 배지를 표시하고, 사이드바에서 항목을 선택하면 AI 진단이 가능함을 안내하는 설명 텍스트를 노출한다.	의사	중	DISEASE_LABELS 정적 정의
DBD-03	누적 리포트 현황	해당 환자에 저장된 전체 진단 리포트 건수를 실시간으로 표시하고, 1건 이상일 시 '진단 기록 있음' 배지를 노출한다.	의사	중	GET /api/patients/:uid/diagnoses
DBD-04	진단 리포트 카드 목록	저장된 전체 진단 이력을 질환별 아이콘·색상으로 구분한 카드 목록으로 출력한다. 카드에는 질환 유형, 진단명, 요약, 진단의, 날짜를 표시한다.	의사	상	GET /api/patients/:uid/diagnoses, DISEASE_LABELS 매핑
DBD-05	리포트 없음 Empty 상태	진단 이력이 없을 경우 안내 문구와 함께 사이드바에서 진단 항목을 선택하도록 유도 메시지를 표시한다.	의사	중	diagnoses.length === 0 분기
DBD-06	데이터 로딩 / 오류 처리	환자 정보 로딩 중 스피너를 표시하고, 환자 조회 실패 시 '환자를 찾을 수 없습니다' 오류 화면을 노출한다.	의사	상	Promise.all 병렬 조회, loading/patient null 분기
DBD-07	현재 환자 컨텍스트 배너	사이드바 상단에 현재 선택된 환자명과 UID를 파란색 배너로 상시 노출한다. 사이드바 축소 시 환자명 첫 글자 아이콘으로 대체한다.	의사	상	URL params patientId, props patientName
DBD-08	질환별 탭 네비게이션	통합 대시보드 및 6개 질환 모듈(뇌종양·허리디스크·대장암·신부전·피부질환·안과)을 사이드바 탭으로 제공하고, 현재 활성 탭을 파란색으로 강조한다.	의사	상	/patients/:uid/:diseaseKey 라우팅
DBD-09	환자 목록 복귀 버튼	사이드바 하단에서 통합 대시보드(/dashboard)로 복귀하는 버튼을 제공한다.	의사	중	navigate('/dashboard')

요구사항정의서

	페이지명	분석 페이지 (신부전 AI)	작성자	김남준	
ID	기능명	요구사항 상세	사용자	우선순위	비고
CRF-01	임상 지표 입력 및 로드	24개 필수 검사 지표 수동 입력 및 CSV 파일 업로드하여 동기화	의사	상	데이터 정규화 포함
CRF-02	AI 신부전 단계 진단	분석 시작' 버튼 클릭 시 AI가 단계를 필독(1~5단계)하고 게이지 표시	의사	상	TabNet 모델 활용
CRF-03	진단 근거(XAI) 분석	SHAP 차트를 통해 어떤 지표(혈압, 당뇨 등)가 진단에 기여했는지 확인	의사	중	설명 가능한 AI
CRF-04	환자 상태 추이 분석	과거 검사 이력과 현재 수치를 비교한 그래프를 보며 환자의 악화 속도를 시각적으로 파악	의사	중	데이터 시각화
CRF-05	맞춤형 지침 자동 검색	판독 결과에 맞는 최신 의학 가이드라인 문구가 RAG를 통해 우측 탭에 자동 노출	의사	상	RAG 검색 엔진
CRF-06	AI 처방 및 식이 제언	LLM이 요약한 약물 조절 및 식단 가이드를 확인하고 실제 문헌 원문 링크로 검증	의사	상	LLM 기반 요약
CRF-07	투약 시뮬레이션	특정 수치(ex. 혈압) 개선 시 예상되는 단계 변화를 가상으로 조정	의사	하	What-if 분석
CRF-08	유사 환자 사례 추천	현재 환자와 유사한 수치를 가졌던 과거 익명 환자의 치료 경과를 참고	의사	하	케이스 스터디 활용
CRF-09	종합 소견서 생성	모든 분석 내용을 통합한 리포트를 생성하고 의사 소견을 추가하여 PDF로 출력	의사	중	상담용 산출물
CRF-10	지식 베이스 업데이트	새로운 가이드라인이 발표될 경우 파일을 업로드 하여 RAG 시스템에 즉시 반영	의사	하	

요구사항정의서

페이지명	분석 페이지 (척추 협착증)	작성자	김담현	
ID	기능명	내용상세	사용자	우선순위	비고
REQ-UI-01	멀티모달 데이터 입력 폼	환자의 척추 MRI 이미지(JPG/PNG) 업로드 기능- 증상, 치료 이력, Red Flag(대소변 장애 등) 체크용 문진표 UI- 파일과 JSON 데이터를 FormData로 병합하여 서버 전송	의사	상	필수 입력값 누락 시 '분석 시작' 버튼 비활성화
REQ-UI-02	진단 결과 대시보드	Vision AI 수치 및 LLM의 최종 리포트를 화면에 렌더링 GitHub 다크 테마 기반의 고대비 UI/UX 적용 '새로운 영상 분석하기' 클릭 시 폼 초기화	의사	상	final_report 텍스트의 줄바꿈 및 포맷 유지 렌더링
REQ-UI-03	장애 대비 Fallback UI	AI 서버 미응답 또는 통신 실패 시, 시스템 중단 없이 템플릿을 렌더링하여 사용자 경험 보호	의사	중	프론트엔드 단 예외 처리
REQ-API-01	Spring-FastAPI 브릿지 통신	React에서 넘어온 MultipartFile과 문진표 String을 FastAPI 규격에 맞춰 라우팅 ByteArrayResource를 활용해 파일명 명시적 매핑	의사	상	통신 규격 일치화 (직렬화/역직렬화)
REQ-DB-01	진단 이력 영속화 (저장)	통합 분석이 완료된 후 원본 파일명, 추출된 Vision JSON 수치, 임상 소견 텍스트를 PostgreSQL 데이터베이스에 INSERT	의사	상	JSONB 데이터 타입 활용 (dl_vision_result, clinical_symptoms)
REQ-AI-01	Vision AI 전처리 및 추론	입력된 MRI를 224x224 리사이즈 및 정규화하여 Tensor로 변환 ResNet18 커스텀 모델을 통해 3가지 질환 x 3단계 중증도(Normal/Moderate/Severe) 확률 도출	의사	상	결과값은 JSON 형태로 직렬화하여 반환
REQ-AI-02	Red Flag 최우선 감지	문진 데이터에 치명적 응급 징후(대소변 장애 등)가 포함된 경우, LLM이 이를 최우선 위험도로 인지하고 '즉각 수술 검토' 권고 생성	의사	상	프롬프트 엔지니어링 필수 반영
REQ-AI-03	RAG 기반 전문 지식 검색	LlamaParse로 적재된 NASS(북미척추학회) 가이드라인을 pgvector 벡터 DB에서 검색하여 전문의 소견(Expert Node)의 근거 자료로 활용	의사	상	데이터 적재 시 API Throttle 및 로컬 캐싱 적용
REQ-AI-04	멀티 에이전트 파이프라인	LangGraph를 활용한 3노드(Analyzer -> Expert -> Communicator) 순차 실행 워크플로우 구축- 영상 데이터(정량)와 문진 데이터(정성)의 교차 검증 및 환자 친화적 번역	의사	상	AgentState를 통한 노드 간 데이터 유실 방지

요구사항정의서

페이지명	분석 페이지 (피부질환 AI)	작성자	김민수	
ID	**기능명**	**요구사항 상세**	**사용자**	**우선 순위**	**비고**
SKN-01	이미지 업로드	의사가 환부 사진을 선택하거나 드래그하여 업로드한다.	의사	상	파일 크기 제한(10MB) 및 전처리
SKN-02	분석 시작 버튼	이미지 업로드 후 'AI 분석 시작' 버튼이 활성화되어야 한다	의사	상	Spring ↔ FastAPI 통신 유도
SKN-03	로딩 및 비활성화	분석 중 로딩 스피너를 표시하고 진단 버튼을 비활성화한다 (중복 요청 방지)	의사	중	
SKN-04	AI 분석 결과 출력	분석 완료 후 질환명(국문/영문)과 판독 신뢰도(%)를 화면에 표시한다	의사	상	용어 관리 시스템 연동
SKN-05	XAI 시각화 제공	Grad-CAM 이미지를 원본 이미지 위에 오버레이하여 병변 위치를 강조한다	의사	상	진단 근거 제시용 (핵심 차별점)
SKN-06	에러 처리	분석 실패 시 "이미지 형식이 부적절하거나 서버 통신에 실패했습니다" 토스트 표시	의사	중	
SKN-07	진단 이력 저장	판독 완료 시 환자 ID와 매핑하여 진단 결과 및 시각화 경로를 DB에 자동 저장한다	의사	상	통합 DB 담당과 협의
SKN-08	다중 환부 부석	동일 환자 내 여러 환부 이미지를 순차적으로 업로드하여 통합 리포트를 생성한다	의사	상	
SKN-09	이미지 퀄리티 체크	업로드 시 이미지의 선명도와 밝기를 자동 체크하여 분석 부적합 시 경고 메시지를 띄운다	의사	중	

요구사항정의서

	페이지명	분석 페이지 (뇌종양)	작성자	변운조	
ID	기능명	요구사항 상세	사용자	우선순위	비고
BTM-01	의사 로그인	의사 계정(ID/PW)으로 로그인하고 JWT 토큰 기반 세션을 유지한다	의사	상	Spring Security 연동
BTM-02	환자 목록 조회	로그인한 의사가 담당 환자 목록을 검색/페이징하여 조회한다	의사	상	이름/차트번호 검색 지원
BTM-03	환자 선택	목록에서 환자를 선택하면 해당 환자의 진단 화면으로 진입한다	의사	상	환자 ID 세션 저장
BTM-04	MRI 파일 업로드	NIfTI(.nii, .nii.gz) 또는 DICOM 파일을 드래그/선택하여 업로드한다	의사	상	파일 크기 제한(500MB)
BTM-05	파일 유효성 검증	업로드 파일의 확장자, 헤더(Affine, 차원), 해상도를 자동 검증한다	의사	중	부적합 파일 차단 및 안내
BTM-06	분석 시작 버튼	업로드·검증 완료 후 'AI 진단 시작' 버튼이 활성화된다	의사	상	
BTM-07	로딩 및 비활성화	분석 중 진행률 스피너 표시, 중복 요청 방지를 위해 버튼 비활성화	의사	중	3D 추론 시간 고려
BTM-08	AI 진단 결과 출력	뇌종양 유무 + 종양 유형(Glioma/Meningioma/Pituitary/None) + 신뢰도(%)를	의사	상	
BTM-09	Grad-CAM 시각화	MRI 슬라이스 위에 히트맵을 오버레이하여 병변 의심 영역을 강조한다	의사	상	축방향/관상면/시상면 3축 지원
BTM-10	3D 슬라이스 뷰어	업로드된 NIfTI 파일을 슬라이더/스크롤로 슬라이스별 탐색한다	의사	상	
BTM-11	RAG 기반 리포트 생성	진단 결과 + 의학 문헌 Vector DB 기반으로 판독 소견문을 자동 생성한다	의사	상	
BTM-12	리포트 편집/확정	자동 생성 리포트를 의사가 검토·수정한 뒤 '확정' 버튼으로 최종 저장한다	의사	상	
BTM-13	진단 이력 DB 저장	환자 ID, 원본 영상 경로, 진단 결과, Grad-CAM, 확정 리포트를 DB에 저장한다	의사	상	통합 DB 담당과 스키마 협의
BTM-14	에러 처리	분석 실패 시 사유별 메시지("파일 손상", "서버 통신 실패", "모델 추론 오류") 토스트	의사	중	

요구사항정의서

	페이지명	분석 페이지(안과 AI)	작성자	홍승현	
요구사항 ID	요구사항명	상세 설명	사용자	우선순위	비고
OPH-01	환자 선택 및 대시보드 진입	담당 환자의 URL(/patients/:id/eye-disease) 진입 시 안과 전용 진단 대시보드로 자동 로드.	의사	상	환자 ID는 URL 파라미터(:id)로 수신. 환자 목록·사이드바는 공통 영역
OPH-02	안저 이미지 업로드	1. 환자의 안저(Fundus) 이미지를 드래그 앤 드롭으로 업로드. 2. 지원 형식: JPG, PNG, TIFF / 최대 20MB. 3. OCT 이미지 지원은 시간 여유 시 추가 예정.	의사	상	단일 파일. OCT는 추후 구현
OPH-03	1차 임상 소견 입력	의사가 환자의 현재 증상(시야 흐림, 비문증 등)을 텍스트로 입력할 수 있는 텍스트 에어리어를 제공.	의사	상	API 파라미터(clinical_note) 존재. 입력 UI 추후 추가
OPH-04	이미지 유효성 자동 검증	업로드 이미지에 9단계 Fail-Fast 검증 수행. (파일크기, 형식, 디코딩, 해상도, 종횡비, 안저영역, Red채널, 밝기, Laplacian 블러)	시스템	중	Laplacian 필터 최전방 배치. 백엔드 SSE 결과 기반
OPH-05	AI 분석 버튼 및 취소	1. 'AI 통합 분석 시작' 버튼 활성화 및 중복 클릭 방지. 2. 분석 중 즉시 중단 가능한 '분석 취소(Abort)' 버튼 제공. 3. 법적 면책 조항 상시 노출.	의사	상	AbortController 구현. AiDisclaimer 컴포넌트
OPH-06	DL 1차 진단 결과 출력	지식 증류(KD) 모델의 결과인 5개 질환(정상, 녹내장, 백내장, 당막, 황반변성) 유무와 신뢰도를 시각적 게이지로 출력.	의사	상	EfficientNet-B0(제자) 모델 사용. 추론: ONNX / GradCAM: .pth 분리
OPH-07	위험/응급 강제 알림	DL 결과가 특정 조건(Stage 3 이상, 고신뢰도+중증, 고령+당뇨병합 등) 충족 시 화면 전체를 덮는 응급 모달 강제 표시.	의사	상	3가지 복합 조건 적용. 응급 레벨 0~3 단계 표시
OPH-08	추론 진행률 표시	이미지 분석 직후 백엔드 워크플로우 진행 상황을 7단계 스텝으로 표시. (업로드→품질검증→AI분석→진단완료→시각화→문헌검색→소견서)	의사	중	SSE 이벤트 기반 AnalysisStepper 컴포넌트
OPH-09	소견서 스트리밍 (SSE)	완성된 임상 소견서를 실시간 타자 효과로 렌더링. 마크다운 형식으로 6개 항목(진단요약/중증도/치료방향 등) 포함.	의사	상	SSE report_chunk 이벤트. 타자기 효과(30ms/15자)
OPH-10	근거 문헌 인용	생성된 리포트 하단에 Hybrid RAG 및 Graph RAG를 통해 발췌된 AAO 가이드라인 근거 문헌을 인용 카드로 표시.	의사	상	Hybrid RAG + Graph RAG 통합. 원문 팝업은 추후 예정
OPH-11	통합 DB 저장	환자 ID, 진단 결과 수치, 소견서 텍스트를 PostgreSQL에 저장.	시스템	상	Spring Boot SH 패키지 (EyeDiagnosis 엔티티)
OPH-12	에러 및 예외 처리	모델 서버 통신 실패, 이미지 품질 불량 등 에러 발생 시 사유별 친화적 메시지 표시.	시스템	중	표준 에러 코드(IMAGE_QUALITY_LOW 등) 정의
OPH-13	GradCAM 히트맵 시각화	AI 모델이 주목한 병변 영역을 원본 이미지 위에 오버레이. 나란히/토글/슬라이더 3가지 뷰 모드 제공.	의사	상	GradCAM++ 적용. Base64 인코딩 SSE 전송

기능 명세서

표10

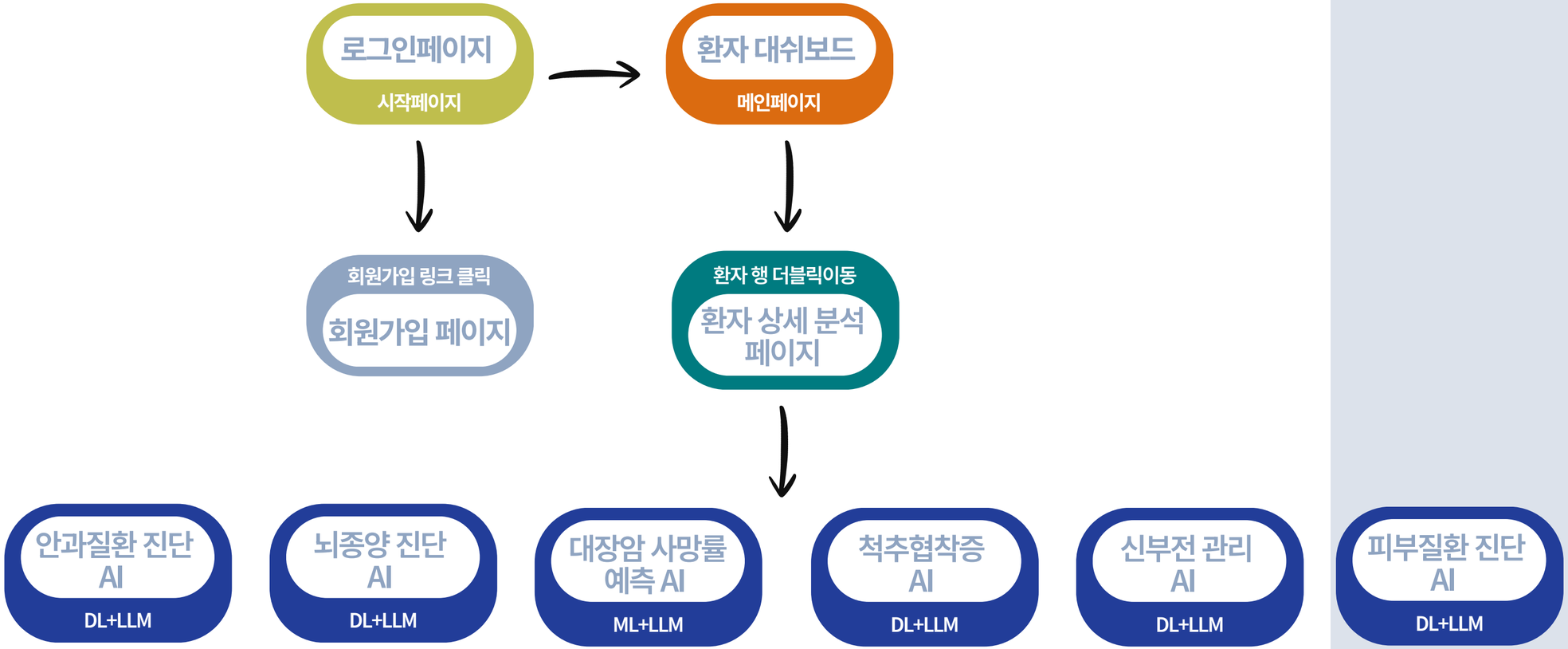
# 번호	Depth1	Depth2	기능 설명
1	**사용자 관리**	로그인	의사 면허 번호 기반 로그인 및 JWT 토큰 발급
2		로그아웃	세션 종료 및 로컬 토큰 파기
3	**환자 대시보드**	통합 환자 검색	이름 및 환자 번호를 활용한 데이터 통합 검색
4		환자 목록 조회	등록된 전체 환자 명단 및 기본 인적 사항 열람
5		환자 상세 정보	선택된 환자의 과거 진료 이력 및 분석 리포트 요약
6	**AI 진단 (피부)**	환부 분석	이미지 업로드 및 AI 기반 피부 질환 분류 (SKN)
7		진단 근거(XAI)	Grad-CAM을 활용한 병변 부위 시각화 강조
8	**AI 진단 (신장)**	임상 지표 분석	24개 지표 입력 및 신부전 단계(1~5) 진단
9		의학 가이드라인	RAG 기반 맞춤형 가이드라인 자동 검색 및 추천
10	**AI 진단 (척추)**	MRI 분석	ResNet18 기반 모델이 이미지를 3x3 매트릭스(질환별/중증도별) 확률값으로 변환
11		통합 진단 에이전트	LangGraph 기반 영상 수치와 문진표 데이터를 비교 분석, 소견서 작성, 환자용 리포트 생성
12	**AI 진단 (대장)**	위험도 분류	결측치 처리 후 대장암 저/중/고 위험군 예측
13		소견서 자동 생성	LLM을 활용한 분석 결과의 자연어 요약 제공
14	**AI 진단 (뇌종양)**	뇌종양 분할	MRI 영상 내 종양 영역 자동 마스킹 및 위치 식별
15		종양 유형 분류	종양의 양성 / 악성 판별 및 확률 출력
16	**AI 진단 (안과)**	안저 질환 탐지	안저 카메라 영상 기반 3대 질환(백내장, 녹내장 등) 탐지
17		병변 위치 강조	시신경 유두 및 황반 변성 의심 부위 시각화 제공
18	**시스템 관리**	모델 상태 모니터링	7개 AI 서버의 실시간 가동 상태 및 부하 표시
19		진단 이력 관리	의사별/전체 환자의 AI 판독 결과 DB 저장 및 관리

정책 정의서

정책정의서

	정책명	세부 항목	소개	정책 정의
1	회원가입 정책	의사 자격 검증	시스템 사용자의 자격 요건	의료 면허 번호 검증을 통과한 의료진만 가입 가능
2	서비스 이용 정책	AI 진단 책임 범위	AI 진단의 역할과 한계	AI 진단은 참고용이며, 최종 진단과 처방은 반드시 의사가 결정
3	데이터 프라이버시 정책	환자 정보 익명화	환자 개인정보 보호 방식	환자 식별자는 암호화 처리하여 진료 목적 외 환자의 프라이버시 보장
4	사용자 권리 정책	진단 기록 관리	사용자의 데이터 통제권 보장	본인 작성 진단 기록만 조회 및 삭제 가능, 삭제 시 30일 후 영구 파기
5	AI 데이터 활용 정책	외부 API 데이터 전송	LLM 등 외부 API 호출 시 데이터 처리	환자 식별 정보 제거 후 임상 수치만 전송, 출처 없는 응답 차단
6	보고서 저장 정책	종합 소견서 관리	진단 보고서 저장과 수정	의사 명시적 저장시에만 DB 기록, 저장 후 수정 불가(변경 시 새 버전)

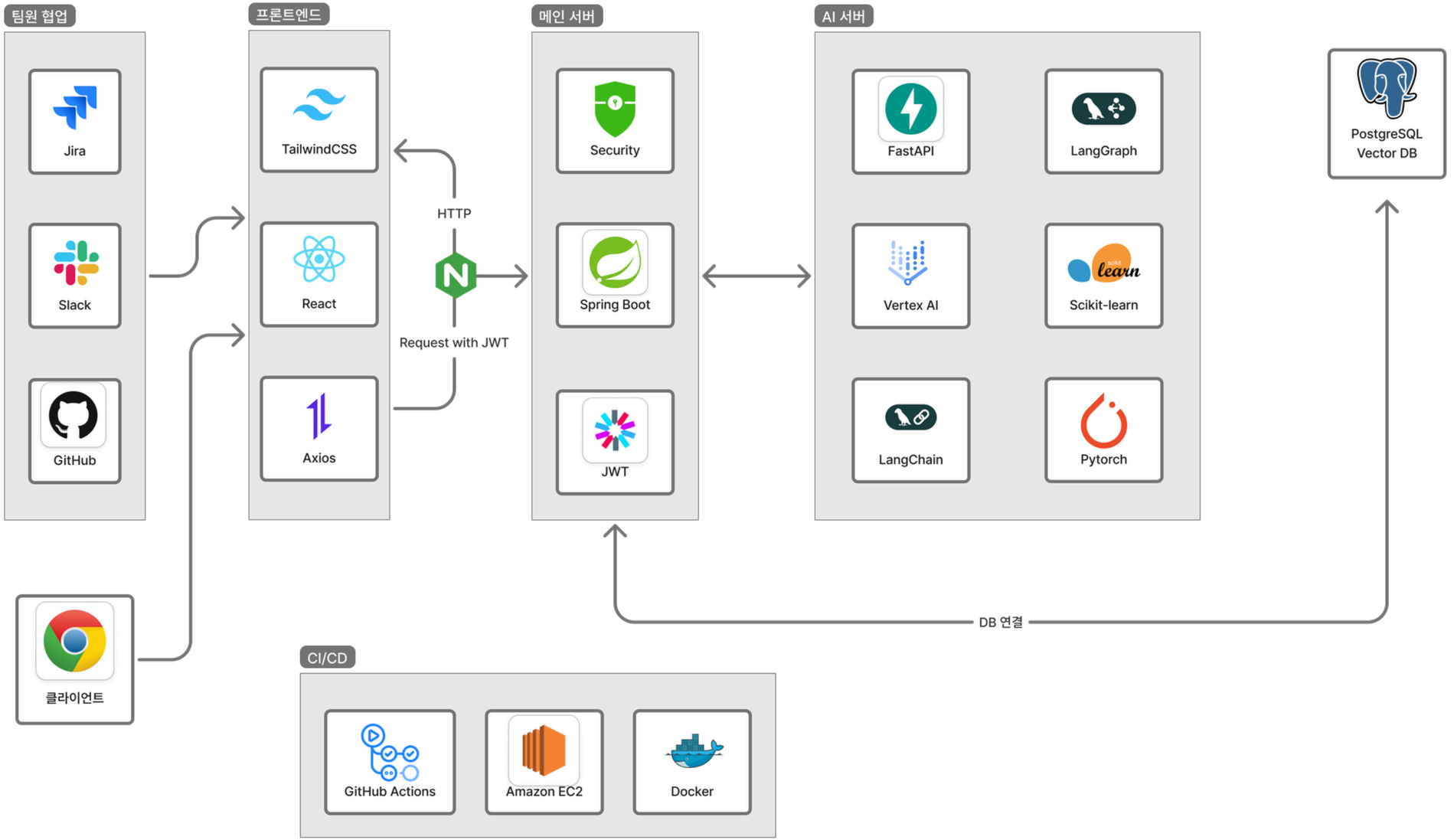
메뉴구조도


로그인페이지
시작페이지
환자 대쉬보드
메인페이지
회원가입 링크 클릭
회원가입 페이지
환자 행 더블릭이동
환자 상세 분석 페이지
안과질환 진단 AI
DL+LLM
뇌종양 진단 AI
DL+LLM
대장암 사망률 예측 AI
ML+LLM
척추협착증 AI
DL+LLM
신부전 관리 AI
DL+LLM
피부질환 진단 AI
DL+LLM

프로젝트 설계

순서	시스템 아키텍쳐, 시스템 흐름도, 네트워크 구조, ERD 설계

아키텍처



라이브러리

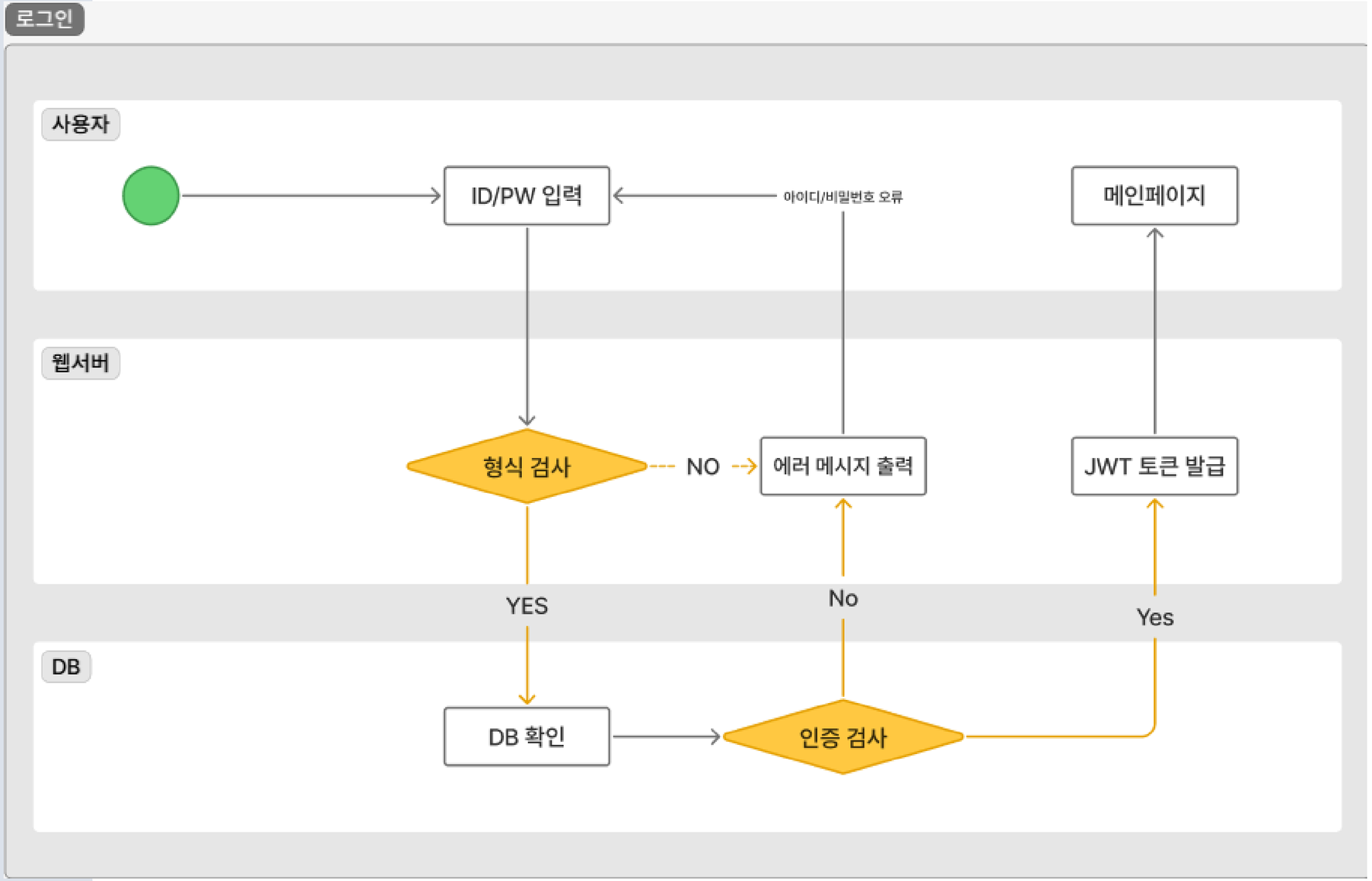
[JAVA / SPRING BOOT]
- FASTAPI
- UVICORN[STANDARD]
- ULTRALYTICS
- TORCH
- TORCHVISION
- LANGCHAIN
- LANGGRAPH
- PGVECTOR
- PSYCOPG2-BINARY

[JAVA / SPRING BOOT]
- SPRING WEB
- SPRING REACTIVE WEB **
- SPRING DATA JPA
- POSTGRESQL DRIVER

[NODE.JS / REACT]
- AXIOS
- ZUSTAND
- TAILWINDCSS
- LUCIDE-REACT
- REACT-MARKDOWN
- @MICROSOFT/FETCH-EVENT-SOURC
-

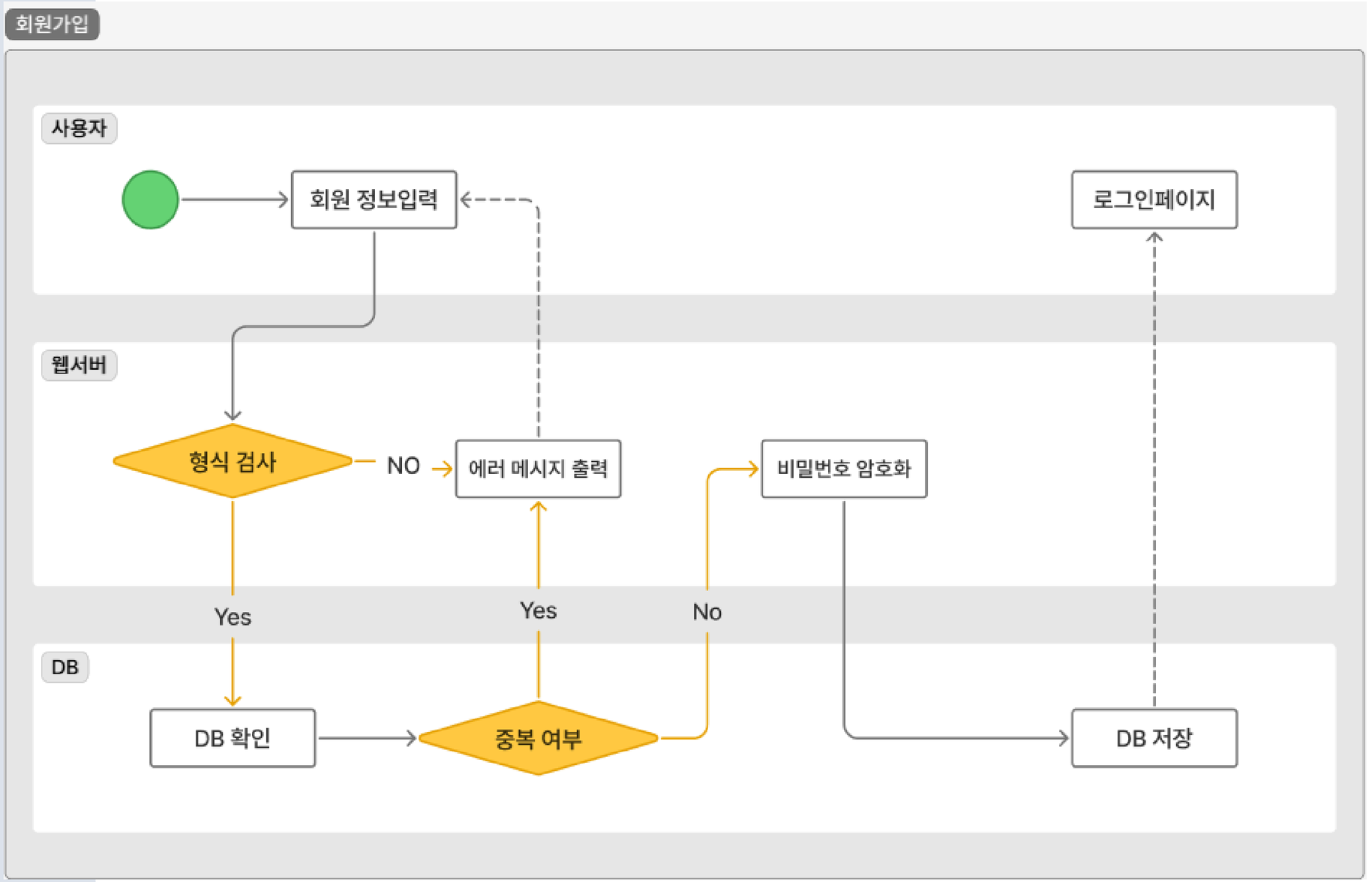
시스템흐름도

로그인


로그인
사용자
ID/PW 입력
아이디/비밀번호 오류
메인페이지
웹서버
형식 검사
NO
에러 메시지 출력
JWT 토큰 발급
YES
No
Yes
DB
DB 확인
인증 검사

시스템흐름도

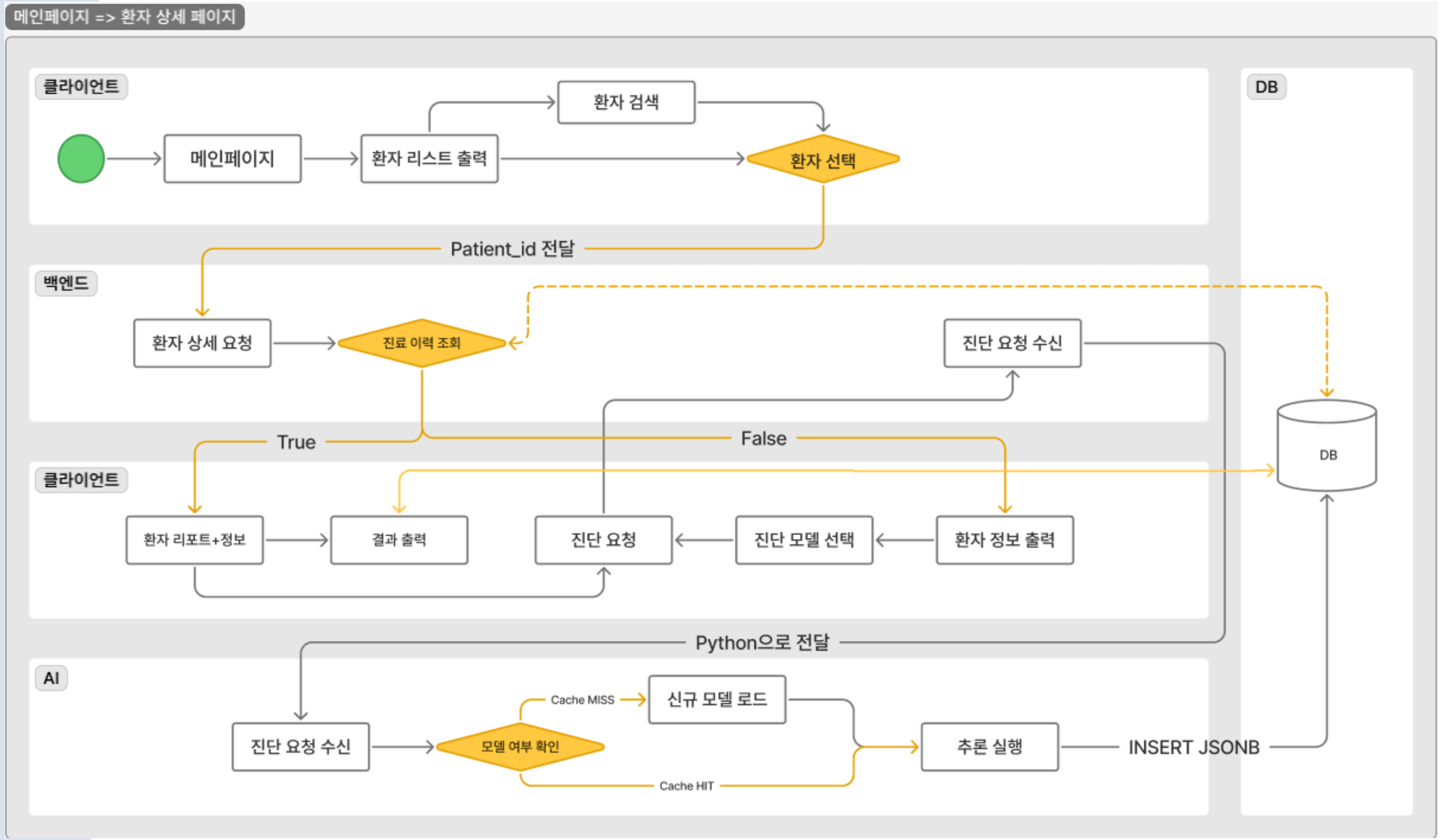
회원가입


회원가입
사용자
회원 정보입력
로그인페이지
웹서버
형식 검사
NO
에러 메시지 출력
비밀번호 암호화
Yes
Yes
No
DB
DB 확인
중복 여부
DB 저장

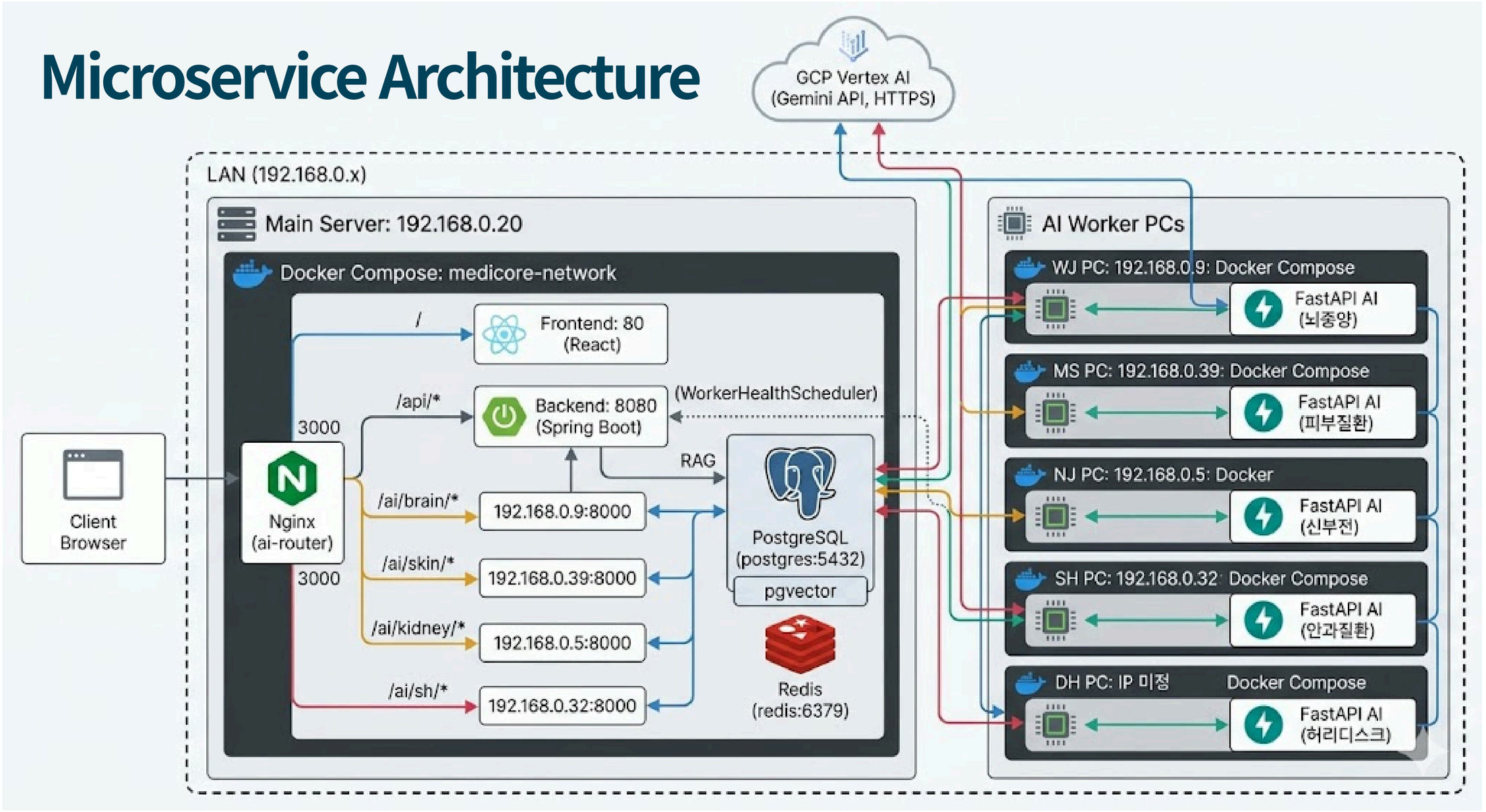
시스템흐름도

핵심 기능 흐름도

메인페이지 => 환자 상세 페이지



네트워크구조

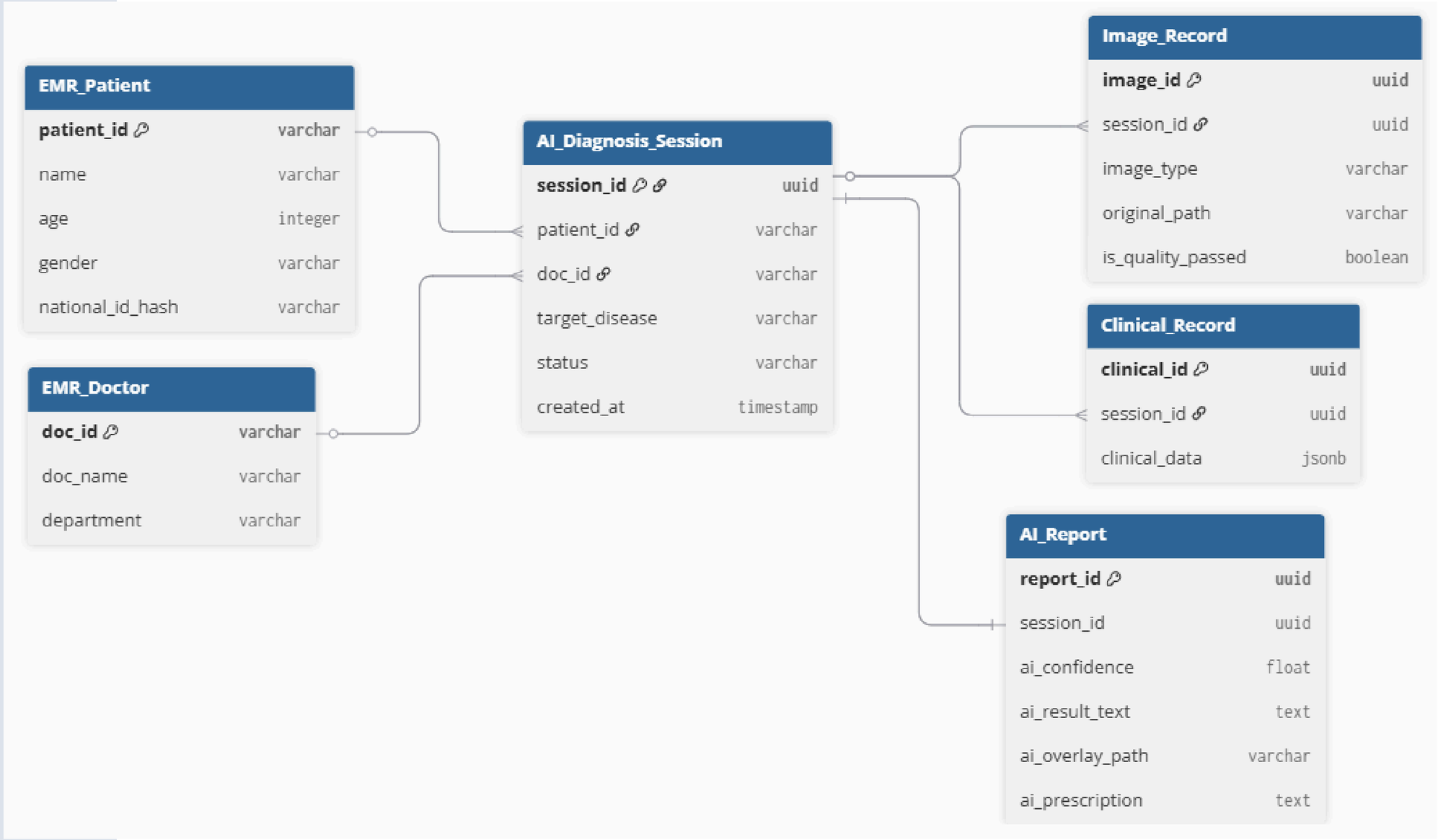

Microservice Architecture
GCP Vertex AI
(Gemini API, HTTPS)
LAN (192.168.0.x)
Main Server: 192.168.0.20
Docker Compose: medicore-network
Client
Browser
Nginx
(ai-router)
3000
3000
/
Frontend: 80
(React)
/api/*
Backend: 8080
(Spring Boot)
(WorkerHealthScheduler)
RAG
/ai/brain/*
192.168.0.9:8000
/ai/skin/*
192.168.0.39:8000
/ai/kidney/*
192.168.0.5:8000
/ai/sh/*
192.168.0.32:8000
PostgreSQL
(postgres:5432)
pgvector
Redis
(redis:6379)
AI Worker PCs
WJ PC: 192.168.0.9: Docker Compose
FastAPI AI
(뇌종양)
MS PC: 192.168.0.39: Docker Compose
FastAPI AI
(피부질환)
NJ PC: 192.168.0.5: Docker
FastAPI AI
(신부전)
SH PC: 192.168.0.32: Docker Compose
FastAPI AI
(안과질환)
DH PC: IP 미정
Docker Compose
FastAPI AI
(허리디스크)

ERD 논리

EMR_환자
환자_차트번호 varchar
이름 varchar
나이 integer
성별 varchar
주민번호_암호화 varchar
EMR_의사
의사_사번 varchar
이름 varchar
소속과 varchar
AI_진단_세션
세션_ID uuid
환자_차트번호 varchar
의사_사번 varchar
대상_질환 varchar
진행_상태 varchar
생성_일시 timestamp
영상_입력_기록
영상_ID uuid
세션_ID uuid
영상_포맷 varchar
원본파일_경로 varchar
화질_검증통과 boolean
임상_수치_기록
수치_ID uuid
세션_ID uuid
임상_데이터 jsonb
AI_분석_리포트
리포트_ID uuid
세션_ID uuid
AI_신뢰도 float
분석_결과 text
시각화_경로 varchar
AI_요약처방 text

작성자 : 홍승현

ERD 물리



화면 구현

순서 로그인, 회원가입, 메인페이지, 상세 분석 페이지, 진단페이지

화면 구현

페이지명 - login.js 담당자 - 변운조

Description

1	의료진이 MEDICORE 시스템에 접근하기 위한 사용자 인증 화면

Action

	버튼명	기능
1	로그인	입력된 아이디(또는 사번)와 비밀번호를 서버로 전송하여 인증을 요청
2	회원가입	신규 의료진 등록을 위한 화면으로 이동
3	아이디 / 비밀번호 찾기	계정 복구 및 비밀번호 재설정 페이지로 이동

Validation

1	아이디 또는 비밀번호 입력란이 비어있는 상태로 로그인 시도 시, “ID를 입력해 주세요 or 비밀번호를 입력해 주세요” 알림창
2	정보가 틀렸을 경우 보안을 위해 어느 항목이 틀렸는지 특정하지 않고 통합 메시지를 띄움

화면 구현

페이지명 - register.js 담당자 - 변운조

Description

1	신규 의료진이 시스템 사용 권한을 얻기 위해 계정을 생성하고 신원을 증명하는 화면

Action

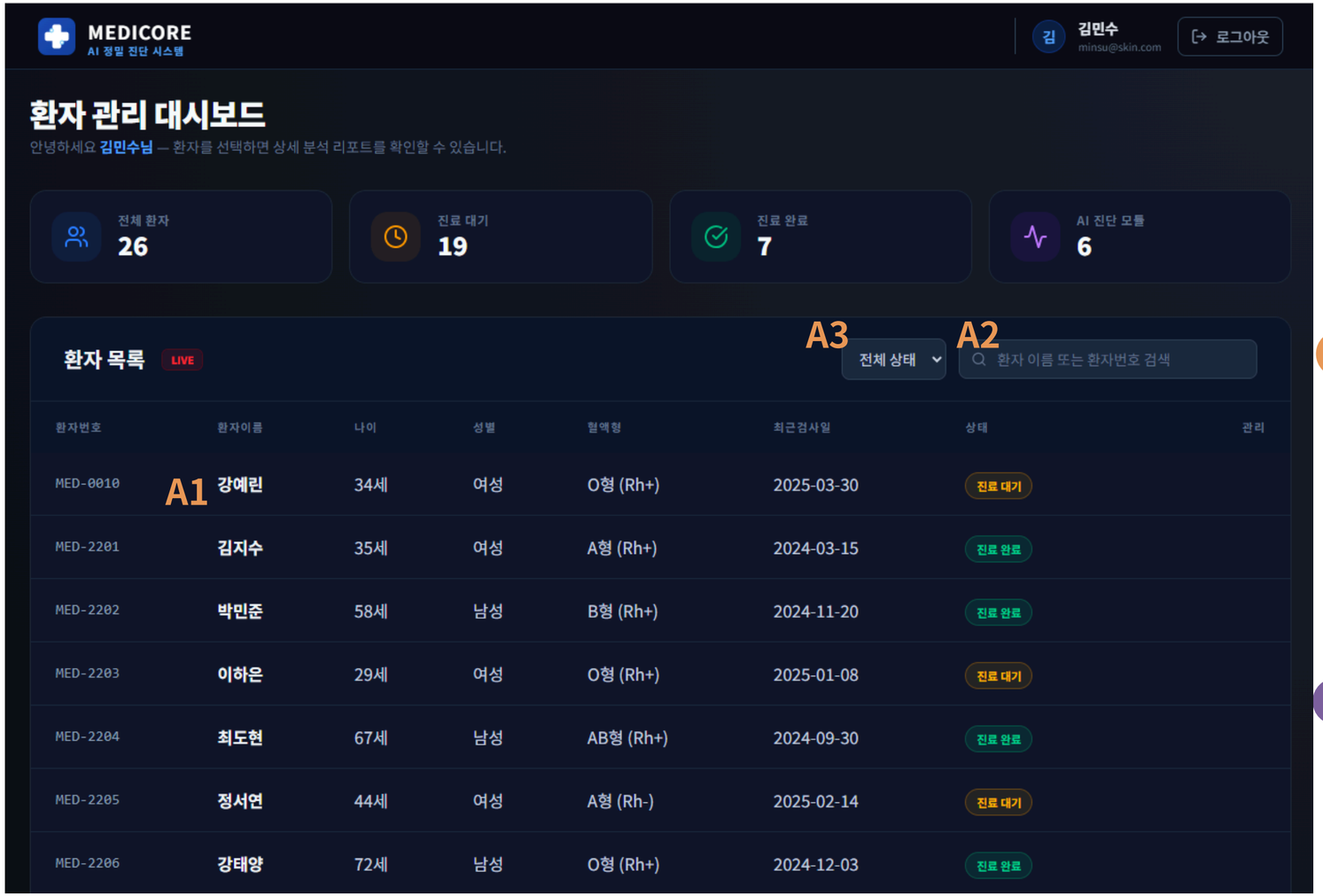
	버튼명	기능
1	병원코드 입력	병원 DB에 저장된 병원코드를 입력
2	주민등록 번호	병원 DB에 저장된 주민등록번호를 입력
3	회원가입	병원 DB에 저장된 데이터와 비교하여 일치하는지 확인

Validation

1	제휴병원, 사원번호, 주민번호를 기존 병원DB정보와 비교하여 일치하는 정보가 있을 경우만 회원가입을 할 수 있음
2	사원번호, 사용할ID 중복체크

화면 구현

페이지명 - main.js 담당자 - 김민수



Description

1	환자 정보를 리스트업하여 일괄적으로 확인 할수있는 화면
2	환자별로 진료 상태를 구별한다.
3	환자의 최신 정보 수정일 기준으로 정렬한다
4	환자의 최근 진료 내역을 시각적으로 보여준다.

Action

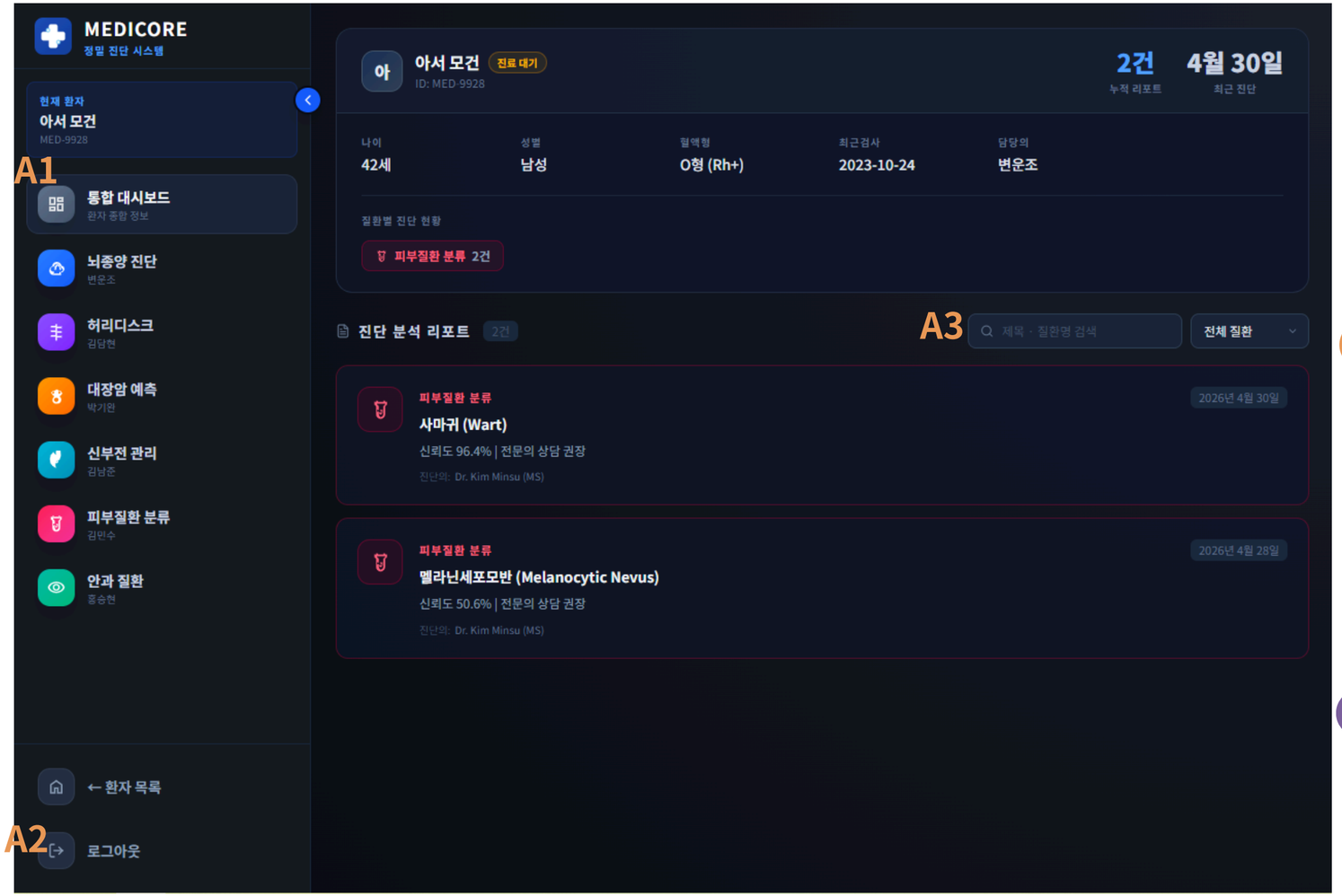
	버튼명	기능
1	환자 이름	환자 분석 페이지로 이동합니다.
2	검색 하기	환자명이나 환자 코드를 이용해서 검색한다.
3	필터	환자 진단명 이나 상태를 이용해서 필터링해서 본다

Validation

1	환자 리포트 작성후 환자상태 완료로 변경

화면 구현

페이지명 - detailpainter.js 담당자 - 김민수



Description

1	환자 상세 분석 리포트를 확인하는 페이지입니다.
2	진단 기능 별로 진단 후 통합 리포트가 대시보드에 저장됩니다.
3	환자의 프로필을 들어가서 해당 환자의 대한 진료를 진행합니다.
4	환자가 진료를 본 이력을 보여줍니다.

Action

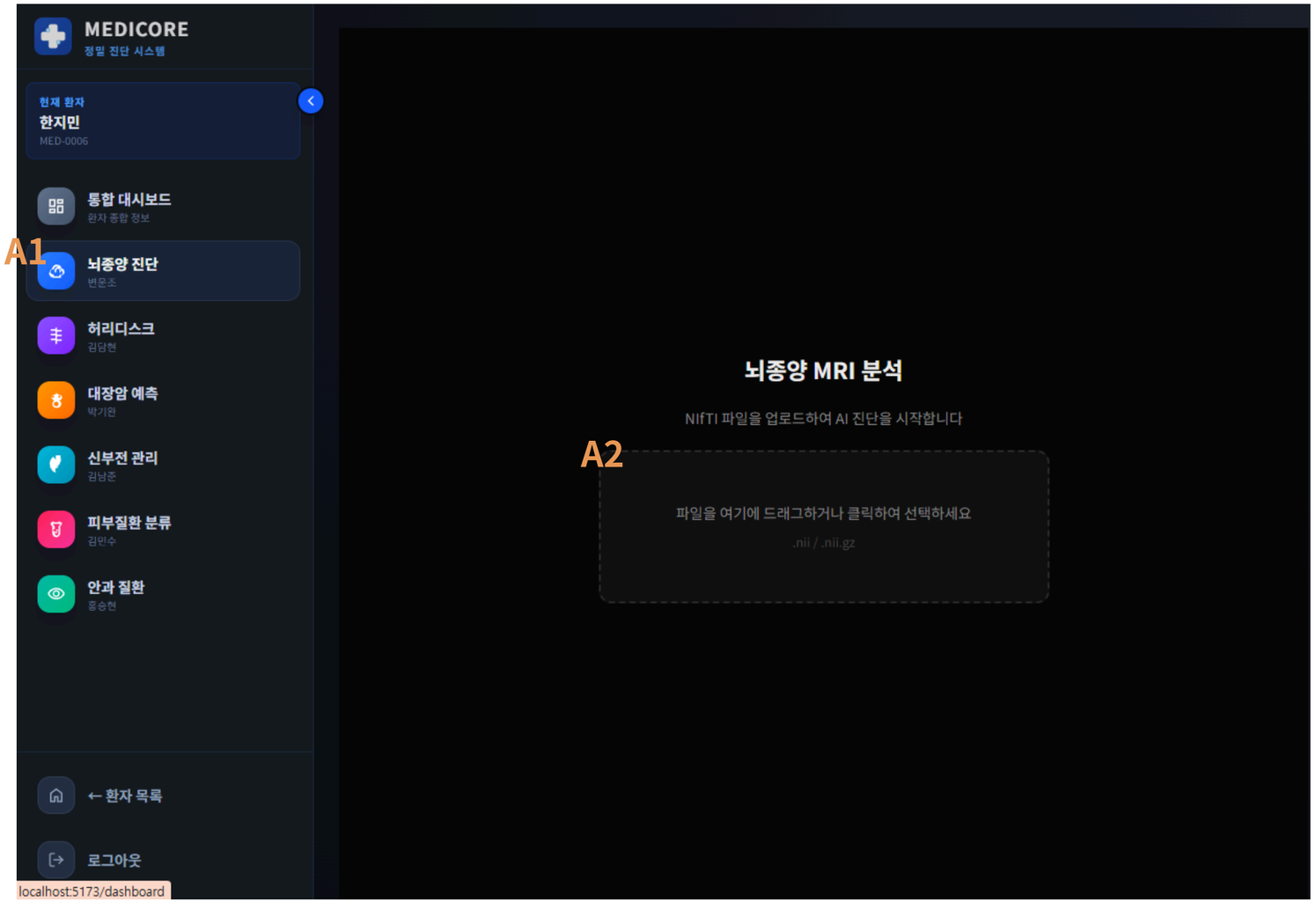
	버튼명	기능
1	진단 탭	해당하는 진단 기능으로 이동합니다.
2	로그아웃	웹사이트에서 로그아웃을 진행합니다.
3	검색하기	질환명을 검색하여 진단했던 기록을 확인합니다.

Validation

1	이전 데이터 비교 검증
2	EMR 환자 정보 일치

화면 구현

페이지명 - brain_aimodel.js 담당자 - 변운조



Description

1	좌측 사이드바에서 뇌종양 진단 버튼 클릭시 화면이 리다이렉트
2	NIFTI 파일을 드래그앤드롭 하면 NII 파일 형식을 검사 후 분석
3	화면을 클릭하면 파일탐색창이 생성되고 파일을 첨부 시 형식을 검사 후 분석

Action

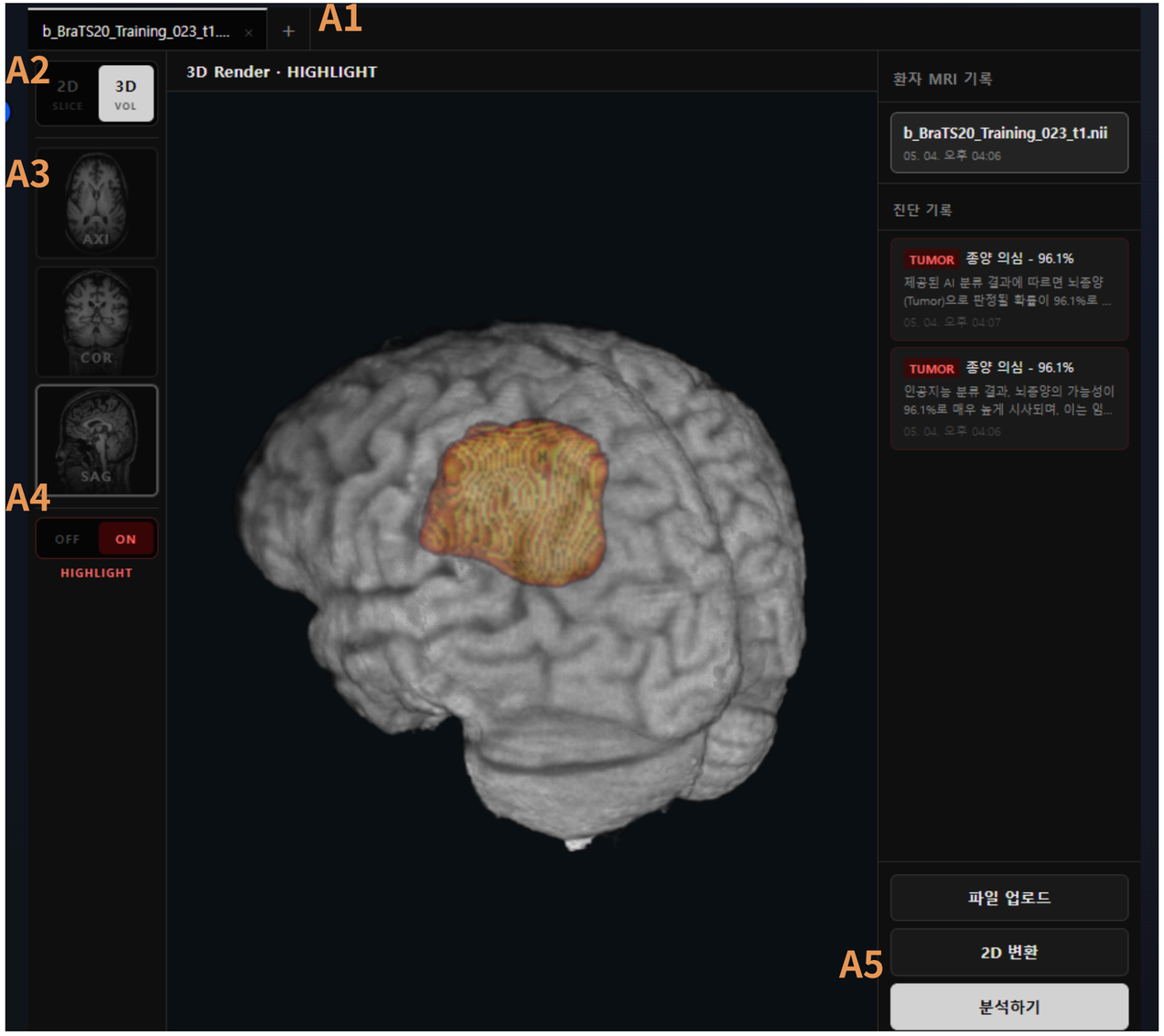
	버튼명	기능
1	뇌종양 진단	NIFTI 파일 로드화면 변경
2	파일탐색창	드래그앤드롭 또는 파일생성창 생성을 통해 파일을 첨부

Validation

1	유효성 검사를 통해 NII 파일만 로드 가능

화면 구현

페이지명 - brain_aimodel.js 담당자 - 변운조



Description

1	2D(3축 영상), 3D를 통해 병변의 입체적 위치를 정밀하게 판독
2	전체 MRI 슬라이스를 신속하게 추적하고 이동
3	AI가 전문적인 판독 소견서 초안을 스크롤 형태로 제공

Action

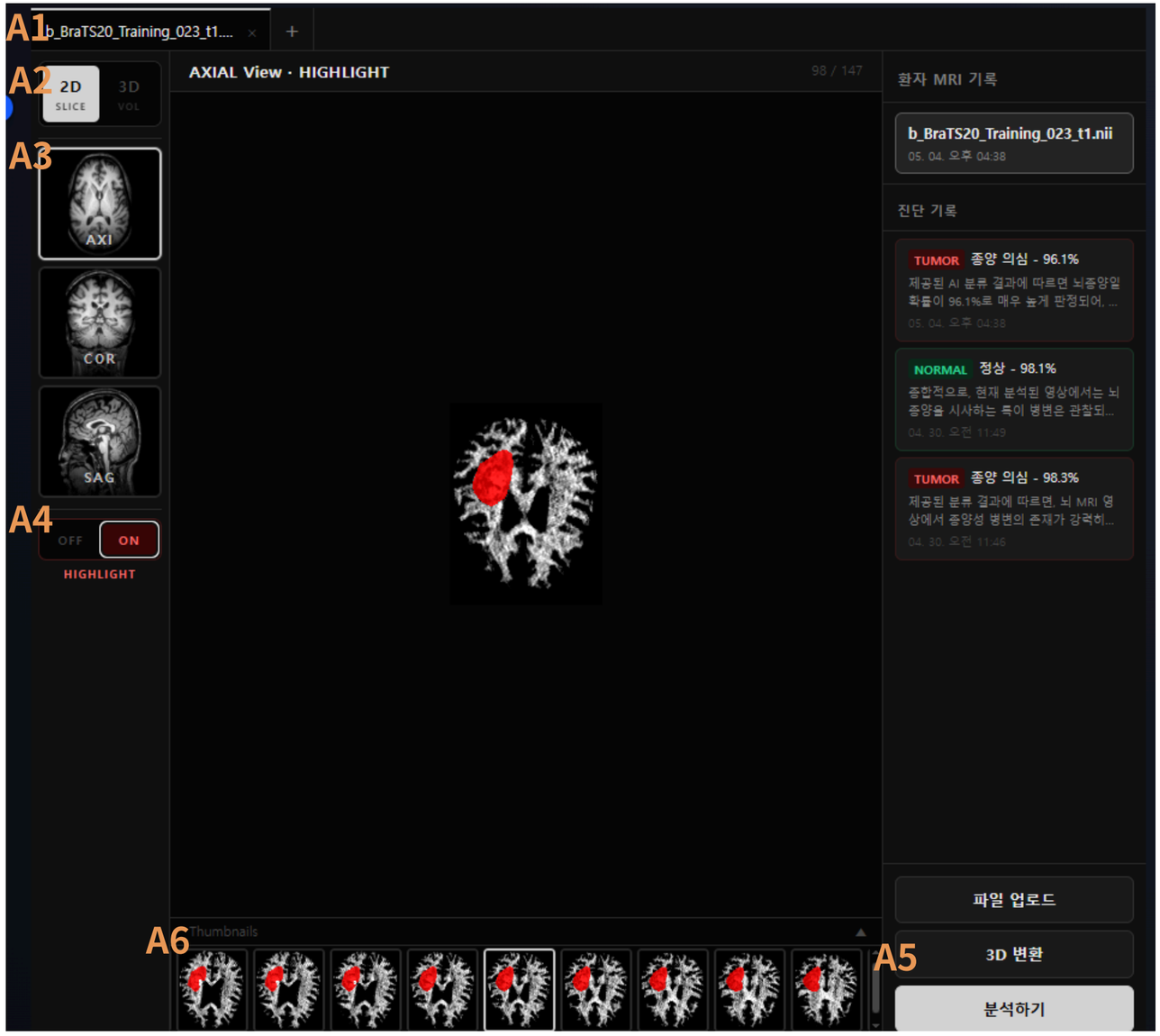
	버튼명	기능
1	탭 버튼	+ 버튼을 클릭하여 새로운 탭 활성화
2	2D/3D	화면에 랜더링할 2D/3D 방식 선택
3	2D 단면 별 뷰어	뇌 구조를 세 가지 해부학적 단면으로 전환
4	하이라이트	뇌종양 판별 시 활성화, on/off로 하이라이트
5	분석하기	NII 파일을 분석하고 LLM으로 리포트 작성

Validation

1	종양 확률 50% 이상 복셀만 3D 오버레이로 시각화

화면 구현

페이지명 - brain_aimodel.js 담당자 - 변운조



Description

1	2D(3축 영상), 3D를 통해 병변의 입체적 위치를 정밀하게 판독
2	전체 MRI 슬라이스를 신속하게 추적하고 이동
3	AI가 전문적인 판독 소견서 초안을 스크롤 형태로 제공

Action

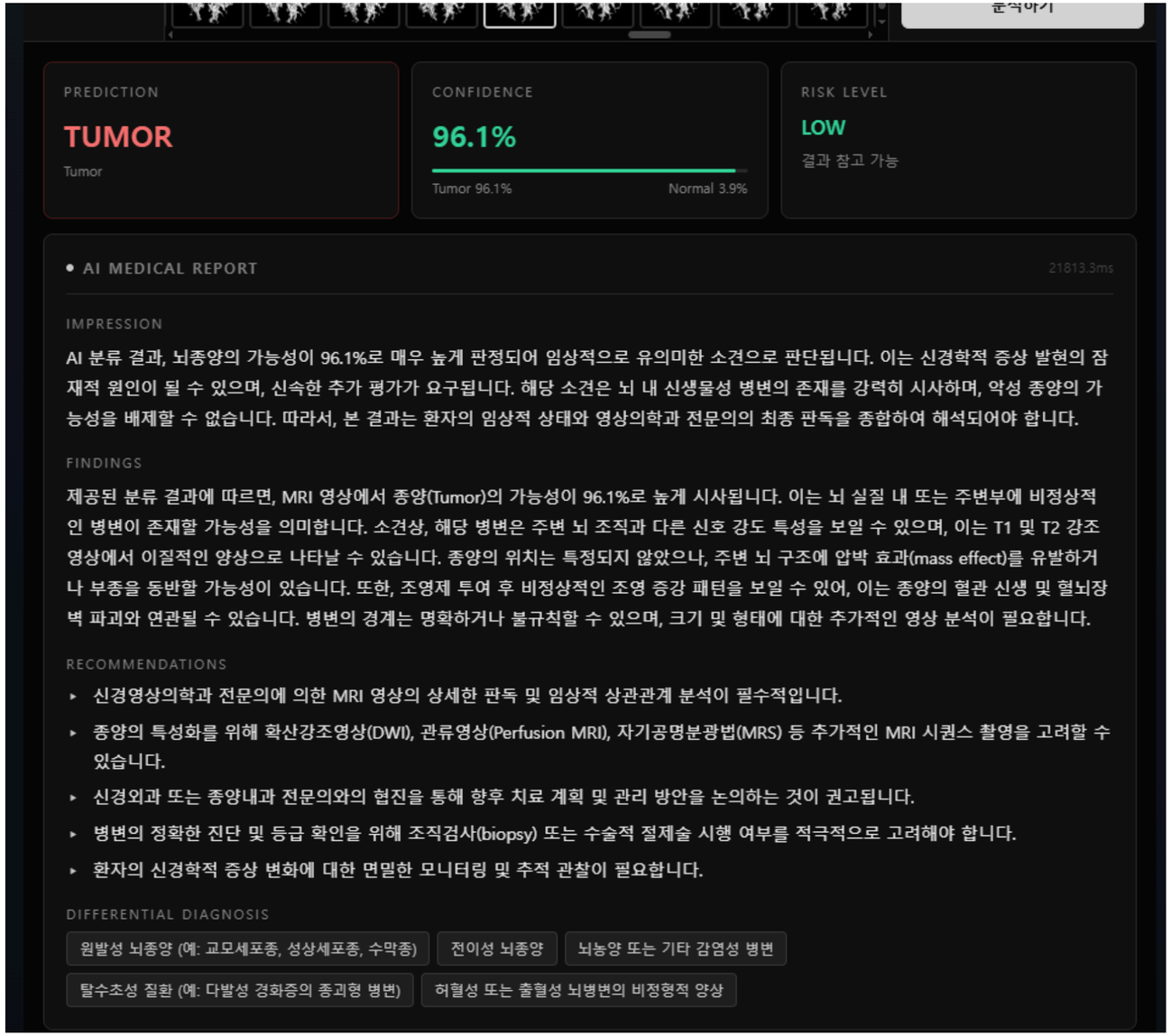
	버튼명	기능
1	탭 버튼	+ 버튼을 클릭하여 새로운 탭 활성화
2	2D/3D	화면에 랜더링할 2D/3D 방식 선택
3	2D 단면 별 뷰어	뇌 구조를 세 가지 해부학적 단면으로 전환
4	하이라이트	뇌종양 판별 시 활성화, on/off로 하이라이트
5	분석하기	NII 파일을 분석하고 LLM으로 리포트 작성
6	슬라이드	하단의 이미지를 클릭하거나 휠로 슬라이딩

Validation

1	확률 맵 기반 종양 영역 빨간색 오버레이 표시

화면 구현

페이지명 - brain_aimodel.js 담당자 - 변운조



Description

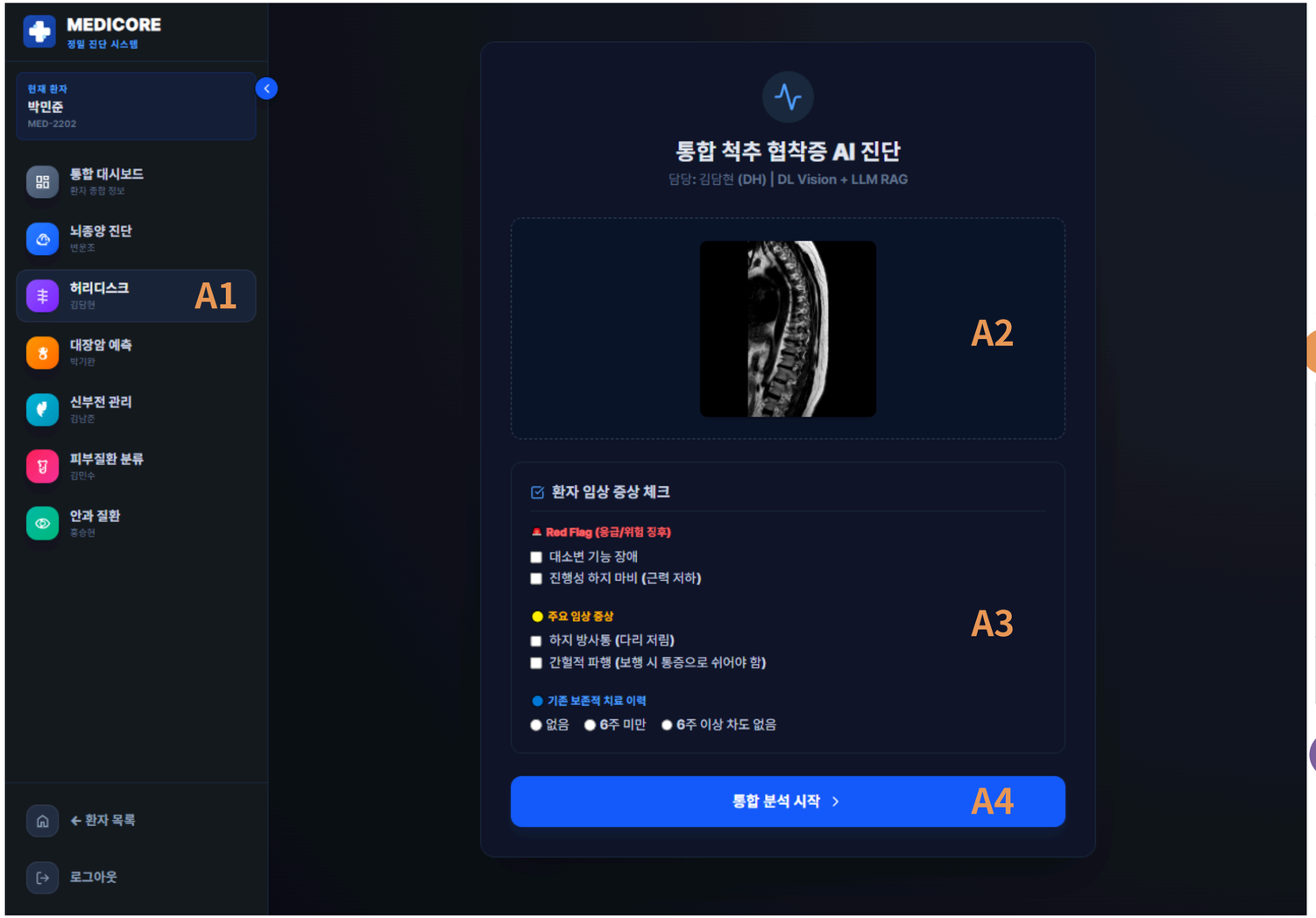
1	2D(3축 영상), 3D를 통해 병변의 입체적 위치를 정밀하게 판독
2	전체 MRI 슬라이스를 신속하게 추적하고 이동
3	AI가 전문적인 판독 소견서 초안을 스크롤 형태로 제공

Validation

1	pgvector 검색 결과를 프롬프트에 주입, findings/impression/recommendations 구조화 출력

화면 구현

페이지명 - Spine_aimedel.js 담당자 - 김담현



Description

1	환자의 척추 MRI(DL Vision) 데이터와 체크된 임상 증상 데이터 (LLM RAG)를 결합하여, 보다 정밀한 AI 진단 결과를 제공
2	MRI 영상 데이터 입력단
3	응급/위험 징후, 주요 증상, 치료이력 체크박스

Action

	버튼명	기능
1	네비게이션	척추 협착증 메뉴 진입
2	업로드	MRI 데이터 업로드
3	체크박스	환자 임상 증상 체크' 폼에서 증상 입력
4	분석하기	ai/spine/analyze API로 분석 요청 전송

Validation

1	파일 검증 : MRI 업로드 시 .dcm .nii .nii.gz .jpg .png 확장자만 ㅎ
2	상태 검증 : MRI 자료가 업로드 되지 않으면 분석 버튼 비활성화

화면 구현

페이지명 - report.js 담당자 - 김담현

Description

1	초기 환자 정보에 이상치를 표시해서 보여는 화면
2	탐지 DL 모델의 결과값을 보여주는 화면
3	분류 DL 모델의 결과값을 보여주는 화면
4	DL결과를 토대로 LLM이 처방을 내려주는 화면

Action

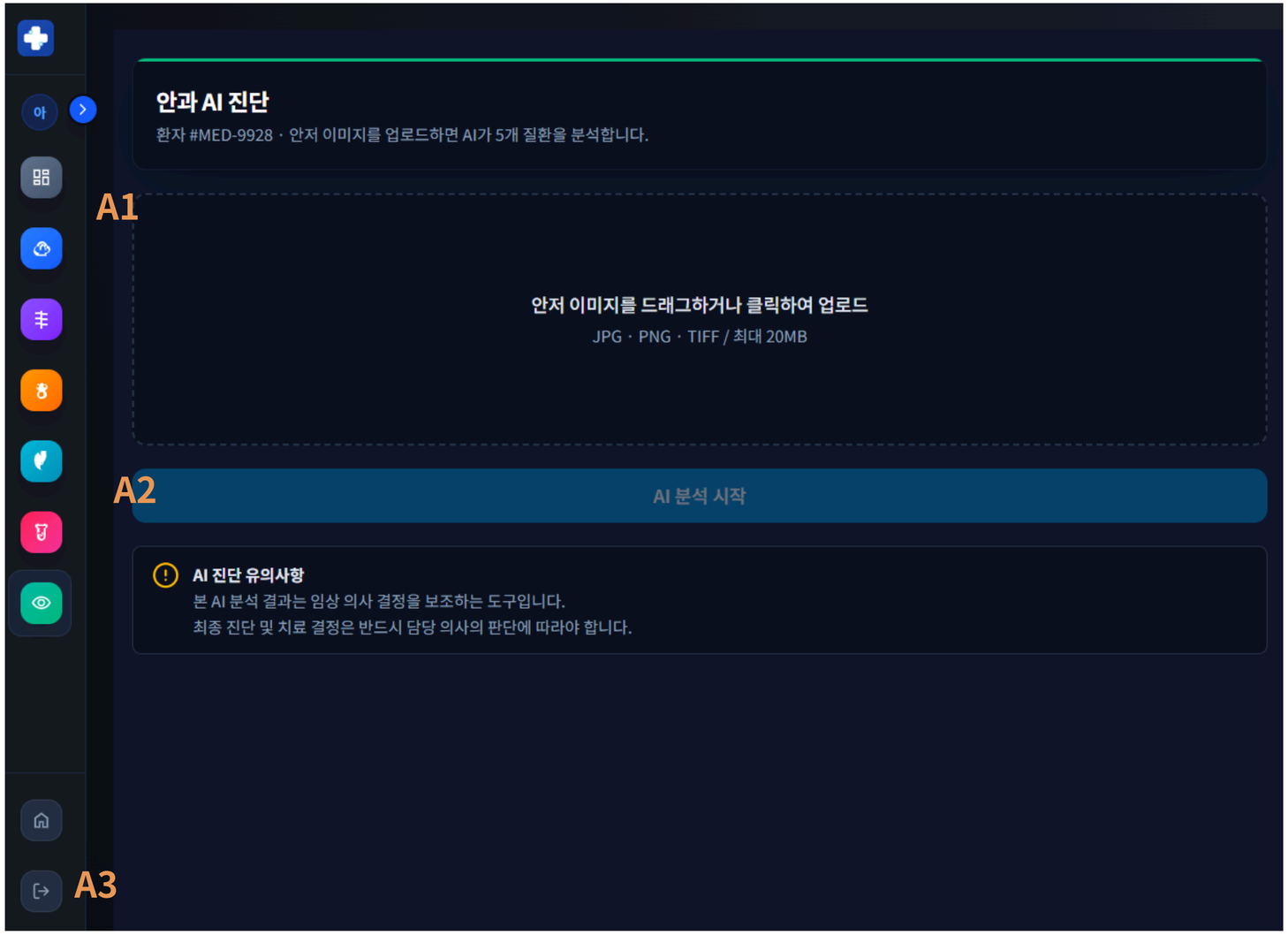
	버튼명	기능
1	뒤로가기	메인페이지(환자 목록페이지)로 이동
2	저장	분석 결과 값을 DB에 저장하는 버튼

Validation

1	DL모델의 신뢰도를 표시함
2	사용 청크가 포함된 문서의 제목을 표시함
3	청크조각중 몇번째 청크인지 표시함

화면 구현

페이지명 - eyes_main.js 담당자 - 홍승현



Description

1	안과 AI 진단 진입 화면으로, 환자 ID와 분석 안내 문구를 상단에 표시한다
2	드래그 앤 드롭 업로드 영역이 중앙에 위치하며 지원 포맷과 용량 제한을 안내한다
3	화면 하단에 AI 면책 조항이 상시 노출된다

Action

	버튼명	기능
1	안저 파일 업로드	이미지 드래그 앤 드롭 또는 클릭하여 파일 선택
2	AI 분석 시작	이미지 선택 완료 시 AI 분석 시작 버튼 활성화
3	버튼 비활성화	분석 시작 후 버튼 비활성화 (중복 클릭 방지)

Validation

1	이미지 미선택 시 버튼 비활성 유지
2	지원 형식(JPG/PNG/TIFF) 외 파일 업로드 시 오류 메시지
3	20MB 초과 시 업로드 차단

화면 구현

페이지명 - report.js 담당자 - 홍승현

Description

1	분석 완료 후 추론 시간과 상태를 상단 바로 즉시 표시한다
2	좌측 카드에 이미지 썸네일과 질환명·확신도·중증도·응급 레벨을 가로로 나열한다 (응급 시 빨간 테두리)
3	우측에 5개 질환 확신도 바 차트를 표시하며 임계값 기반 색상 강조를 적용한다

Action

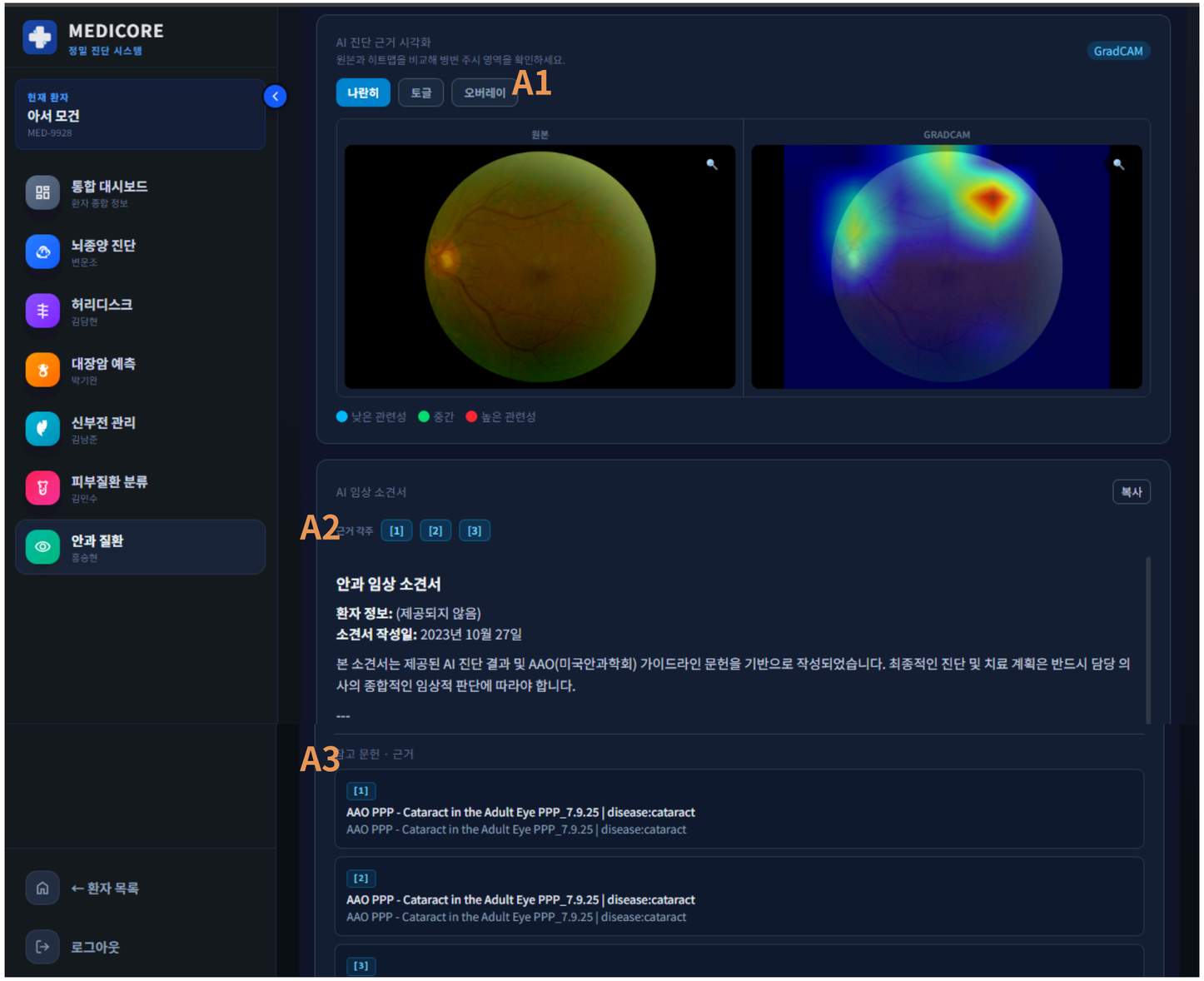
	버튼명	기능
1	새 분석 시작	상태 초기화 후 업로드 화면으로 복귀
2	이미지 제거	업로드 이미지 및 분석 결과 초기화
3	스크롤	GradCAM, 소견서, 근거 문헌 확인

Validation

1	응급 레벨 1~3에 따라 테두리 색상 구분 (노랑·주황·빨강)
2	확신도 35% 이상 주황 테두리, 55% 이상 빨강 테두리 자동 적용
3	응급 조건(Stage 3↑ 등) 충족 시 응급 모달 강제 표시

화면 구현

페이지명 - report.js 담당자 - 홍승현



Description

1	원본 이미지와 GradCAM 히트맵을 나란히·토글·오버레이 3가지 모드로 비교 표시하며, 클릭 시 확대 가능하다
2	AI 소견서가 타자 치듯 실시간 렌더링되며, 근거 각주 배지([1][2][3])와 복사 버튼을 제공한다
3	소견서 하단에 RAG 검색 결과 기반 참고 문헌 카드가 나열된다

Action

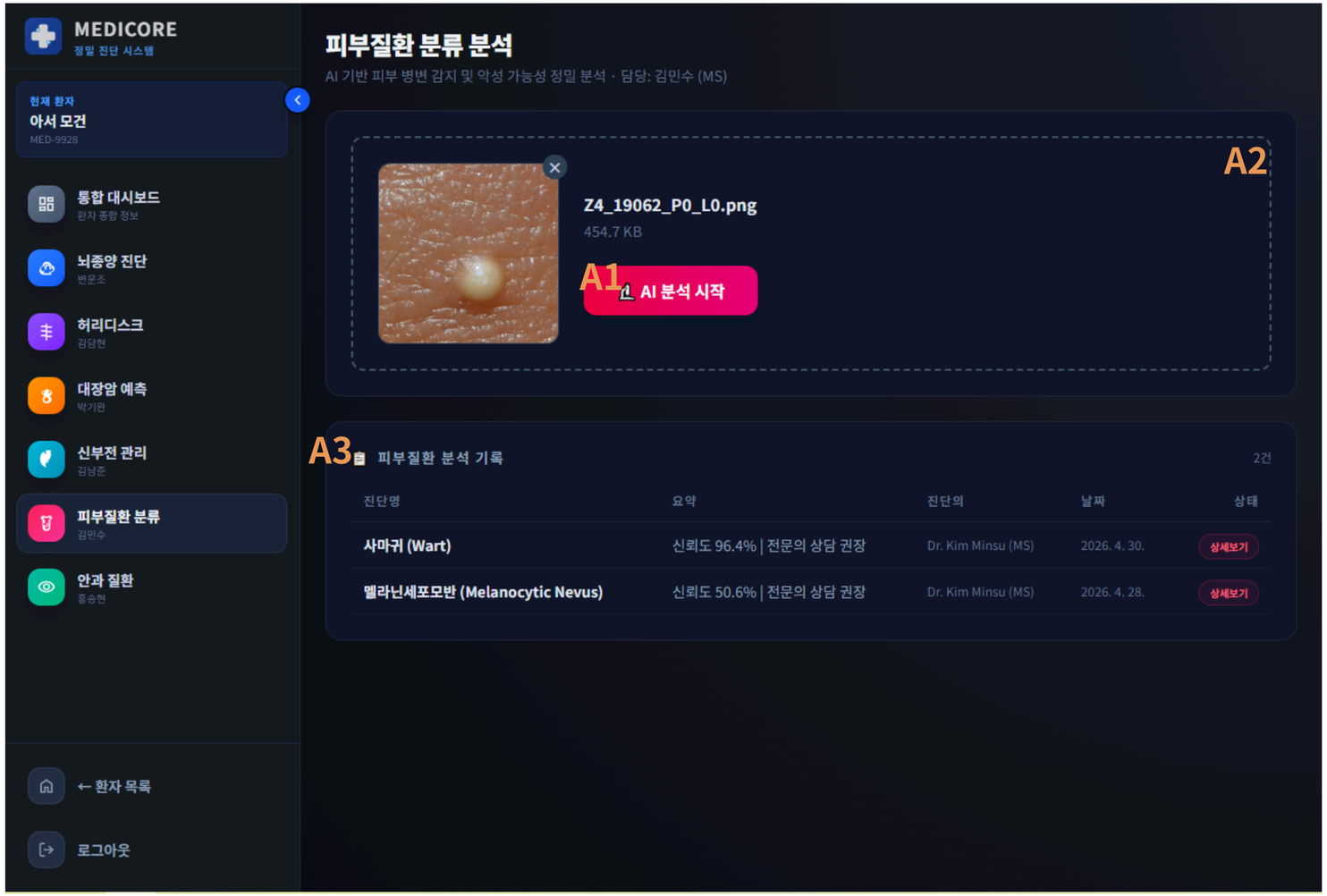
	버튼명	기능
1	나란히 / 토글 / 오버레이 탭 전환	히트맵 비교 방식 변경
2	이미지 우측 상단 🔍 클릭	원본 또는 GradCAM 확대 모달 표시
3	소견서 '복사' 버튼	클립보드 복사
4	근거 각주 [N] 클릭	해당 참고 문헌 카드로 스크롤 이동

Validation

1	GradCAM 생성 전 로딩 스피너 표시, 완료 시 자동 렌더링
2	오버레이 모드에서 슬라이더로 히트맵 투명도 조절 (0~100%)
3	근거 문헌 없을 경우 "RAG 검색 결과 없음" 문구 표시

화면 구현

페이지명 - Skin_aimedel.js 담당자 - 김민수



Description

1	환자 분석페이지에서 피부 질환 분석 페이지입니다.
2	피부 사진을 업로드하여 어떤 피부질환인지 확인합니다.
3	해당 피부 질환 상세 리포트를 받아서 확인한다.

Action

	버튼명	기능
1	분석 시작	해당하는 피부 질환의 대한 분석을 시작합니다
2	파일 드래그 앤 드랍	파일을 드래그 한 뒤 드랍하여 파일을 선택합니다
3	피부질환 분석 기록	진단했던 피부질환 목록을 나열합니다.

Validation

1	파일확장자를 .jpg, png로 제한합니다.
2	진단 모델 최적화 범위(황인)
3	최소 해상도를 224x224 픽셀 이상의 이미지만 업로드 가능

화면 구현

페이지명 - Skin_aimedel.js 담당자 - 김민수

Description

1	피부 질환을 분석하여 어떤 질환인지 AI가 진단한 결과내역입니다.
2	어떤점을 주로 참조하여 비교했는지 분석하였고, 근거가 어떠한 점인지 분석합니다.
3	해당하는 질환과 유사한 3개의 질환을 추천하여 다시금 확인할 수 있도록 합니다.
4	이러한 내용을 바탕으로 하단의 AI 분석 리포트를 작성하여 의사가 참고용으로 활용할 수 있습니다.

Action

	버튼명	기능
1	파일 저장	해당 질환을 저장하여 합니다.

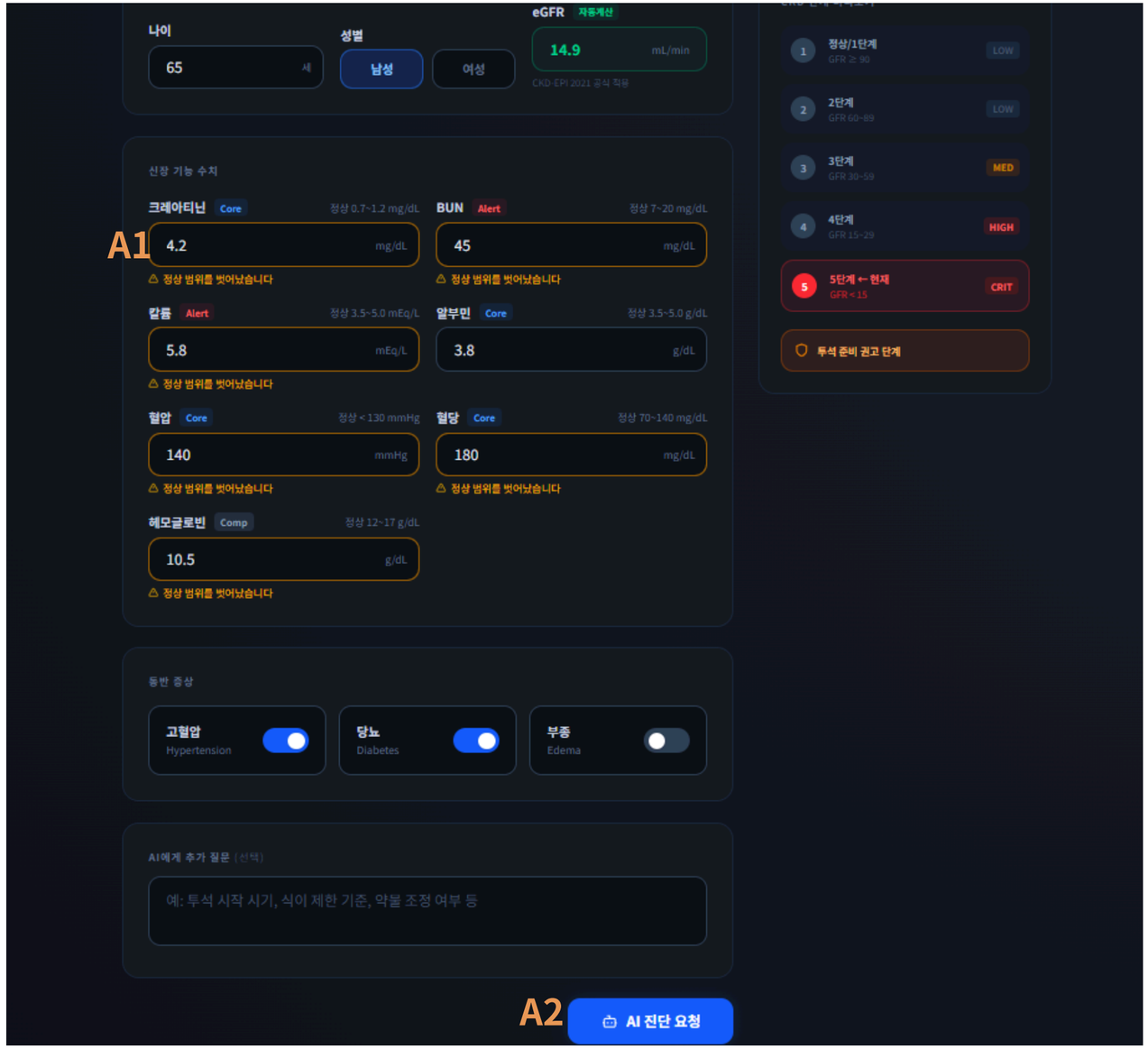
Validation

1	진단 모델 최적화 범위(황인)

작성자 : 김남준

화면 구현

페이지명 - kidney_aimedel.js 담당자 - 김남준



Description

1	단계 판정에 필수 7가지 임상 수치를 입력받는 화면
2	입력 편의성을 위해 각 수치별 정상 범위를 입력 필드 상단에 노출
3	주요 동반 질환 유무를 직관적인 스위치 형태 토글로 입력
4	GFR 수치 입력 시 우측 예상 CKD 단계를 실시간 시각화

Action

	버튼명	기능
1	임상치 입력	각 입력 필드에 수치 기입 및 증상 토글 버튼 선택
2	AI 진단 요청	버튼 클릭을 통한 백엔드로 데이터 전송 실행

Validation

1	Core 지표는 필수 입력이며, 하나라도 누락 시 “AI 진단 요청“ 버튼이 비활성화
2	모든 수치 입력란에 음수 값이 입력될 경우 즉시 붉은 테두리 및 에러
3	GFR 값이 15 미만인 경우 하단에 투석 준비 권고 안내 활성화

작성자 : 김남준

화면 구현

페이지명 - kidney_report.js 담당자 - 김남준

Description

1	AI가 생성한 진단 소견, 추천 의료안, 근거 문헌을 통합하여 보여주는 화면
2	입력값 중 어떤 값이 위험 수준인지 시각적인 지표 제공
3	실제 의료 가이드라인 출처와 해당 페이지 번호를 리스트 형태로 노출
4	의사가 AI 소견을 최종 검토, 추가 판단 기재 가능한 별도 텍스트 영역 제공

Action

	버튼명	기능
1	저장	생성된 보고서를 확정 상태로 DB에 기록

Validation

1	최고 확률 단계가 70% 미만이면 “재검토 권장“ 배지가 자동 표시
2	eGFR 진단 이력이 1건뿐인 경우 추세 차트 대신 “이전 진단 기록 없음” 안내
3	DB 저장 버튼은 보고서 데이터 로딩 완료 후에만 활성화

화면 구현

페이지명 - colon_aimedel.js 담당자 - 박기완

Description

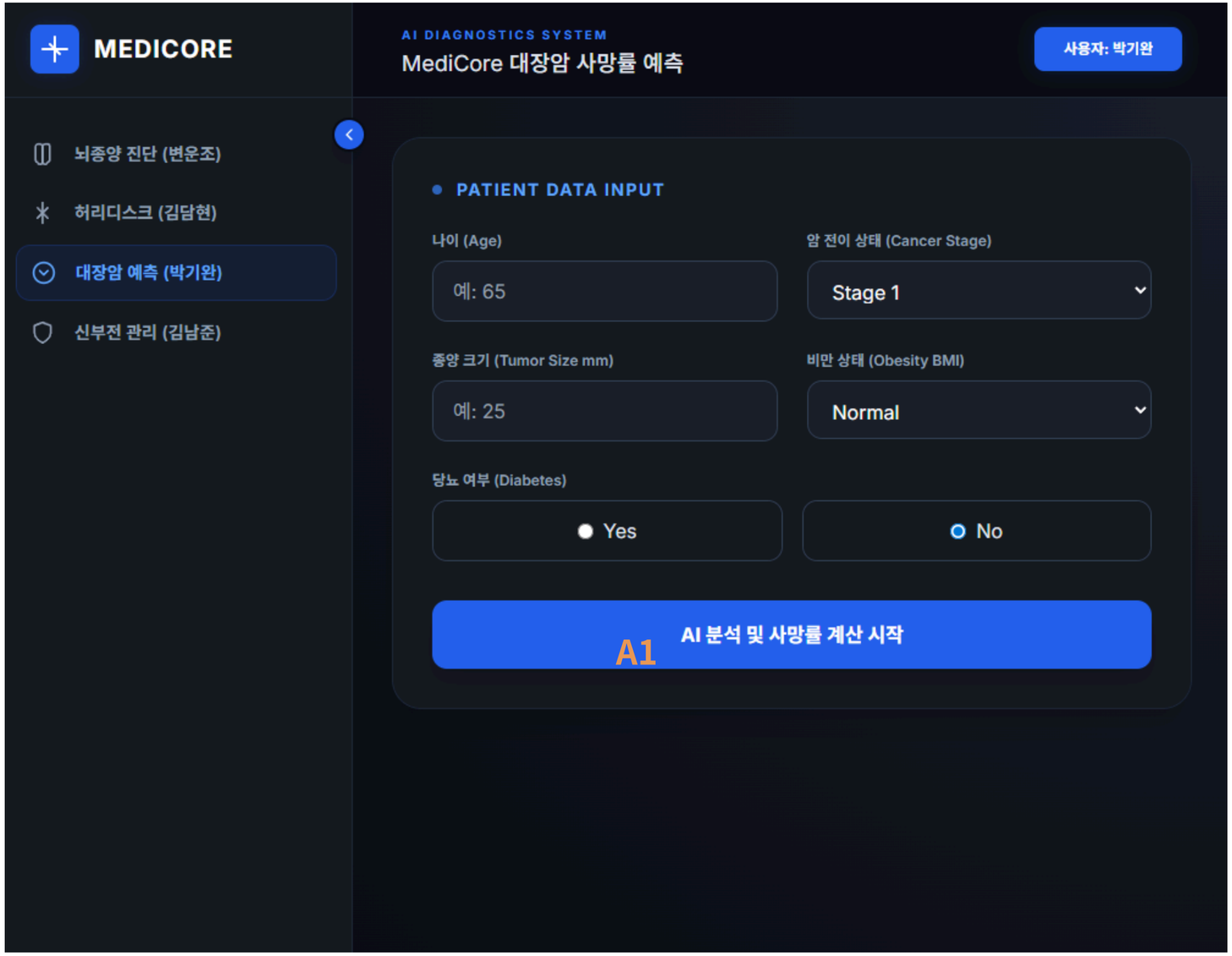
1	데이터를 로드하여 모델학습을 진행하고 우수한모델을 선택한 후 저장한다.

Action

	버튼명	기능
1	데이터로드	학습에 사용할 데이터를 로드 후 탐색, 전처리를 진행한다.
2	학습진행	둘 이상의 모델을 사용하여 학습을 진행하고 우수한 모델을 선정한다.
3	저장	우수한 모델로 선택된 모델을 저장한다.

화면 구현

페이지명 - report.js 담당자 - 박기완



Description

1	환자의 상태를 분야별로 입력하고 사망률을 계산한다.

Action

	버튼명	기능
1	사망률 계산	입력된 자료를 기초로 사망률을 계산한다.

모델 구현 - 척추 협착증

작성자 : 김담현

척추 협착증

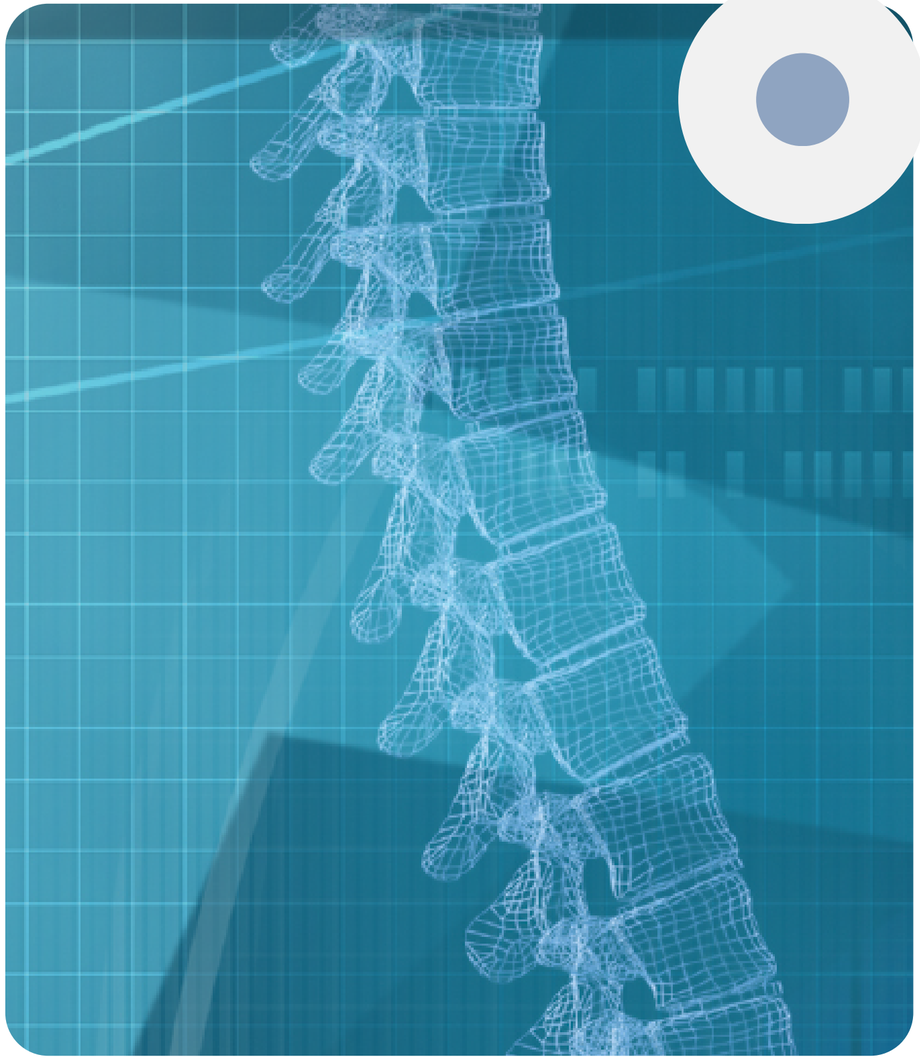
Vision DL + LangGraph

AI 척추 협착증 통합 진단 및 RAG 시스템

실제 임상의의 진료 프로세스*(MRI 확인 → 문진 → 종합 리포트)* 구현

- ResNet18 모델이 MRI 영상을 분석하여 척추 부위별 중증도를 객관적인 수치(확률값 JSON)로 추출.
- 프론트엔드에서 수집된 환자의 '임상 증상', '치료 이력', 'Red Flag(응급 징후)' 데이터를 Vision 결과와 하나의 컨텍스트로 통합
- Analyzer (분석), Expert (RAG), Comunicator(변환) - 멀티에이전트

단순한 '이미지 판독기'를 넘어, 맥락을 이해하고 환자 맞춤형 치료 방향을 제시하는 진정한 의미의 임상결정지원시스템*(CDSS)* 완성.

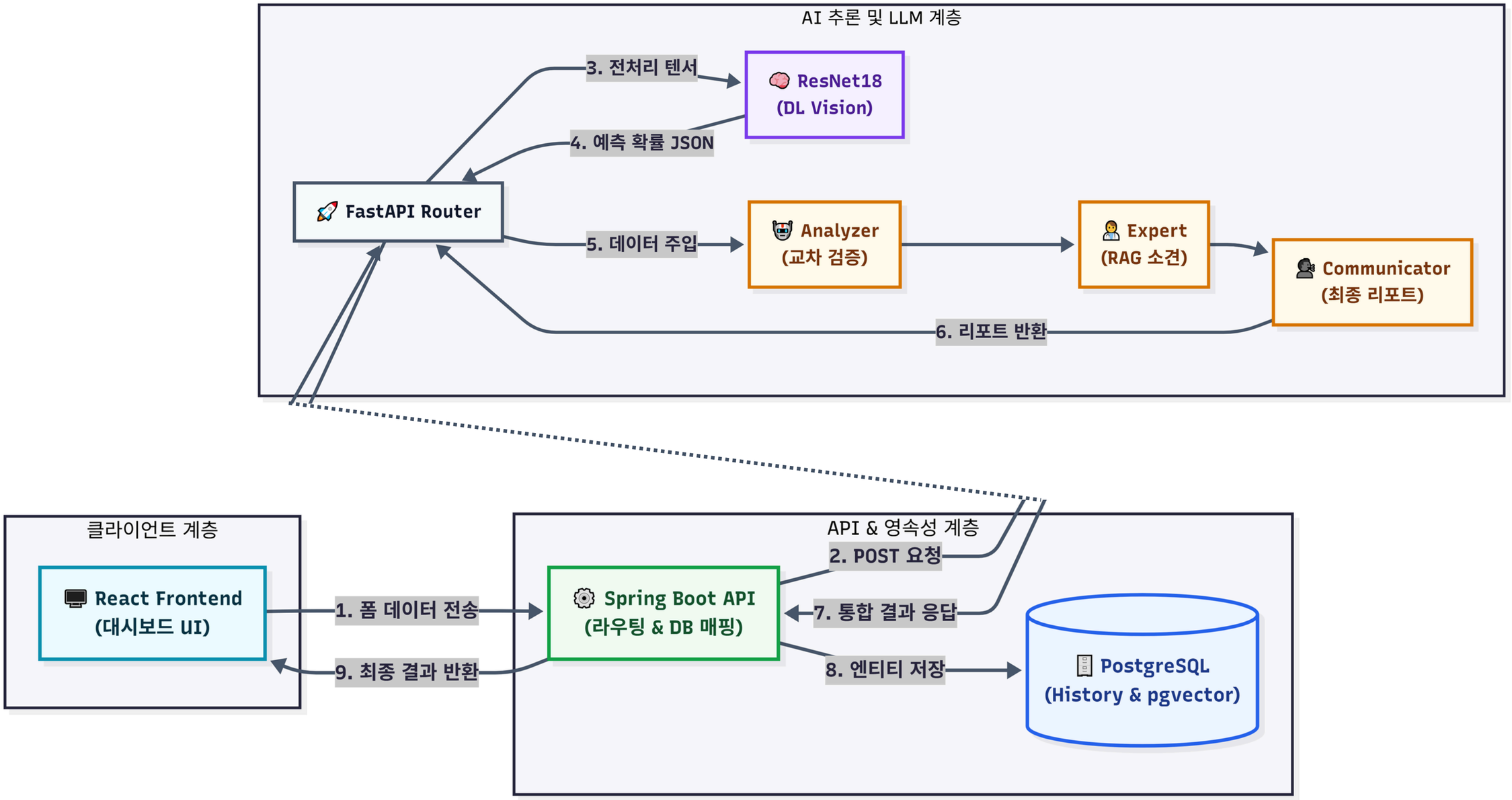


Tech Stack

React, SpringBoot, Fast API, Pytorch, LangGraph, pgvector

작성자 : 김담현

척추 협착증



척추 협착증

ResNet 사용이유

Skip Connection

H(X) = F(X) + X 라는 공식을 도입, 입력값 X를 다음 층으로 바로 건너뛰어(SKIP) 더해버림으로써, 신경망은 기존 정보 X에 추가적으로 필요한 변화량(RESIDUAL, 잔차)인 F(X)만 학습하면 되게 설계되있다. 고속도로에 비유하자면, 톨게이트를 일일이 거치지 않고 바로 목적지로 갈 수 있는 '하이패스 차로'를 뚫어준 셈입니다. 이 덕분에 층을 50층, 100층 이상 깊게 쌓아도 안정적으로 학습이 가능하다.
'기울기 소실(VANISHING GRADIENT)'의 완벽한 해결

ResNet-18

온프레미스 서버 환경과 실시간 추론 속도 최적화

- 클라우드의 무한한 GPU 자원을 쓰는 것이 아니라, 팀원의 로컬 PC에서 도커 컨테이너 형태로 돌아가는 제한된 온프레미스 환경입니다. 사용자가 웹에서 MRI 이미지를 올렸을 때, 백엔드 서버는 이미지 분석뿐만 아니라 RAG(LANGGRAPH) 시스템도 동시에 돌려야 하는데 무거운 50이상의 모델을 쓰면 GPU 메모리를 엄청나게 잡아먹고 추론 시간(INFERENCE TIME)이 길어져 사용자 경험(UX)이 떨어집니다. RESNET-18은 연산량이 훨씬 적어(FLOPS 감소), 가벼운 서버 환경에서도 병목 없이 실시간에 가까운 빠른 분석 결과를 웹으로 반환할 수 있습니다.
- 층수는 18층으로 줄였지만, RESNET 가문의 가장 위대한 유산인 'SKIP CONNECTION (잔차 연결)' 기술은 RESNET-18에도 똑같이 적용되어 있습니다
- RESNET-18은 척추의 골절이나 협착 같은 핵심적인 병변을 파악하기에는 충분한 깊이(18층)를 가지면서도, 불필요한 과적합을 막아주는 아주 훌륭한 스펙을 자랑합니다.

척추 협착증

PyTorch기반 ResNet18 커스텀 아키텍처

목적 : 척추 MRI 기반 3대 질환 × 3단계 중증도 동시 판별

Pre-trained Weights 활용

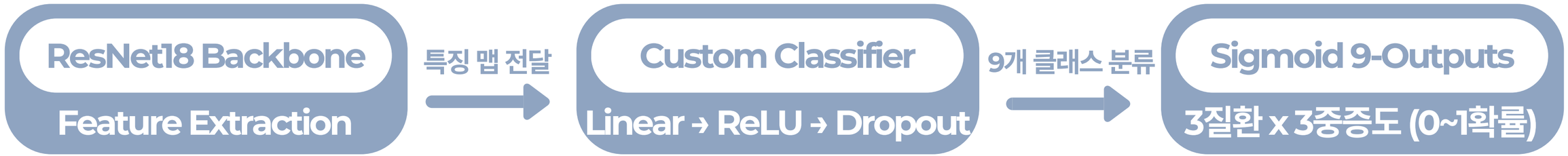
- 검증된 최적의 가중치 (.PTH) 를 메모리에 로드하여 추론 전용 파이프라인 구축

Architecture Design

- TIMM 라이브러리의 RESNET18 백본(BACKBONE)을 활용해 영상 특징(FEATURE) 추출
- 척추 질환에 특화된 커스텀 분류기 부착

Stable Prediction

- 추론 시 MODEL.EVAL() 모드를 강제하여 일관된 결과 보장
- SIGMOID 활성화 함수를 통과시켜 각 질환/중증도별 독립적인 확률값(0~1) 추출 및 JSON 직렬화



척추 협착증

ResNet18

```python
class MRIConditionModel(nn.Module):
    def __init__(self, backbone='resnet18', num_conditions=3):
        super().__init__()
        # timm 라이브러리에서 사전 학습된 ResNet18 뼈대를 가져옴
        self.backbone = timm.create_model(
            backbone,
            pretrained=False,    # False = 모델 구조만 빌려오고 가중치는 덮어씌우기 위해
            in_chans=3,          # mri는 흑백이지만, resNet18이 RGB 형식으로 되어있기때문
            features_only=False,
            num_classes=0        # 기존 분류항목을 없애고, 척추 분류항목으로 갈아끼우기 위해
        )
        self.num_features = self.backbone.num_features

        # Custom Head, 9개의 결과(3가지 질환 * 3가지 심각도)로 분류
        self.classifier = nn.Sequential(
            nn.AdaptiveAvgPool2d((1, 1)),
            nn.Flatten(),
            nn.Linear(self.num_features, 512), # 특징추출
            nn.BatchNorm1d(512),               # 안정성 부여
            nn.ReLU(),                         # 비선형모델 사용
            nn.Dropout(0.5),                   # 과적합 방지 (일반화)
            nn.Linear(512, 256),               # 특징추출
            nn.BatchNorm1d(256),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(256, num_conditions * 3) # 최종 출력: 3가지 질환 x 3가지 심각도
        )

    def forward(self, x):
        features = self.backbone.forward_features(x)
        x = self.classifier(features)
        return x
```

Pre-trained 모델을 활용한 전이 학습

TIMM.CREATE_MODEL을 호출할 때 NUM_CLASSES=0 옵션을 주어, 쓸모없는 기존의 1,000개짜리 분류기(머리)를 완전히 날려 오직 MRI 이미지에서 특징(FEATURE)만 뽑아내는 '순수한 비전 추출기'로 뼈대를 튜닝

과적합(Overfitting) 방지망

NN.BATCHNORM1D, NN.DROPOUT(0.5), NN.DROPOUT(0.3)

- DROPOUT (드롭아웃): 노드를 무작위로 50%, 30%씩 꺼버려서 모델이 특정 픽셀(노이즈)에만 의존하지 않고 척추의 전반적인 형태를 골고루 학습하도록 강제했습니다.
- BATCH NORMALIZATION (배치 정규화): 학습 과정에서 데이터의 분포가 튀는 것을 막아주어, 적은 데이터로도 학습이 안정적이고 빠르게 수렴하도록 튜닝했습니다.

9개를 예측하는 다중 출력 아키텍처

NN.LINEAR(256, NUM_CONDITIONS * 3)

보통의 모델은 "질환이 있다/없다" 1개만 출력하지만, 끝단을 9개의 출력(3가지 질환 × 3가지 심각도)으로 설계

척추 협착증

ResNet18

```python
# 가중치 부여 프로세스
def load_spine_model(weights_path='best_mri_model.pth'):
    try:
        # 1. 하드웨어 결정
        device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
        print(f"사용 중인 디바이스: {device}")          # 유연한 환경 구축

        # 2. 빈 모델 생성
        model = MRIConditionModel(backbone='resnet18', num_conditions=3)

        # 3. 가중치 파일(.pth) 불러오기
        checkpoint = torch.load(weights_path,
                                map_location=device, # 학습환경과 작동장치 환경 분리
                                weights_only=True)   # 가중치 데이터만 가져옴

        # 4. 빈 모델에 가중치 덮어씌우기
        model.load_state_dict(checkpoint['model_state_dict'])
        model.to(device)
        model.eval()                                        # 평가 모드로 변경!
              # 학습할 때는 성능 향상을 위해 넣었던 Dropout이나 BatchNorm 같은 기법들이
                진단을 할 때는 결과를 Random하게 만들기때문

        print("모델 및 가중치 로딩 성공!")
        return model, device

    except Exception as e:
        print(f"❌ 모델 로딩 실패: {e}")
        return None, None
```

유연한 환경 구축

'CUDA' IF TORCH.CUDA.IS_AVAILABLE() ELSE 'CPU'

- GPU)가 있다면 CUDA를 써서 광속으로 계산하고, 없으면 일반 CPU를 씁니다

설계도대로 빈 모델 생성

BACKBONE='RESNET18', NUM_CONDITIONS=3

가중치 로드

MODEL.LOAD_STATE_DICT(CHECKPOINT['MODEL_STATE_DICT'])

가중치 주입

MODEL.LOAD_STATE_DICT(CHECKPOINT['MODEL_STATE_DICT'])

평가 모드로 변경

MODEL.EVAL()

- 진단 결과가 누를 때마다 랜덤하게 바뀌지 않고 일관되게 고정

척추 협착증

LangGraph 기반 멀티 에이전트 RAG 파이프라인

모델 : GOOGLE GEMINI 2.0 FLASH & LANGGRAPH
목적 : VISION 결과와 임상 데이터를 종합한 3단계 맞춤형 의료 리포트 생성

Multi-Modal 데이터 융합

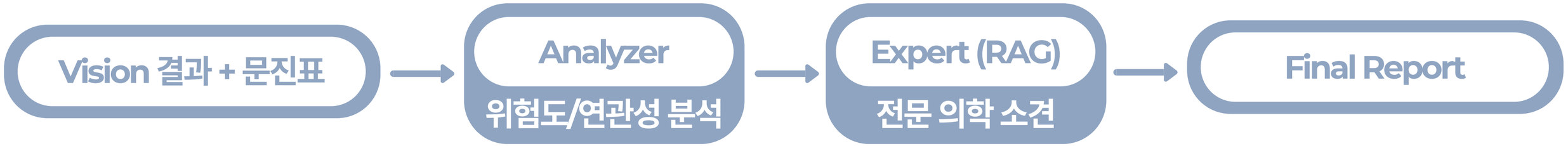
- VISION AI의 영상 판독 수치(중증도 확률)와 의사의 문진 데이터(RED FLAG, 증상, 치료 이력)를 결합하여 종합적인 컨텍스트 제공

LangGraph 기반 멀티 에이전트 분업화

- ANALYZER : 영상 소견과 실제 임상 증상의 연관성 분석 및 응급 징후 최우선 감지
- EXPERT : RAG 시스템을 통해 검색된 전문 가이드라인을 바탕으로 의학적 소견서 작성
- COMMUNICATOR (소통): 복잡한 의학 소견을 환자 눈높이에 맞춘 친절한 가이드로 번역.

RAG 기반 전문성 강화

- LLAMAPARSE로 척추 전문 가이드라인(NASS) 문헌을 파싱하고, PGVECTOR와 GEMINI 임베딩을 활용해 환자 상태에 맞는 정확한 근거 검색



척추 협착증

LangGraph - State

```
#State
class AgentState(TypedDict):
    raw_data: dict          # 비전 AI 모델이 준 JSON (MRI 판독 수치)
    clinical_data: dict     # [NEW] 의사가 체크한 환자 문진표 데이터
    analysis: str           # 수치 분석 결과
    medical_note: str       # 의학적 소견
    final_report: str       # 환자용 최종 리포트
```

State

CLASS AGENTSTATE(TYPEDDICT)

- 비전 AI 결과, 문진표, 분석 내용, 소견서, 리포트 공통 변수들을 선언

LangGraph - Node

```
#Node
workflow.add_node("analyzer", analyze_data)
workflow.add_node("expert", write_medical_note)
workflow.add_node("communicator", write_final_report)
```

Node

- ANALYZER: 비전+문진 데이터를 분석함
- EXPERT: 전문 소견서를 작성함.
- COMMUNICATOR: 환자용 리포트를 작성함.

LangGraph - Edge

```
#Edge
workflow.set_entry_point("analyzer")
workflow.add_edge("analyzer", "expert")
workflow.add_edge("expert", "communicator")
workflow.add_edge("communicator", END)
```

Edge

WORKFLOW.

- ENTRY_POINT 시작 노드는 ANALYZER(분석)
- ANALYZER -> EXPERT : 분석이 끈나면 소견서
- EXPERT -> COMMUNICATOR : 소견서 끝나면 리포트작성!

척추 협착증

analyzer 프롬프트

```
# 노드 프롬프트
def analyze_data(state: AgentState):
    mri_data = state['raw_data']
    clinical_symptoms = state['clinical_data']

    prompt = f"""
    당신은 척추 질환 전문 AI 'Analyzer'입니다.
    다음 두 가지 데이터를 종합하여 환자의 상태를 분석하고,
    다음 단계의 '전문의 AI'가 참고할 수 있도록 구조화된 분석 리포트를 작성

    [1. 비전 AI의 MRI 판독 수치 (질환별 중증도 확률)]:
    {mri_data}

    [2. 의사 문진 데이터 (임상 증상 및 치료 이력)]:
    {clinical_symptoms}

    작업 지시:
    1. [영상 소견]: MRI 판독 결과 중 'Moderate' 또는 'Severe'에 해당하는 가장 심각한 부위 특정
    2. [상호 연관성]: 영상의학적 결과(1번)가 환자의 실제 임상 증상과 논리적으로 일치하는지 분석,
       만약 영상은 심각한데 증상이 없거나, 그 반대의 '불일치' 상황이라면 이를 반드시 지적하세요.
    3. [위험도 평가]: 문진 데이터에 'Red Flag'가 포함되어 있다면 최우선 경고.
    4. [치료 방향성]: 수술적 치료가 필요한지, 보존적 치료가 필요한지 방향성을 근거와 함께 제시

    출력 형식:
    반드시 아래의 마크다운 헤더를 사용하여 명확하게 구분해 주세요.
    - **Primary Finding (주요 영상 소견)**:
    - **Clinical Correlation (임상-영상 연관성)**:
    - **Risk Assessment (위험도 평가)**:
    - **Treatment Direction (치료 방향성)**:
    """
    response = llm.invoke(prompt)
    return {"analysis": response.content}
```

analyze node : 데이터 통합 및 추론

숫자(DL)'와 '텍스트(문진)'라는 데이터를 하나의 논리로 엮는 부분

- 페르소나 : 척추 질환 전문 AI 'ANALYZER'라고 명시
- 멀티모달 통합 전략 : 비전 AI가 뱉은 '확률값'과 의사가 기록한 '증상'을 대조. 특히 '불일치 상황'을 지적은 의료 AI에서 매우 중요한 검증(VALIDATION) 로직.
- SAFETY 가드레일: RED FLAG를 최우선 강조하라는 명령은 응급 상황을 놓치지 않게 하는 안전장치
- CHAIN OF THOUGHT (단계적 사고): 영상 소견 -> 연관성 분석 -> 위험도 평가 -> 치료 방향 순으로 사고를 유도하여 논리적 비약을 방지

척추 협착증

Expert 프롬프트

```
# 노드 프롬프트
def write_medical_note(state: AgentState):
    analysis = state['analysis']
    prompt = f"다음 통합 분석 결과를 바탕으로, 전문의가 RAG 가이드라인에 맞춰 참고할 만한 전문적인 의학
              적 소견서를 작성해줘: {analysis}"
    response = llm.invoke(prompt)
    return {"medical_note": response.content}
```

expert node : 전문성 강화 (RAG)

앞선 분석 결과를 '의학적 표준'에 맞게 가공하는 부분

- CONTEXTUAL SHIFT: 분석 결과(ANALYSIS)를 기반으로 하되, 'RAG 가이드라인'이라는 외부 지식 참조 가능성을 열어둠.
- 전문 용어 최적화: '전문의가 참고할 만한 전문적인 의학적 소견서'라는 지시는 LLM이 일상어 대신 의학 전문 용어(JARGON)를 사용하도록 유도함.
- FOCUS: '진단'보다는 '기록'에 집중하여, 나중에 의사가 이 리포트를 봤을 때 이질감이 없도록 톤앤매너를 맞춤.

Communicator 프롬프트

```
# 노드 프롬프트
def write_final_report(state: AgentState):
    note = state['medical_note']
    prompt = f"다음 전문의 소견을 바탕으로, 환자가 이해하기 쉬운 친절한 최종 리포트를 작성해줘. 향후 치
              료 방향과 일상생활 주의사항을 명확히 포함해줘: {note}"
    response = llm.invoke(prompt)
    return {"final_report": response.content}
```

communicator node : 사용자 중심 번역

어려운용어, 영어 등을 쉽고 친절한 언어로 변경하는 부분

- ACTION-ORIENTED (행동 중심): 단순히 "당신은 아프니다"가 아니라, "앞으로 어떻게 해야 하는지(치료 방향, 주의사항)"를 명시하게 하여 데이터에 가치를 줌.
- DISTILLATION (증류): 복잡한 소견서(MEDICAL_NOTE)에서 환자에게 꼭 필요한 정보만 걸러내는 필터링 역할

척추 협착증

트러블슈팅 1 : RAG 파이프라인 API 할당량(Rate Limit) 방어 및 캐싱

PROBLEM

NASS(북미척추학회) 가이드라인 PDF 파싱 및 임베딩 과정에서 대량의 요청이 몰려 Gemini API 할당량 초과(HTTP 429 Too Many Requests) 에러 발생 및 LlamaParse 크레딧 과다 소모.

CAUSE

API 제공 업체의 RPM(분당 요청 수) 및 TPM(분당 토큰 수) 제한을 고려하지 않은 일괄 비동기/배치 처리.

SOLUTION

이중 방어 로직 구현 (Throttle & Caching)

1. 안전성 중심의 Throttle(속도 조절) 로직 도입

- 데이터를 50개 단위의 청크(Chunk) 배치로 분할. batch_size = 50
- 배치 전송 후 time.sleep(35) 대기열 삽입 및 예외 발생 시 60초 완전 휴식 후 재시도하는 Fail-safe 로직 구현 (rag_ingester.py).

2. 로컬 파일 캐싱(Local Caching) 적용

- LlamaParse로 추출한 마크다운 결과와 Gemini가 반환한 임베딩 벡터 리스트를 각각 .md와 .json 파일로 로컬에 저장.
- 재실행 시 캐시 파일 존재 여부를 검사하여 불필요한 중복 API 호출 원천 차단.

RESULT & IMPACT

- 에러율 0%: 대규모 데이터 적재 시 끊김 없는 안정적인 파이프라인 구축.
- 비용 최적화: 개발 및 테스트 단계에서 불필요한 API 크레딧 소모 100% 방어.

척추 협착증

트러블슈팅 2 : 다중 데이터(이미지+JSON) 통합 라우팅 및 직렬화

PROBLEM

React → Spring Boot → FastAPI로 이어지는 흐름에서, MRI 이미지 파일과 문진표 객체를 동시에 전송할 때 MediaType 매핑 충돌 및 파싱(Parsing) 에러 발생

CAUSE

HTTP REST API 표준상 파일 전송은 multipart/form-data를 사용하지만, 내부의 복잡한 중첩 객체(JSON)를 프레임워크(Spring, FastAPI)가 각기 다른 방식으로 해석하여 데이터가 유실되거나 타입 에러 발생

SOLUTION

데이터 캡슐화 및 명시적 브릿지(Bridge) 패턴 적용

1. Frontend (React): JSON 명시적 직렬화

- 문진표 객체(clinicalData)를 그대로 싣지 않고 JSON.stringify()를 통해 단일 문자열(String)로 직렬화하여 FormData에 통합.

2. API Gateway (Spring Boot): 파일 리소스 강제 매핑

- 수신받은 MultipartFile을 Python 서버로 중계할 때, ByteArrayResource로 래핑.
- 특히 getFilename() 메서드를 오버라이딩하여 파일명을 명시적으로 꽂아줌으로써 FastAPI가 정상적인 파일 스트림으로 인식하도록 브릿지 역할 수행.

RESULT & IMPACT

- 데이터 정합성 확보: 이미지(정량 데이터)와 문진표(정성 데이터)가 하나의 요청 컨텍스트로 묶여 유실 없이 AI 모델까지 도달.
- 마이크로서비스 통신 안정성: 각기 다른 언어(Java, Python) 간의 데이터 교환 규격을 완벽하게 통일.

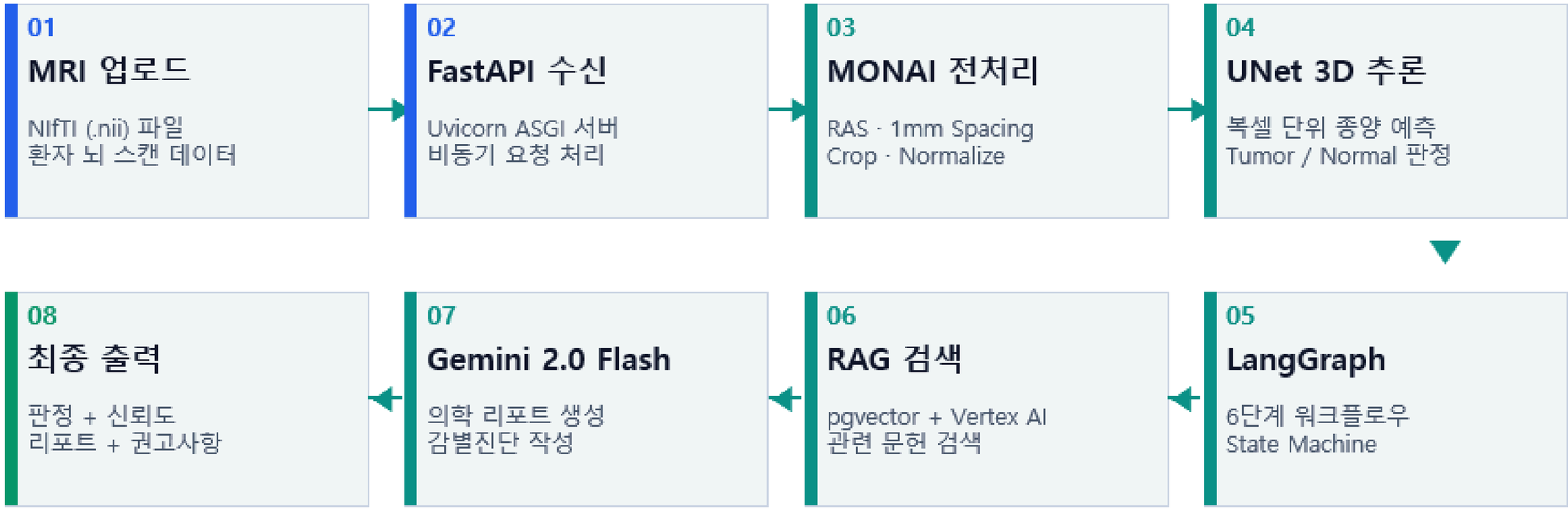
모델 구현 - 뇌종양 판별 모델

작성자 : 변운조

뇌종양 모델

01 MediCore 뇌종양 AI 진단 시스템 — 전체 파이프라인

환자 MRI 업로드부터 의학 리포트 생성까지 — End-to-End AI 워크플로우



핵심 기술 스택

FastAPI · MONAI · UNet 3D · LangGraph · pgvector · Gemini 2.0 Flash

MediCore · Brain Tumor AI

뇌종양 모델

02 학습 데이터셋 — BraTS 2020 + IXI

BraTS 2020

종양 데이터셋

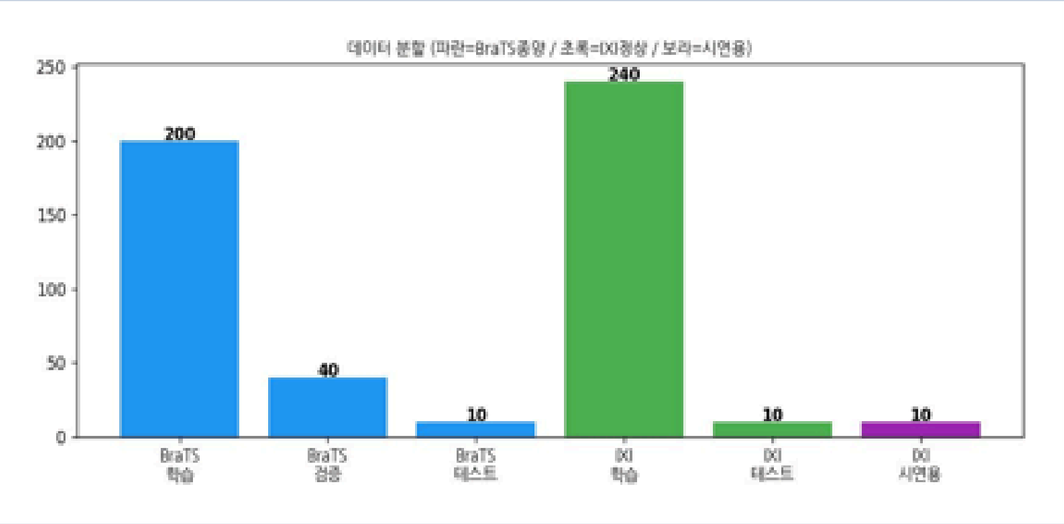
- Brain Tumor Segmentation Challenge 공개 데이터셋
- T1 가중 MRI · Skull-stripped · 1mm Isotropic
- **학습 200 · 검증 40 · 테스트 10 케이스**

IXI

정상 데이터셋

- Information eXtraction from Images 데이터셋
- 종양 없는 정상 뇌 MRI
- **학습 240 · 테스트 10 · 시연 10 케이스**

데이터 분할 — 총 510 케이스



510	250+260	30
총 케이스	종양 + 정상	테스트 세트

뇌종양 모델

03 전처리 파이프라인 — MONAI Transforms

다양한 스캐너·기관에서 수집된 MRI를 표준화하여 모델 입력 형태로 변환

1. **LoadImaged** — NIfTI 파일 로드
2. **EnsureChannelFirstd** — 채널 차원 앞으로
3. **Orientationd** — RAS 방향 통일
4. **Spacingd** — 1mm³ Isotropic 리샘플링
5. **CropForegroundd** — 뇌 영역 자동 크롭
6. **NormalizeIntensityd** — 비제로 복셀 z-score
7. **ToTensord** — PyTorch Tensor 변환

출력 (1, 1, H, W, D) float32 Tensor

비제로 정규화

배경 복셀(0)은 정규화에서 제외 →
뇌 조직만 정확히 z-score 정규화

Spacing 통일

1mm³ Isotropic으로 통일 →
다양한 스캐너·기관 데이터 호환

Foreground Crop

뇌 외 배경을 자동 제거 →
불필요한 연산량 대폭 감소

뇌종양 모델

04 1차 시도 — DenseNet 3D 분류 모델 접근

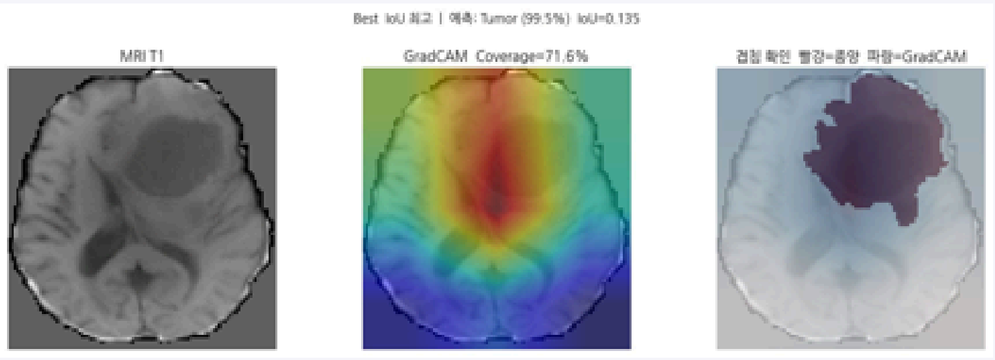
APPROACH

전체 볼륨을 통째로 보고 분류하기

모델	DenseNet121 (3D 버전)
입력	전체 3D MRI 볼륨
구조	GAP → FC → Tumor / Normal
시각화	GradCAM으로 집중 영역 표시

⚠ 다음 슬라이드: 실제로는 심각한 문제 발생

Best Case — 운 좋은 케이스



71.6% GradCAM Coverage

0.135 IoU

종양 위치를 일부 포착하나, 모델이 전체 뇌 영역을 보는 경향

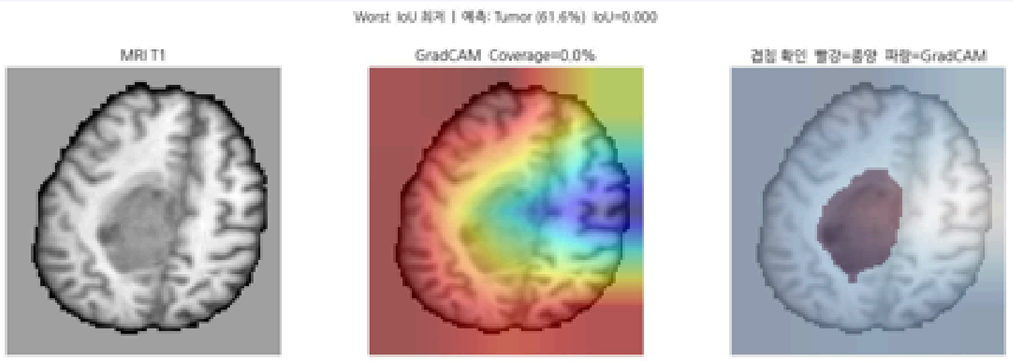
뇌종양 모델

05 DenseNet의 한계 — 왜 UNet으로 전환했는가

발생한 5가지 문제점

1. **공간 정보 손실**
   GAP으로 위치 정보 소멸 → 종양 위치 알 수 없음
2. **시각화 불가**
   종양 영역 하이라이트 기능 구현 불가
3. **정상 뇌 과민 반응**
   작은 아티팩트 하나에도 Tumor 오분류
4. **GradCAM 불안정**
   Worst case Coverage 0% (오른쪽 이미지 참고)
5. **체크포인트 구조 불일치**
   모델 저장/로드 시 구조 호환성 오류 발생

Worst Case — 모델이 완전히 빗나간 케이스



0.0%	0.000	61.6%
Coverage	IoU	신뢰도

→ 결론: 복셀 단위 세그멘테이션 모델로 전환 필요

뇌종양 모델

06

2차 시도 — UNet 3D 세그멘테이션 기반 분류

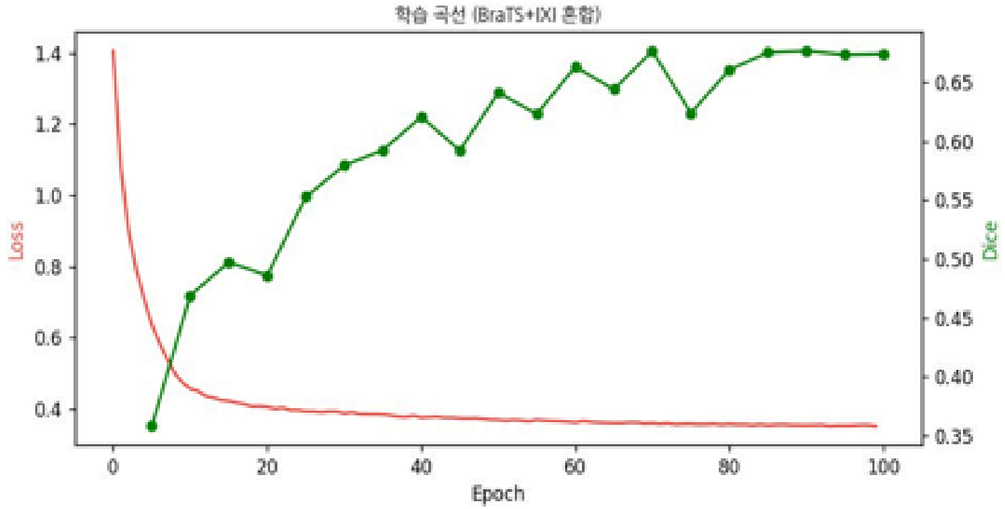
ARCHITECTURE

MONAI UNet-3D + Skip Connections

구조	Encoder–Decoder + Skip
채널	1 → 16 → 32 → 64 → 128 → 256
출력	Background / Tumor 2채널 확률 맵
추론	Sliding Window (128³ 패치)
분류	tumor_fraction > 0.5% → Tumor

✓ 위치 정보 보존 → 종양 영역 시각화 가능

학습 결과 — Dice 0.67 도달

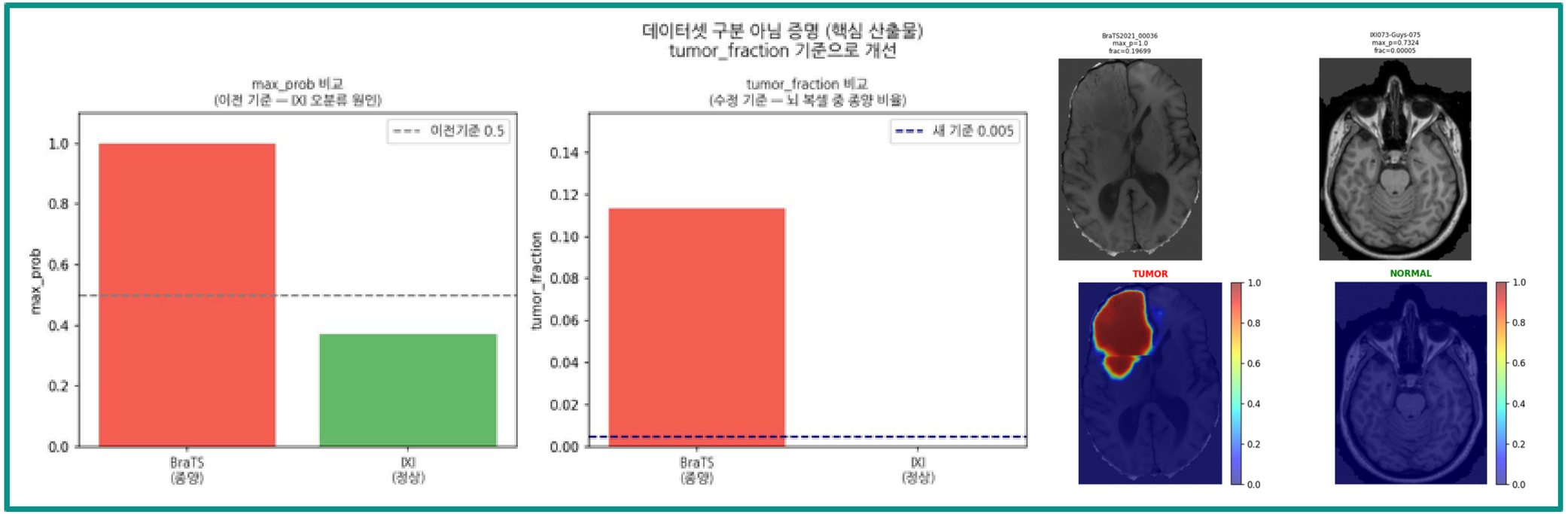


440 케이스	100	128³
혼합 학습 데이터	Epoch	Patch

뇌종양 모델

07 핵심 개선 — max_prob → tumor_fraction 전환

문제 기존 max_prob 기준에서 IXI 정상 뇌 **100%**를 Tumor로 오분류 (정상 분류율 **0%**)



0.677 BraTS Dice
종양 세그멘테이션

0.641 BraTS IoU
테스트 세트

100% IXI 정상 분류율
오분류 0건 — 완전 해결

모델 구현 - 신부전 관리 모델

작성자 : 김남준

신부전 관리

프로젝트 선정 개요 및 목적

문제 정의

- 만성 신부전(CKD)은 초기 증상이 없어 발견이 늦음
- GFR 수치만으로 단계를 판단하는 의사 의존도 높음
- 환자별 맞춤 치료 권고사항 생성에 많은 시간 소요

개발 목적

- 혈액 검사 수치 입력만으로 CKD 단계 즉시 분류
- KDIGO 가이드라인 기반 AI 소견서 자동 생성
- 의료진의 진단 보조 도구로 활용
- 신장내과 전문 지식을 RAG로 검색·활용

혈액 수치 → TabNet 5단계 분류 + KDIGO RAG 검색 → LLM 소견서 생성 → 의료진 보조

신부전 관리

TabNet 모델 선정 이유

기준	Random Forest	XGBoost	MLP (DNN)	TabNet ✓
해석 가능성	△ 부분적	△ 부분적	✗ 낮음	✅ 피처별 기여도
표 형식 데이터	✅ 우수	✅ 우수	△ 보통	✅ 최적화
소규모 데이터	✅ 강함	✅ 강함	✗ 약함	✅ 강함
순차적 어텐션	✗ 없음	✗ 없음	✗ 없음	✅ 핵심 구조
피처 선택	△ 중요도만	△ 중요도만	✗ 불투명	✅ 단계별 선택
과적합 방지	✅ 우수	✅ 우수	△ 보통	✅ Ghost BN

선정 근거 : 의료 데이터는 소규모 표 형식이며, 의사에게 '왜 이 단계인지' 설명할 수 있는 해석 가능성이 필수.
TabNet의 순차적 어텐션 메커니즘은 단계별로 중요 피처를 선택하여 예측 근거를 시각화할 수 있음.

신부전 관리

데이터셋 및 전처리

데이터셋

출처 : Kaggle Kindey Disease Dataset

레이블 : CKD 5단계 (Normal~Stage5)

피처 : 혈액 검사 수치 + 범주형

입력 피처 (9개+범주형)

sc 크레아티닌 bu BUN
egfr 사구체여과율 pot 칼륨
bp 혈압 hemo 헤모글로빈
bgr 혈당 al 알부민
htn 고혈압 dm 당뇨 pe 부종

전처리 단계

① 결측값 중앙값 대체 (Imputer)
② 표준화 (StandardScaler)
③ SMOTE 오버샘플링 (클래스 불균형 보정)
④ 피처 가중치 적용

SMOTE 적용 후 클래스 균형 분포

※ SMOTE: 소수 클래스를 합성 오버샘플링하여 단계별 균등 분포 확보

신부전 관리

전체 파이프라인 아키텍처

System Architecture

Frontend (React)	Backend (Spring Boot)	AI Server (FastAPI)	RAG Engine (LangChain)	LLM (Vertex AI)
KidneyFailurePage.jsx eGFR 자동 계산 결과 표시 · 이력 조회	KidneyDiagnosisController /api/patients/{id} /diagnoses/kidney	router.py predict() · explain() TabNet 5단계 분류	retriever.py · engine.py BGE-M3 임베딩 multi_search()	gemini-2.5-flash ChatVertexAI PydanticOutputPar ser

pgvector (PostgreSQL)

medical_knowledge 테이블 | module_tag='kidney' | KDIGO 가이드라인 청킹 → BGE-M3 1024차원 임베딩 저장 | cosine similarity 검색

신부전 관리

핵심 코드 - 신부전 진단 모델 호출

```python
@router.post("/ai/kidney/diagnose")
async def kidney_diagnose(req: KidneyDiagnoseRequest):
    # 1. TabNet 예측
    dl_result = predictor.predict(clinical_data)

    # 2. 위험 알림 (eGFR < 15, 칼륨 >= 6.0 등 임계값 기반 룰)
    alerts = check_alerts(req, prediction)

    # 3. RAG + 소견서 생성
    rag_result = query_and_generate(query, prediction,
...)
    # 4. 피처 기여도 (TabNet attention mask)
    feature_importance = predictor.explain(clinical_data)
}
```

- 진단 요청이 들어오면 4단계를 순서대로 실행하고 결과를 하나의 응답으로 조립하는 파이프라인 진입점
- PREDICTOR.PREDICT()로 TABNET 추론을 먼저 실행해 CKD 단계를 확정한 뒤, 해당 단계와 원본 수치를 동시에 CHECK_ALERTS()에 전달해 모델이 놓칠 수 있는 즉각적인 위험 수치를 별도로 감지.
- 이후 예측 단계 기반으로 RAG 검색과 LLM 소견서 생성을 QUERY_AND_GENERATE()에서 처리하고 마지막으로 PREDICTOR.EXPLAIN()을 호출해, 이번 판단에 가장 영향을 준 임상 수치를 피처 기여도로 수치화.
- 4개의 결과를 최종 응답 객체에 합산해 반환.

신부전 관리

핵심 코드 - 입력 수치 기반 진단

```
학습 구조:

# eGFR 수치 → KDIGO 기준 5단계로 변환 (타겟 생성)
def assign_stage(egfr):
    if egfr >= 90:  return 'Normal_Stage1'
    elif egfr >= 60: return 'Stage2'
    elif egfr >= 30: return 'Stage3'
    elif egfr >= 15: return 'Stage4'
    else:            return 'Stage5'   # 투석 필요

# 클래스 불균형 해결: SMOTE 오버샘플링
smote = SMOTE(random_state=42)
X_train, y_train = smote.fit_resample(X_train, y_train)

# TabNet 학습 (n_d=64, n_steps=5)
model = TabNetClassifier(n_d=64, n_a=64, n_steps=5, ...)
model.fit(X_train, y_train, max_epochs=300, patience=30)

추론 구조:

class KidneyPredictor:
    def predict(self, input_data):
        # 결측값 → SimpleImputer(median)로 채움
        # 범주형 → LabelEncoder
        # 수치형 → StandardScaler 정규화
        X = self._preprocess_input(input_data)
        probs = self.model.predict_proba(X)[0]
        # → {"Normal_Stage1": 0.02, "Stage2": 0.05, "Stage3": 0.88, ...}

    def explain(self, input_data):
        # TabNet attention mask → 각 피처가 예측에 기여한 비율
        # → {"eGFR": 0.42, "크레아티닌": 0.18, "혈압": 0.11, ...}
```

학습 구조

- 원본 데이터셋에는 진단 레이블이 없고 EGFR 수치만 존재. ASSIGN_STAGE()가 KDIGO 국제 기준에 따라 EGFR 값을 5단계로 변환해 타겟 변수를 직접 생성.
- 모델이 편향되는 문제를 SMOTE 오버샘플링으로 해결 — 소수 클래스 샘플 주변에 합성 데이터를 생성해 클래스별 균등 학습 보장.
- TABNET은 N_STEPS=5의 순차적 어텐션 구조로 각 단계마다 다른 피처 조합에 집중하며 학습.

추론 구조

- SIMPLEIMPUTER(MEDIAN)으로 결측값을 중앙값으로 대체한 뒤 학습 시와 동일한 STANDARDSCALER로 정규화.
- PREDICT_PROBA()로 5단계 각각의 확률을 반환하고, TABNET 고유의 ATTENTION MASK를 집계해 어떤 수치가 이번 판단에 얼마나 기여했는지 비율로 출력.
- 일반 신경망과 달리 모델 내부에서 피처 선택 과정이 명시적 기록되어 별도 해석 도구 없이 설명 가능한 AI(XAI) 구현.

신부전 관리

핵심 코드 - 의료 가이드라인 참조

```
def embed_query(text):
    if USE_GEMINI:
        # Vertex AI gemini-embedding-001 (1024차원)
        return TextEmbeddingModel.get_embeddings(text)
    else:
        # 로컬 BGE-M3 (1024차원, 동일 차원 → DB 재임베딩 불필요)
        return _BGEWrapper().encode(text)

def search(query, top_k=4):
    vec = embed_query(query)
    # module_tag = 'kidney' 필터 필수 — 타 팀 데이터 오염 방지
    sql = """
        SELECT content, source,
               1 - (embedding <=> %s::vector) AS score
        FROM   medical_knowledge
        WHERE  module_tag = 'kidney'
        ORDER  BY embedding <=> %s::vector
        LIMIT  %s
    """
```

- GCP_PROJECT_ID 환경 변수 설정 여부에 따라 임베딩 모델을 자동 전환.
- VERTEX AI 연결 시 GEMINI-EMBEDDING-001, 미연결 시 로컬 BGE-M3를 사용하되 양쪽 모두 1024차원으로 통일되어 DB 재임베딩 없이 모드 전환 가능.
- 검색 쿼리에 WHERE MODULE_TAG = 'KIDNEY' 필터를 반드시 포함 — 미적용 시 다른 질환 의학 문서까지 검색되어 전혀 다른 치료 지침이 소견서에 섞이는 환각 발생.
- MULTI_SEARCH()는 CKD 단계별로 사전 정의된 서브쿼리 2~5개를 THREADPOOLEXECUTOR로 병렬 실행해 단일 쿼리보다 다각도로 관련 문서를 확보.
- 결과 병합 시 동일 내용 중복 제거와 동일 출처 최대 2개 제한으로 편향된 검색 결과 방지.

신부전 관리

핵심 코드 - 소견서 생성

```
# LLM 우선순위
USE_VERTEX_LLM = bool(GCP_PROJECT_ID)   # 1순위: Vertex AI Gemini
USE_OLLAMA     = bool(OLLAMA_BASE_URL)   # 2순위: Ollama 로컬 LLM
# 3순위: 규칙 기반 폴백 (LLM 없이도 동작)

def _generate_report(prediction, confidence, input_data, contexts, query,
llm)# LCEL 체인: llm | PydanticOutputParser
    # → JSON 형식으로 강제 파싱 (섹션별 구조화)
    chain = llm | parser
    result = chain.invoke([SystemMessage(...), HumanMessage(prompt)])

# 시스템 프롬프트 — 한국어 강제
_KOREAN_SYSTEM = (
    "당신은 한국어로만 응답하는 신장내과 전문의 보조 AI입니다. "
    "반드시 한국어로만 작성하십시오..."
)
```

- GCP_PROJECT_ID → OLLAMA_BASE_URL → 규칙 기반 템플릿 순의 3단계 폴백 구조로 네트워크나 LLM 연결 없이도 소견서 생성 가능.
- LANGCHAIN LCEL 체인 LLM | PYDANTICOUTPUTPARSER로 LLM 출력을 KIDNEYREPORT PYDANTIC 모델에 강제 파싱 — LLM이 자유 형식으로 답하더라도 6개 섹션으로 구조화된 JSON만 허용.
- 한국어 강제 시스템 프롬프트로 영어·베트남어 등 외국어 혼입 방지.
- 파싱 실패 시 해당 섹션만 규칙 기반 폴백 문구로 대체해 전체 응답이 무너지지 않도록 섹션 단위 예외 처리.

신부전 관리

핵심 코드 - 소견서 템플릿

```python
# Pydantic 모델로 LLM 출력 구조 강제
class KidneyReport(BaseModel):
    sec2: str = Field(description="위험도 평가 및 원인 추정")
    sec3: str = Field(description="이상 소견 해석")
    sec41: str = Field(description="약물 처방")
    sec42: str = Field(description="생활 습관 및 비약물 치료")
    sec43: str = Field(description="추적 검사 계획")
    sec5: str  = Field(description="종합 진단 소견")

# 섹션 1은 코드로 직접 생성 (수치 기반, LLM 불필요)
def build_section1(prediction, confidence, input_data):
    # eGFR, 크레아티닌, 칼륨 등 주요 수치 표로 출력
    # → 항상 일정한 형식 보장

# LLM 실패 시 규칙 기반 폴백
def build_fallback_sections(prediction, contexts, ...):
    # RAG 검색 결과 + 단계별 정형 문구로 소견서 생성
    # → GCP 없이도 기본 소견서 제공 가능
```

- 섹션 1은 실제 입력값을 코드로 직접 표로 출력 — EGFR, 크레아티닌, 칼륨 같은 핵심 수치는 LLM에 맡기면 값을 변형하거나 없는 수치를 생성하는 환각 위험이 있어 코드 생성으로 100% 정확도 보장.
- 섹션 2~5는 LLM이 검색된 가이드라인 문서를 근거로 생성하되, PYDANTICOUTPUTPARSER가 프롬프트에 JSON 스키마를 자동 주입해 구조화된 출력을 강제.
- BUILD_FALLBACK_SECTIONS()는 RAG 검색 결과와 단계별 정형 문구를 조합해 LLM 없이도 기본 소견서를 제공. 한글 2바이트 너비를 고려한 _WC_LJUST()로 한글·영문 혼합 표에서 열 정렬이 틀어지는 문제를 보정.

신부전 관리

모델 성능 지표

Model Performance

최종 검증 정확도

98.13%

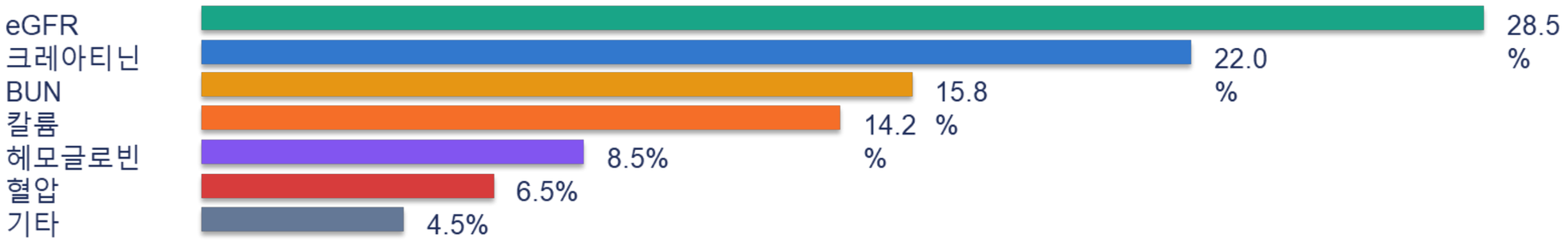
TabNet 5-Class

Kaggle CKD Dataset

단계별 분류 성능

단계	Precision	Recall	F1-Score
Normal/Stage1	99%	98%	98%
Stage 2	97%	97%	97%
Stage 3	98%	99%	98%
Stage 4	99%	98%	98%
Stage 5	99%	99%	99%

피처 기여도 (TabNet Attention)



eGFR 자동 계산 및 입력 모순 차단

eGFR Auto-Calculation (CKD-EPI 2021)

발견된 문제 (입력값 의학적 모순)

크레아티닌 35 mg/dL (말기 신부전 수준) + eGFR 100 mL/min (정상) 동시 입력 가능
→ 모델이 eGFR 100을 신뢰 → Stage 1 오출력

해결 — CKD-EPI 2021 자동 계산

eGFR 직접 입력 필드 제거
크레아티닌 + 나이 + 성별 입력 시 CKD-EPI 2021 공식으로 실시간 자동 계산

CKD-EPI 2021 Race-Free Formula (KidneyFailurePage.jsx)

```
function calcEgfr(sc, age, sex) {
  const kappa  = sex === 'F' ? 0.7   : 0.9      // 성별 기준값
  const alpha  = sex === 'F' ? -0.241 : -0.302   // 성별 지수
  const sexFac = sex === 'F' ? 1.012  : 1.0      // 성별 보정 계수
  const ratio  = sc / kappa
  return 142 * Math.min(ratio,1)**alpha
         * Math.max(ratio,1)**(-1.2)
         * 0.9938**age * sexFac }          // 연령 보정
```

eGFR 구간별 CKD 단계 매핑

≥ 90	60–89	30–59	15–29	< 15
Normal / Stage 1	Stage 2	Stage 3	Stage 4	Stage 5 (ESRD)

신부전 관리

RAG 파이프라인 구현

Retrieval-Augmented Generation

① 쿼리 생성	② 병렬 임베딩	③ 벡터 검색	④ 결과 병합	⑤ 컨텍스트
CKD 단계 + 서브쿼리 목록 (_STAGE_SUBQUERIES)	BGE-M3 1024차원 ThreadPoolExecutor (최대 5스레드)	pgvector cosine similarity module_tag='kidney'	중복 제거 (80자) 출처 다양성 보장 Top 6 반환	KDIGO 가이드라인 최대 5개 발췌문 → LLM 프롬프트

retriever.py — multi_search() 핵심 로직

with ThreadPoolExecutor(max_workers=min(len(queries),5)) as ex: futures=[ex.submit(search,q,top_k_each) for q in queries] → 중복 제거(content[:80]) → 출처 다양성(max_per_source=2) → merged[:6] 반환

BGE-M3 임베딩 모델 : BAAI/bge-m3 | 출력 차원 1024 | FlagEmbedding 실패 시 sentence-transformers 자동 폴백 | CUDA_VISIBLE_DEVICES="" (CPU 전용)

신부전 관리

AI 소견서 생성 - LangChain LCEL

Report Generation via LangChain LCEL

소견서 5개 섹션 구조

섹션 1	핵심 수치 요약 + CKD 단계	규칙 기반
섹션 2	위험도 평가 및 원인 추정	LLM 생성
섹션 3	이상 소견 해석	LLM 생성
섹션 4	치료 권고사항	LLM 생성
4-1	약물 처방	
4-2	생활 습관·비약물 치료	
4-3	추적 검사 계획	
섹션 5	종합 진단 소견	LLM 생성

engine.py — LangChain LCEL 체인

```
class ReportSections(BaseModel):
    sec2: str = Field(default="", description="위험도 평가 및 원인 추정")
    sec3: str = Field(default="", description="이상 소견 해석")
    sec41: str = Field(default="", description="약물 처방")
    sec42: str = Field(default="", description="생활 습관 및 비약물 치료")
    sec43: str = Field(default="", description="추적 검사 계획")
    sec5: str = Field(default="", description="종합 진단 소견")

llm    = ChatVertexAI(model="gemini-2.5-flash",
                      max_output_tokens=8192)
parser = PydanticOutputParser(pydantic_object=ReportSections)
chain  = llm | parser

result   = chain.invoke([SystemMessage(...), HumanMessage(...)])
sections = result.model_dump()

# 빈 섹션은 규칙 기반 폴백으로 자동 보완
for key in ("sec2","sec3","sec41","sec42","sec43","sec5"):
    if not sections.get(key):
        sections[key] = fallback[key]
```

default="" → 일부 섹션 누락 시 파싱 실패 방지 | max_output_tokens=8192 → 한국어 6개 섹션 전체 출력 보장 | 폴백 규칙 → LLM 장애 시에도 소견서 자동 생성

신부전 관리

트러블 슈팅

Key Troubleshooting

eGFR 피처 제외 버그

문제 predict()에서 eGFR 제외 → imputer 중앙값(~45)으로 대체 → Stage 2 오예측

해결 eGFR 포함 후 재학습 → 정확도 22% → 98.13% 회복

BGE-M3 meta tensor 오류

문제 PyTorch 2.6 + accelerate → 모델이 meta 디바이스 로드 → .to() 호출 충돌

해결 _BGEWrapper 구현: FlagEmbedding 실패 시 sentence-transformers 자동 폴백

병렬 임베딩 CUDA 충돌

문제 ThreadPoolExecutor 5스레드가 _get_embedder() 동시 호출 → CUDA 초기화 충돌

해결 CUDA_VISIBLE_DEVICES="" + threading.Lock 이중 잠금으로 해결

Vertex AI 모델 404

문제 gemini-1.5-flash-002, gemini-2.0-flash-001 → 2026.03 이후 접근 제한

해결 gemini-2.5-flash로 변경 (신규 프로젝트 GA 모델)

PydanticOutputParser 파싱 실패

문제 max_output_tokens=2048 → 한국어 6섹션 출력 중 sec2에서 강제 잘림

해결 max_output_tokens=8192 상향 + ReportSections 필드 default="" 추가

load_dotenv 환경변수 미적용

문제 셸에 export된 GEMINI_MODEL이 .env 값을 덮어씌워 변경사항 미반영

해결 load_dotenv(path, override=True) 적용

총 6개 주요 이슈 해결 | eGFR 정확도 22% → 98.13% | RAG 임베딩 안정화 | LLM 출력 파싱 안정화

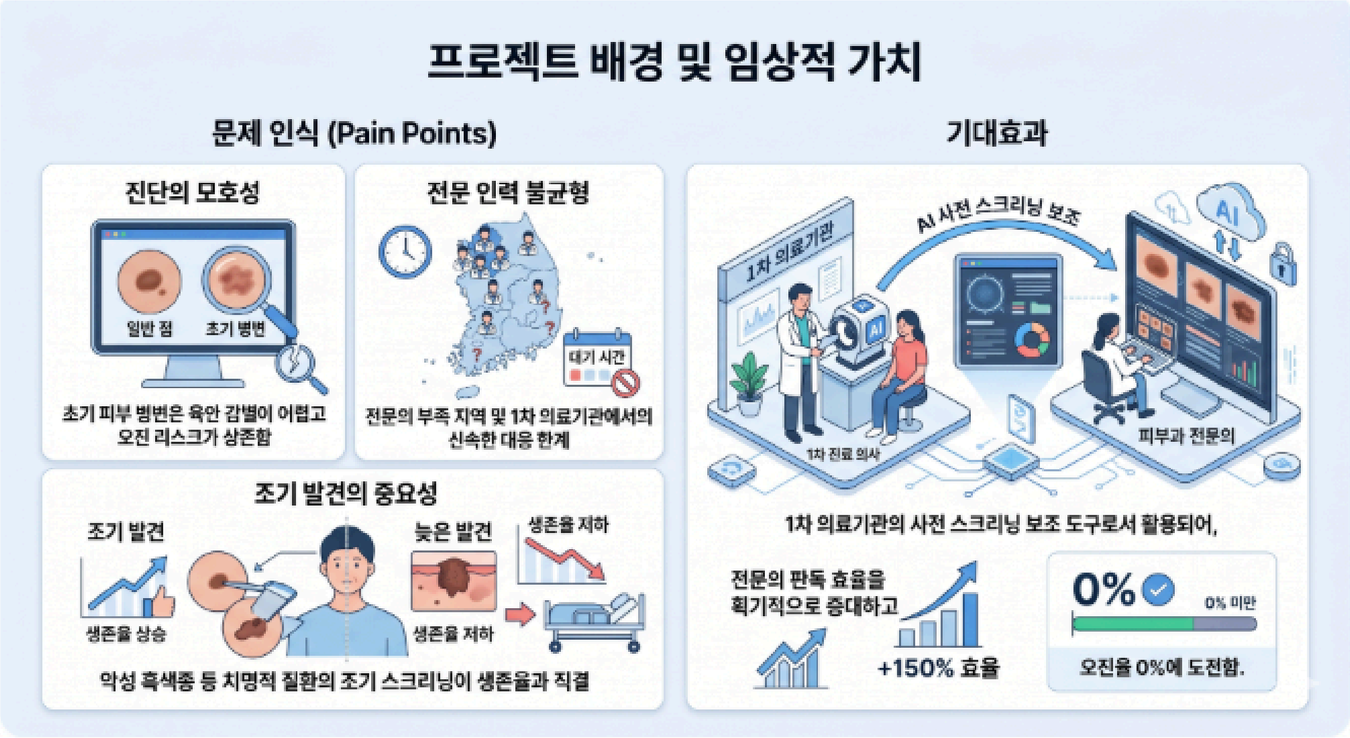
모델 구현 - 피부질환 분류 모델

작성자 : 김민수

프로젝트 배경 및 임상적 가치

문제 인식

- **진단의 모호성**: 초기 피부 병변은 육안 감별이 어렵고 오진 리스크가 상존함
- **전문 인력 불균형**: 전문의 부족 지역 및 1차 의료기관에서의 신속한 대응 한계 (전국 피부과 전문의 수 : 2,950명)
  약 피부과 전문의 1인당 인구 수는 약 17,288명
- **조기 발견의 중요성**: 악성 흑색종 등 치명적 질환의 조기 스크리닝이 생존율과 직결

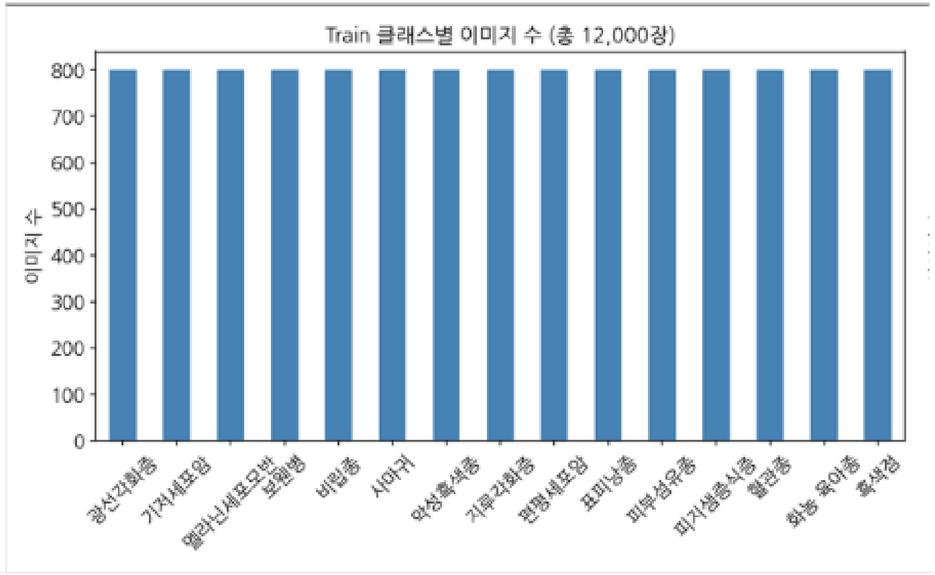


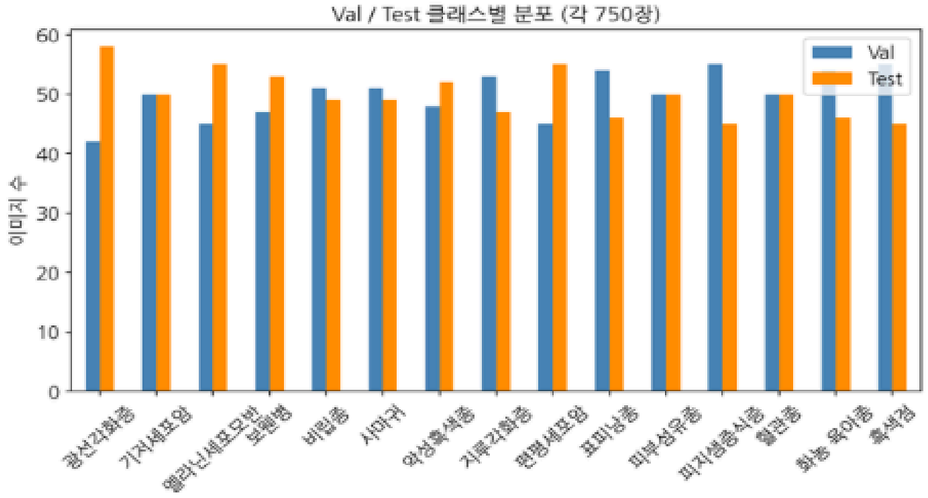
기대 효과

1차 의료기관의 **사전 스크리닝 보조 도구**로서 활용되어, 전문의 판독 효율을 획기적으로 증대하고 오진 보조 감소.

피부질환

데이터 구성 및 분포 분석





편향 없는 학습을 위한 데이터 설계

- **총 13,500장 확보**: Train(12,000장), Val/Test(각 750장) 구성
- **클래스 균등 분포**: 모든 클래스당 800장의 이미지를 학습에 할당하여 특정 질환 편향성 원천 차단
- **검증 데이터 무결성**: 클래스당 약 45~58장의 엄격한 Test set 구성을 통해 실질적인 일반화 성능 검증

* **의미**: 의료 AI에서 가장 치명적인 '다수 클래스 편향' 문제를 데이터 수준에서 해결함.

데이터셋 출처 : AI HUB 피부종양 이미지 데이터

https://aihub.or.kr/aihubdata/data/view.do?pageIndex=1&currMenu=115&topMenu=100&srchOptnCnd=OPTNCND001&searchKeyword=&srchDetailCnd=DETAILCND001&srchOrder=ORDER001&srchPagePer=20&srchDataRealmCode=REALM006&aihubDataSe=data&dataSetSn=71864

데이터 전처리 및 증강 전략

표준화 (Resize)

입력 크기 **224 × 224**로 리사이징하여 모델 연산 효율 및 EfficientNet-B3 아키텍처 최적화.

일반화 증강 (Augmentation)

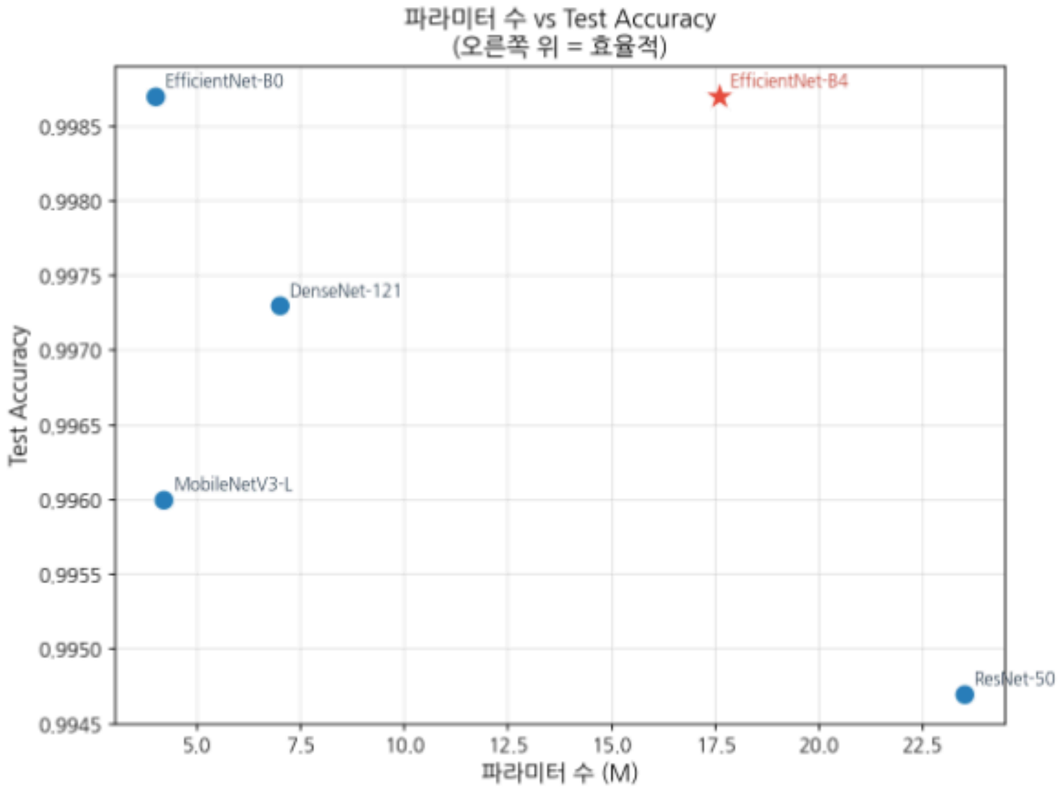
Flip, Rotation, **ColorJitter**를 적용하여 촬영 각도 및 조명 변화에 강건한(Robust) 특징 추출 유도.

정규화 (Normalization)

ImageNet mean/std 기준 정규화를 통해 사전학습 가중치와의 분포를 일치시키고 수렴 속도 가속화.

전략적 의도: 의료 이미지는 장비와 조명에 매우 민감함. 이를 인위적 변형(Augmentation)으로 보완함으로써, 실 진료 환경에서의 **도메인 적응성(Domain Adaptation)**을 확보함.

모델 및 학습 전략: EfficientNet-B4 선정 이유



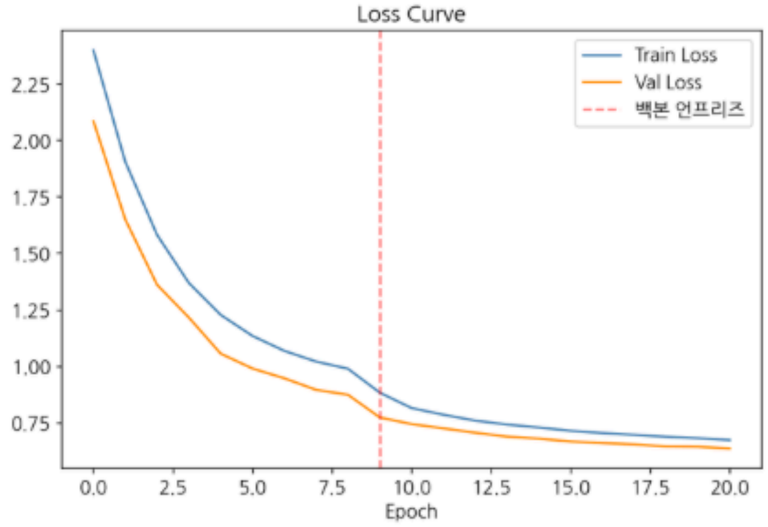
모델 성능 및 효율성 비교

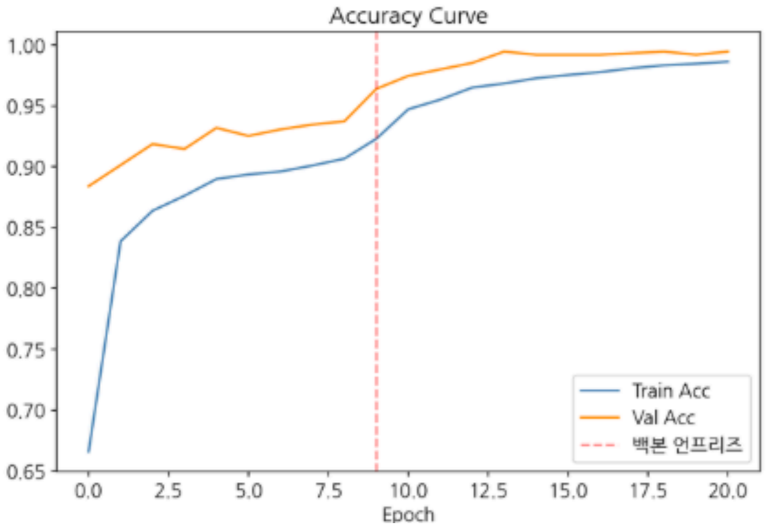
모델	Test Acc	파라미터 수	특이사항
ResNet-50	0.9947	25M	기준선 (Baseline)
DenseNet-121	**0.9973**	**7.0M**	**-**
MobileNetV3-L	0.9960	5M	경량이나 정확도 다소 낮음
EfficientNEt-B0	0.9987	5M	-
EfficientNet-B4	**0.9987**	**19M**	**최적 성능/크기 비율 (380x380)**

선정 근거: 의료 이미지의 복잡한 텍스처와 미세한 특징을 잡아내기 위해, 해상도(380x380)와 네트워크 깊이/너비를 최적으로 복합 스케일링한 **EfficientNet-B4**를 채택함.

피부질환

학습 프로세스 및 하이퍼파라미터





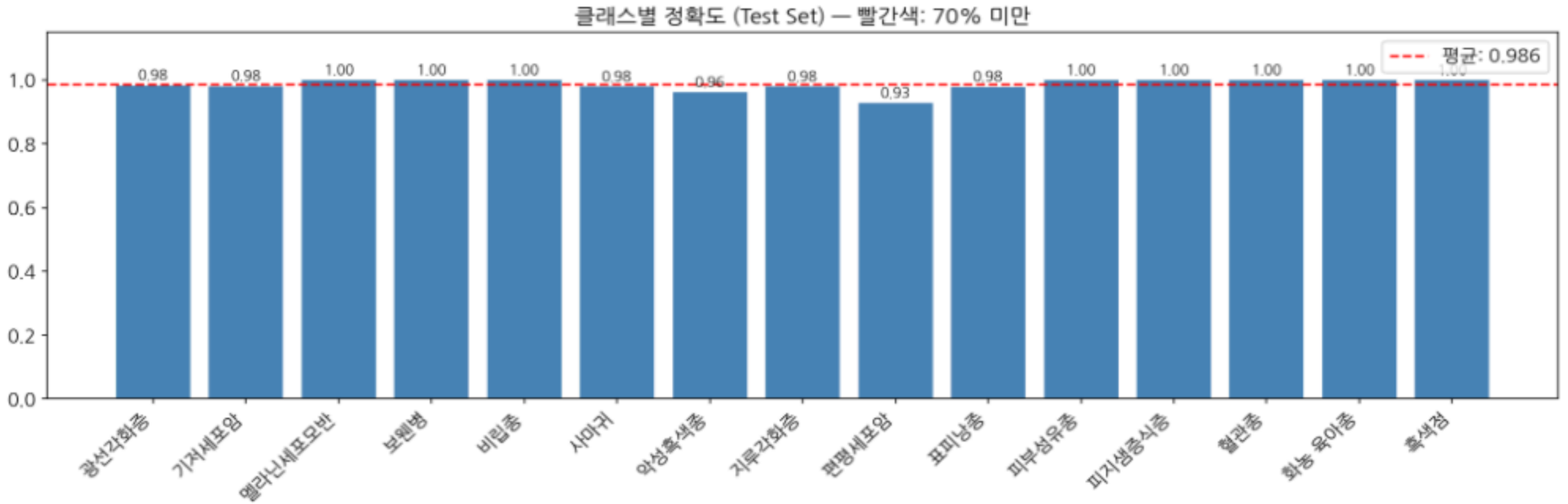
단계적 파인튜닝 (Fine-Tuning)

- **초기 Warm-up**: Epoch 0~8까지 헤드 부분만 학습하여 사전 학습 가중치 보호
- **백본 언프리즈**: **Epoch 9** 지점에서 가중치를 전면 해제하여 의료 도메인 특징 정밀 학습
- **성능 도약**: 언프리즈 직후 Accuracy가 0.96에서 0.99로 급격히 상승하며 수렴

결과: Train/Val Loss가 조화롭게 하강하며 과적합(Overfitting) 없이 최적의 일반화 지점 도달.

항목	Phase 1 (Epoch 1~9)	Phase 2 (Epoch 10~)
학습 대상	분류 헤드만	**전체 (백본 언프리즈)**
LR	1e-4	1e-5 (1/10 감소)
Optimizer	AdamW	AdamW
Weight Decay	1e-4	1e-4
Batch Size	16	16
Scheduler	CosineAnnealingLR	CosineAnnealingLR
AMP	Mixed Precision ✅	Mixed Precision ✅

정량적 성능 평가 및 지표 분석



주요 질환 지표 분석 (의료 관점)

질환	Accuracy	Recall	임상 중요도
악성흑색종	0.96	**0.96**	🔴 치명 — 놓치면 사망
편평세포암	0.93	**0.93**	🔴 치명 — 전이 위험
기저세포암	0.98	0.98	🟡 고위험
광선각화증	0.98	0.98	🟡 전암 병변

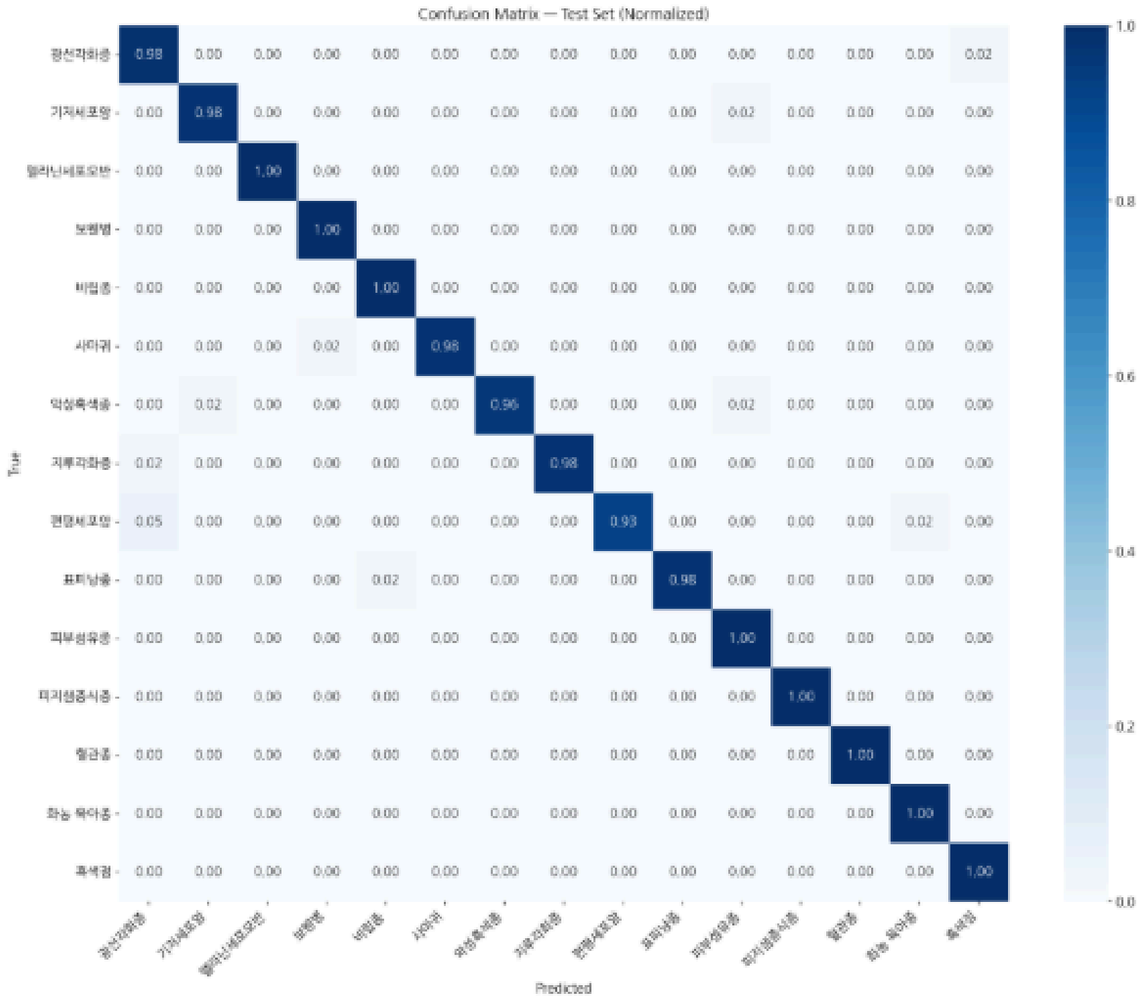
의료 AI 필수 고려사항

의료 AI에서는 **Accuracy보다 Recall(재현율)이 더 중요**합니다. 악성 질환을 놓치는 False Negative는 환자 생명에 직결되기 때문입니다.

악성흑색종 Recall 0.96 = 100명 중 4명만 미탐지 (매우 우수한 스크리닝 성능)

피부질환

혼동행렬 및 오류 케이스 심층 분석



전체 성능 요약

Test Accuracy 평균 98.6% 달성

15개 클래스 중 12개 클래스 1.00 또는 0.98 달성. 완벽에 가까운 대각선 형태를 보임.

주요 오분류 원인 분석

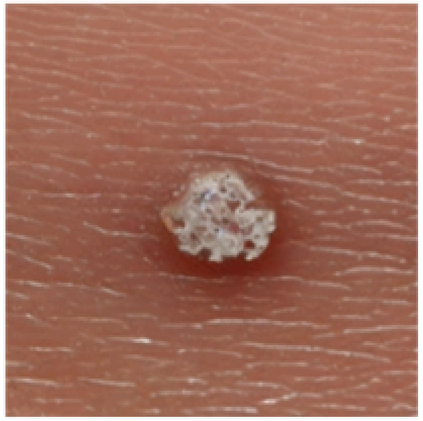
- **편평세포암(0.93) ↔ 광선각화증(0.98)**
  원인: 5% 혼동 발생. 두 질환은 초기 병변 형태가 극도로 유사함.
- **악성흑색종(0.96) ↔ 기저세포암(0.98)**
  원인: 2% 혼동 발생. 색소 침착 패턴이 육안으로도 헷갈리기 쉬움.

> **해석**: 오분류는 모두 임상적으로 감별이 어려운 질환 쌍 간에 발생함. 즉, 모델이 랜덤 오분류가 아닌 **의학적으로 설명 가능한 임상 패턴을 학습**했음을 의미함.

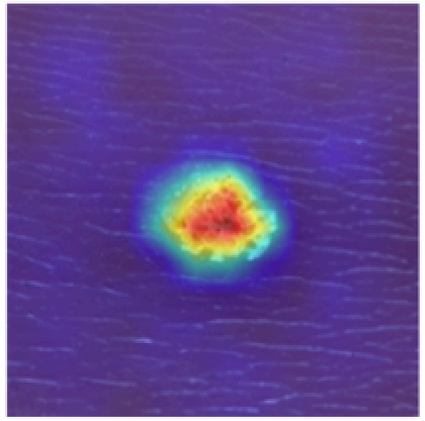
피부질환

Grad-CAM 시각화

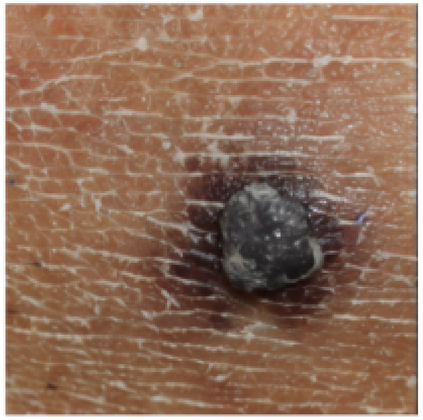
사마귀
원본



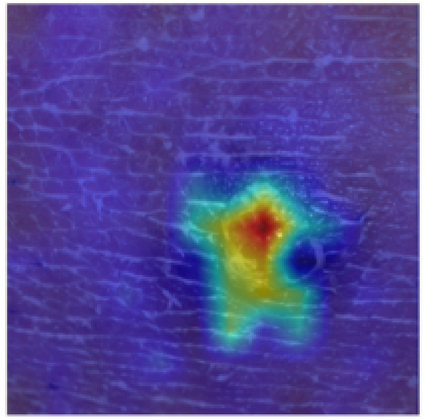
Grad-CAM



악성흑색종
원본



Grad-CAM



Grad-CAM의 역할

CNN 마지막 합성곱 레이어의 그래디언트를 역전파하여 분류에 기여한 영역을 히트맵으로 시각화. **블랙박스 모델의 판단 근거를 인간이 해석 가능하게 변환함.**

분석 결과 요약

질환	집중 영역	임상적 타당성
악성흑색종	병변 중심부 색소	멜라닌 농도 집중 부위
기저세포암	경계 + 혈관 패턴	진주 모양 경계가 특징
편평세포암	궤양·딱지 부위	표면 파괴 패턴
비립종	흰 구진 중심	각질 낭종 위치 일치

결론: 모델이 배경이 아닌 **실제 병변 부위에 집중**함을 확인. 과적합(Overfitting)이 아님을 시각적으로 증명하여 높은 정확도의 신뢰성 검증 완료.

피부질환

지능형 의료 리포트 파이프라인 (RAG)

[1. 이미지 입력]

EfficientNet-B4
→ 질환명 + 신뢰도 도출

↓

[2. 벡터 지식 검색]

pgvector (PostgreSQL)
WHERE module_tag='skin'
ORDER BY emb <=> query LIMIT 3

↓

[3. Grad-CAM 분석]

병변 집중 부위 히트맵
→ Gemini Vision으로 언어 설명 생성

↓

[4. 리포트 생성]

Gemini-2.5-Flash
RAG 지식 + Grad-CAM 설명
→ 임상 리포트 완성

```
## 피부과 AI 진단 보조 리포트

**환자 정보:** (정보 없음)
**진단 일자:** YYYY년 MM월 DD일

---

### [트리아지 등급]
**GREEN**
AI 진단 결과 사마귀(Wart)의 신뢰도가 96.4%로 매우 높으며, 편평세포암이나 보웬병과 같은 악성 질환의 가능성은 1% 내외로 낮게 평가되었습니다. 사마귀는 양성 질환으로, 즉각적인 응급 처치를 요하는 상황은 아닌 것으로 판단됩니다.

### [진단 소견]
AI는 96.4%의 높은 신뢰도로 병변을 사마귀(Wart)로 진단하였습니다. 모델은 병변 내 과각화되고 유두종성 증식을 보이는 붉은색 및 주황색 영역에 집중했으며, 이는 사마귀의 특징적인 거칠고 융기된 표면 소견과 일치합니다. 육안 소견상 피부색에서 갈회색을 띠며, 표면이 거칠고 융기된 과각화성 구진 또는 결절 형태로 나타날 수 있습니다.

### [감별 진단 고려사항]
주요 감별 진단으로는 편평세포암(Squamous Cell Carcinoma)과 보웬병(Bowen's Disease)이 제시되었습니다. 사마귀는 표면이 거칠고 유두종성 증식을 보이며, 병변 내 흑점(혈전성 모세혈관)이 관찰될 수 있는 양성 병변입니다. 반면 편평세포암과 보웬병은 진행성 또는 악성 병변으로, 사마귀와 유사한 과각화 병변으로 나타날 수 있으나, 비대칭성, 계양 형성, 또는 불규칙한 성장 양상을 보일 수 있어 정밀한 육안 및 조직학적 감별이 필요합니다.

### [권고 처치]
우선적으로 1차 표준 치료인 국소 살리실산 제제 도포 또는 냉동치료를 고려할 수 있습니다. 환자의 병변 특성(종류, 위치, 크기, 개수)과 면역 상태 등을 종합적으로 고려하여 개별화된 치료 계획을 수립해야 합니다. 확진 및 치료를 위해 피부과 전문의의 추가적인 진찰 및 진료가 권고됩니다.

### [참고 가이드라인]
본 리포트에서 제시된 치료 전략은 일반적인 사마귀 관리의 임상적 권고안 및 최신 지견과 일치합니다. 구체적인 치료 계획은 환자의 상태에 따라 맞춤형으로 적용되며, 대한피부과학회 또는 국제 피부과학회에서 발행하는 사마귀 치료 가이드라인을 참고할 수 있습니다.

---
※ 본 결과는 AI 보조 진단이며 최종 판단은 임상의가 내려야 합니다.
```

개발 과정 트러블슈팅

문제 1: "Test Accuracy 98.6% — 과적합인가?"

[발견 및 원인 분석]
Test 98.6%, Val 99% 달성. "데이터를 외운 것 아닌가?" 의심. 클래스당 800장 균등 세팅이 과적합에 유리한 환경을 제공. Loss 곡선 수치만으로는 신뢰성 입증 불충분.

[해결]
Grad-CAM 전수 분석 (15클래스 × 샘플) 수행. 전 클래스에서 배경이 아닌 실제 병변 부위에 집중함을 확인하여, 과적합이 아니며 실제 임상 패턴을 학습했음을 증명함.

[교훈]
정량 지표만으로 모델 신뢰성 판단 불가 → 설명 가능한 AI(XAI) 검증이 필수

문제 2: "임의 사진 입력 시 엉뚱한 분류"

[발견 및 원인 분석]
일반 사진 입력 시 관계없는 질환으로 분류됨. 학습 데이터가 **피부과 전용 기기(더마토스코프)** 촬영 이미지라 **도메인 갭(Domain Gap)** 발생.
Closed-set 구조상 무조건 15개 중 하나를 출력함.

[현재 대응]
서비스 UI에 "더마토스코프 촬영 이미지만 지원" 가이드 표시 및 이미지 품질 사전 검증 구현.

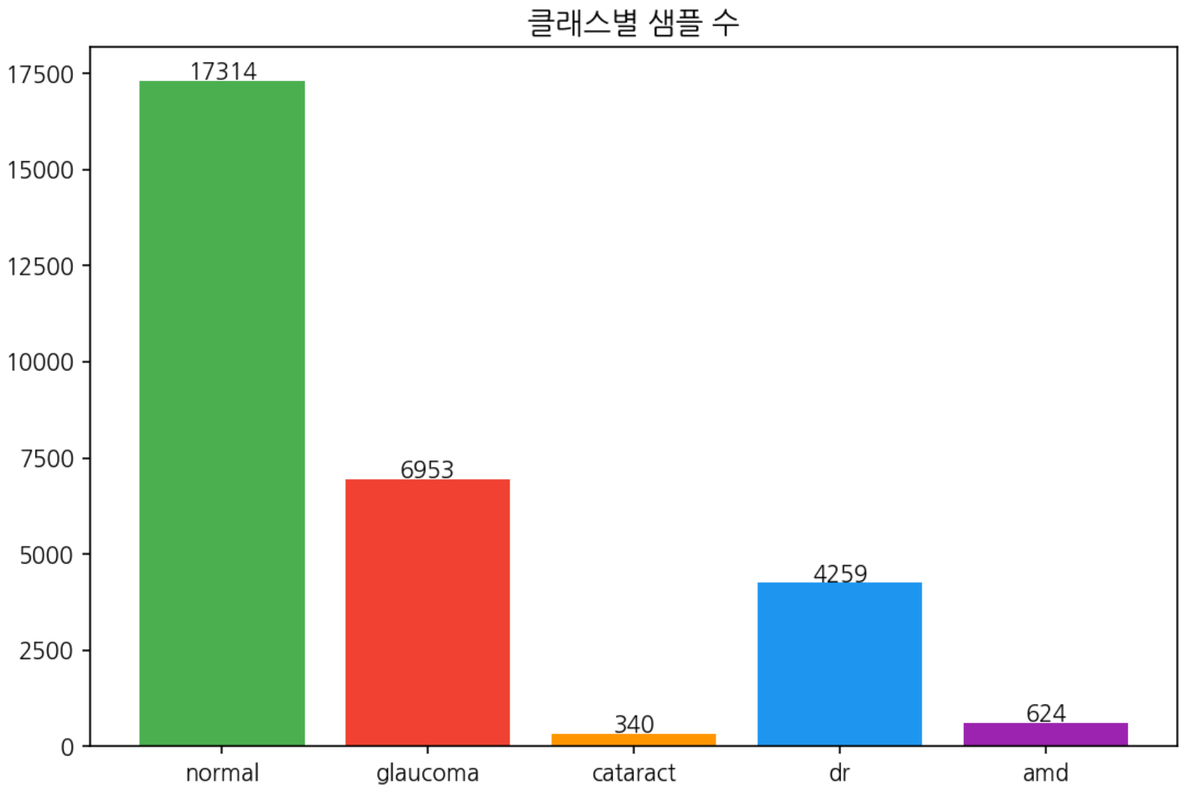
[향후 개선]
OOD(Out-of-Distribution) 탐지 모듈 추가 또는 도메인 적응(Domain Adaptation) 학습

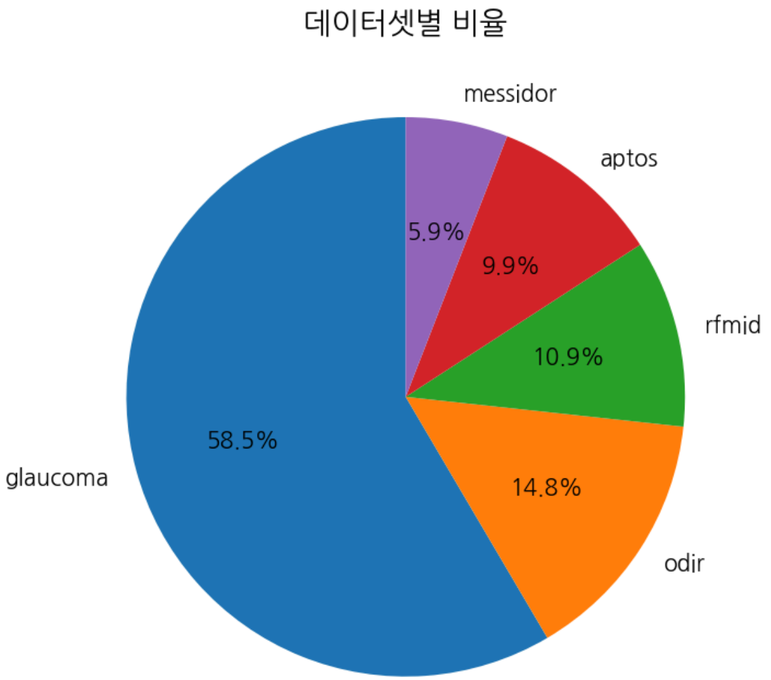
모델 구현 - 안과 진단 분류 모델

작성자 : 홍승현

안과 질환

데이터 분포 비율



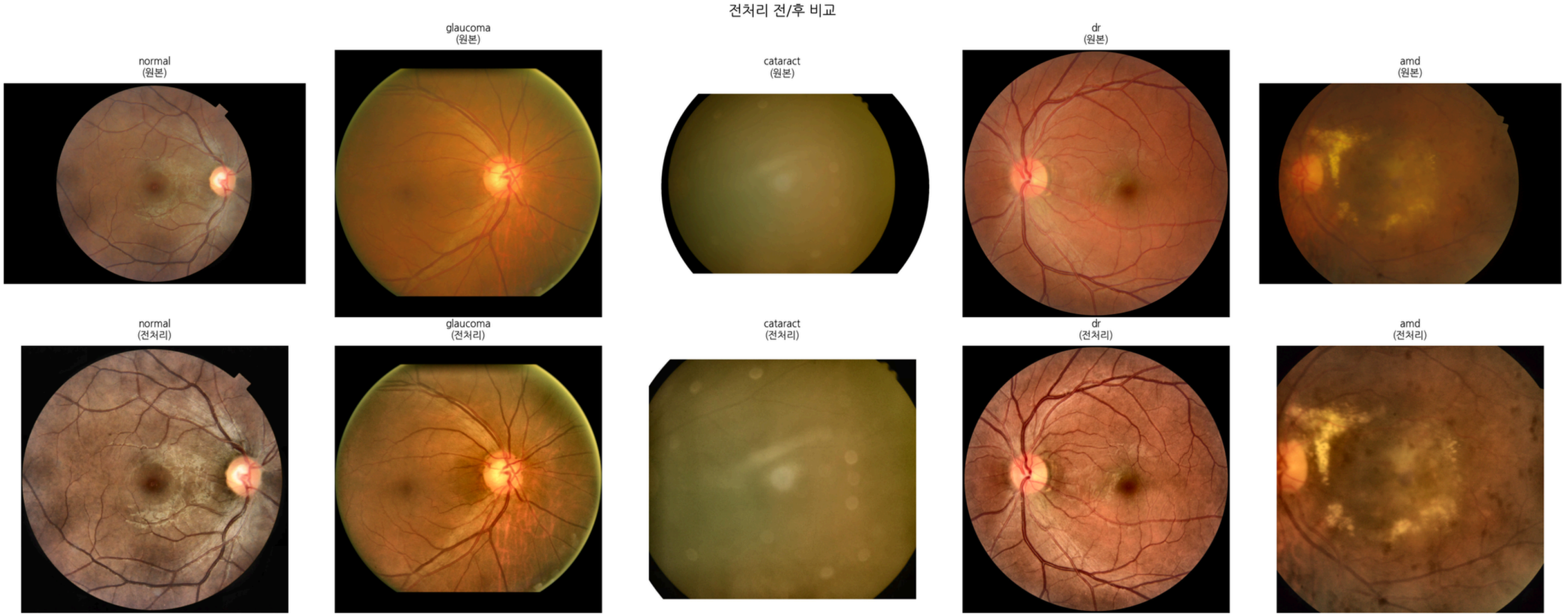


자료 출처

1. **APTOS-2019 DATASET** → HTTPS://WWW.KAGGLE.COM/DATASETS/MARIAHERREROT/APTOS2019
2. **FUNDUS GLAUCOMA DETECTION DATA [PYTORCH FORMAT]** →HTTPS://WWW.KAGGLE.COM/DATASETS/SABARI50312/FUNDUS-PYTORCH
3. **MESSIDOR-2** → HTTPS://WWW.KAGGLE.COM/DATASETS/MARIAHERREROT/MESSIDOR2PREPROCESS
4. **OCULAR DISEASE RECOGNITION** → HTTPS://WWW.KAGGLE.COM/DATASETS/ANDREWMVD/OCULAR-DISEASE-RECOGNITION-ODIR5K
5. **RETINAL DISEASE CLASSIFICATION** → HTTPS://WWW.KAGGLE.COM/DATASETS/ANDREWMVD/RETINAL-DISEASE-CLASSIFICATION

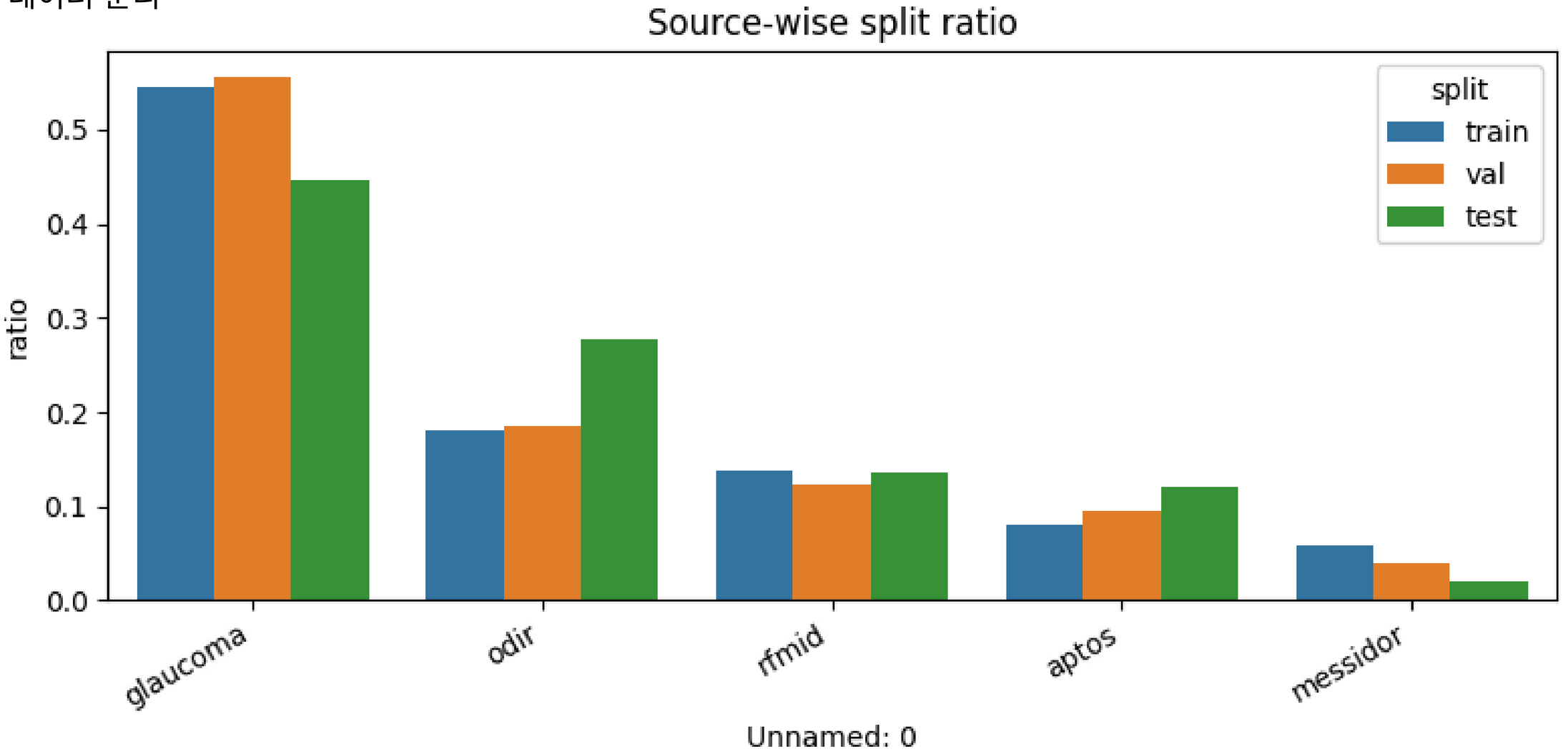
안과 질환

전처리


전처리 전/후 비교
normal (원본)
glaucoma (원본)
cataract (원본)
dr (원본)
amd (원본)
normal (전처리)
glaucoma (전처리)
cataract (전처리)
dr (전처리)
amd (전처리)

안과 질환

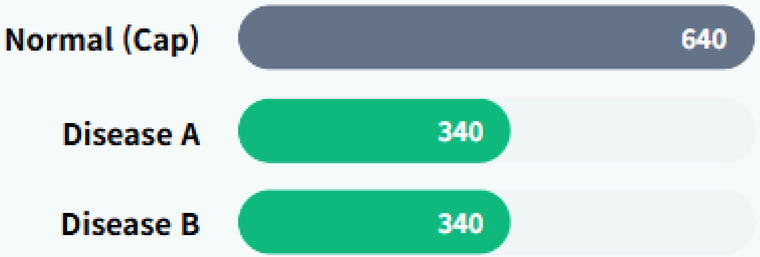
학습 데이터 분리



안과 질환

데이터 분리 의도

Class Distribution Strategy



*CAP_DISEASE=340 적용을 통한 Macro 지표 안정화

Leakage 차단 및 불균형 완화

- **환자 단위 분할:** 그룹키 source::patient_id 기반의 Group Split 적용
- **누수 검증:** Train/Test 교집합 0 확인 프로세스
- **Source 보정:** Test 세트의 Source 결손 방지 및 도메인 평가 확보

안과 질환

1차 학습 순위

Rank	Model (Backbone)	Val AUC (Macro)	Val Accuracy	Val F1-Score	Test AUC (Macro)
1	**Swin-Base (Winner)**	**0.9096**	**0.6393**	**0.6550**	**0.8402**
2	EfficientNet-B0	0.8317	0.5770	0.5470	0.7669
3	ConvNeXt-Tiny	0.7182	0.1574	0.0859	0.6967
4	ResNet-50	0.6359	0.1377	0.0484	0.6077

❗ **CNN 계열(ResNet/ConvNeXt)의 Sensitivity 붕괴 현상 확인 → Transformer 기반 Swin-Base 최종 선정**

안과 질환

최종 모델 학습

TEACHER MODEL

스승 모델 앙상블

- **모델:** Swin-Base (Top Performance)
- **전략:** 5-Fold Cross Validation 기반 앙상블
- **데이터 구성:** 총 6,360장의 안저 이미지 활용
- **역할:** 질병의 미세 패턴과 클래스 간의 상관관계를 매우 정교하게 학습 (High Logic)

지식 전수

(Logits Transfer)

STUDENT MODEL

제자 모델 경량화

- **모델:** EfficientNet-B0 (Lightweight)
- **학습 방식:** 스승 모델 5체의 예측값(Logits)을 합산하여 정답지로 활용
- **이점 1:** 스승 모델의 성능을 98% 이상 유지하면서 파라미터 수 대폭 감소
- **이점 2:** 실시간 CDSS 구동을 위한 최적의 추론 속도 확보

핵심 논리: 단순히 '정답'만 배우는 것이 아니라, 스승 모델이 판단한 '각 질병별 확률 분포'를 통째로 학습하여 제자 모델의 일반화 성능을 극대화합니다.

안과 질환

실시간 배포 및 UX 파이프라인

SSE(Server-Sent Events) 실시간 스트리밍 분석

전체 분석 과정을 7단계로 세분화하여, 사용자에게 실시간으로 진행 상황을 피드백합니다.

수신 → 전처리 → AI 추론 → XAI 생성 → 문헌 검색 → 복합 추론 → **결과 전송**

Dual-Model Serving

속도가 중요한 **추론(Inference)은 ONNX**로, 시각화가 중요한 **Grad-CAM은 PTH** 모델로 이원화하여 운영 효율을 극대화합니다.

SSE 스트리밍의 이점

중간 로딩 시간을 방치하지 않고 실시간 상태를 노출하여 사용자가 체감하는 대기 시간을 혁신적으로 단축합니다.

안과 질환

현재 진행 상황

전처리 파이프라인
NB01/012 완료

백본 벤치마크
Winner: Swin-Base

HPO (NB05)
진행 중 (D+1 목표)

Teacher / KD
예정 (NB07)

Calibration
예정 (NB013)

ONNX 배포
예정 (NB014)

"실패 재발 방지를 위한 Trouble Shooting #20~36 기록 완료"

테스트 - 단위테스트

작성자 : 김남준

단위 테스트

개요

6개 AI 진단 모듈의 핵심 기능 검증 및 품질 보증

59개 총 단위테스트

48개 총 시나리오

100% 완료율 (5/5)

3계층 AI · FE · BE

모듈별 테스트 현황

모듈	담당자	테스트	시나리오	상태
brain (뇌종양)	변운조	8	8	완료
spine (허리디스크)	김담현	8	8	완료
skin (피부질환)	김민수	8	8	완료
kidney (신부전)	김남준	27	27	완료
eyes (안과질환)	홍승현	8	8	완료

테스트 핵심 목표

- **기능 검증** 시스템 주요 기능의 정상 작동 확인
- **품질 보증** 잠재적 결함 조기 발견 및 해결
- **사용자 경험** 서비스 안정성 기반의 신뢰 확보
- **지속 가능성** 코드 수정 후 회귀 오류 방지

테스트 레이어

AI 레이어	모델 예측 정확도 · 트리아지 분류 · 저화질 이미지 거부 · 헬스체크
Frontend 레이어	파일 형식·크기 검증 · 진단 결과 렌더링 · DONE 환자 차단
Backend 레이어	진단 결과 저장 (POST /diagnoses) · 환자별 필터링

단위 테스트

시나리오

시나리오 ID	담당자	모듈	시나리오명	구분	비고
SC-WJ-01	변운조	brain	뇌 MRI 이미지 정상 진단 흐름 검증	AI	종양명·신뢰도·Grad-CAM 반환
SC-WJ-02	변운조	brain	악성 종양 트리아지 RED 분류 검증	AI	악성 클래스 예측 시 RED 설정
SC-WJ-03	변운조	brain	저화질 MRI 입력 시 진단 거부 처리	AI	quality_check 노드 동작
SC-WJ-04	변운조	brain	파일 형식·크기 클라이언트 유효성 검증	Frontend	JPG/PNG만 허용, 5MB 이하
SC-WJ-05	변운조	brain	진단 결과 화면 정상 렌더링	Frontend	뱃지·질환명·Grad-CAM 출력
SC-WJ-06	변운조	brain	진단 결과 백엔드 저장	Frontend	diagnoses API 연동
SC-WJ-07	변운조	brain	진료 완료 환자 분석 차단	Frontend	DONE 상태 시 업로드 비활성화
SC-WJ-08	변운조	brain	AI 서버 헬스체크	Backend	GET /brain/health 응답
SC-DH-01	김담현	spine	척추 MRI + 문진표 통합 분석 흐름 검증	AI	vision_analysis(3질환×3중증도)·LangGraph 리포트 반환
SC-DH-02	김담현	spine	Red Flag 응급 감지 검증	AI	대소변 장애 등 입력 시 응급 처치 권고 포함
SC-DH-03	김담현	spine	중증(Severe) 판정 시 소견 반영 확인	AI	Spinal Canal Stenosis Severe 확률 높을 때 medical_note 반영
SC-DH-04	김담현	spine	이미지 미선택 시 분석 버튼 비활성 검증	Frontend	이미지 없으면 버튼 disabled, API 미호출
SC-DH-05	김담현	spine	분석 결과 화면 final_report 렌더링	Frontend	"AI CDSS Final Report" 섹션에 리포트 텍스트 출력
SC-DH-06	김담현	spine	진단 결과 백엔드 저장	Frontend	diagnoses API 연동
SC-DH-07	김담현	spine	서버 연결 실패 시 템플릿 폴백 처리	Frontend	AI 서버 미응답 시 detailedResults 템플릿으로 화면 표시
SC-DH-08	김담현	spine	새 분석 초기화 동작 검증	Frontend	"새로운 영상 분석하기" 클릭 시 상태 초기화, 업로드 화면 복귀

단위 테스트

시나리오

시나리오 ID	담당자	모듈	시나리오명	구분	비고
SC-NJ-01	김남준	kidney	모델 파일 정상 로딩 검증	Backend	pkl 파일(model, scaler, imputer) 로딩 확인
SC-NJ-02	김남준	kidney	극저값 입력 시 Stage5 예측 검증	AI	eGFR 극저값 입력 시 Stage5 예측 확인
SC-NJ-03	김남준	kidney	정상값 입력 시 Normal_Stage1 예측 검증	AI	정상 수치 입력 시 Normal_Stage1 예측 확인
SC-NJ-04	김남준	kidney	누락값 중앙값 대체 후 정상 예측 검증	AI	일부 수치 None 입력 시 imputer 중앙값 대체 확인
SC-NJ-05	김남준	kidney	모델 파일 미존재 시 예외 처리 검증	Backend	pkl 파일 미존재 시 FileNotFoundError, 서버 크래시 없음 확인
SC-NJ-06	김남준	kidney	eGFR CKD-EPI 2021 공식 적용 검증	AI	race-free 공식으로 eGFR 정확히 계산되는지 확인
SC-NJ-07	김남준	kidney	성별 분기 적용 검증	AI	여성(F) 기준 kappa·sexFactor 적용 시 남성보다 높은 값 반환 확인
SC-NJ-08	김남준	kidney	예외 입력값 NaN/null 처리 검증	Frontend	크레아티닌 0 또는 음수 입력 시 NaN/null 처리, 화면 에러 없음 확인
SC-NJ-09	김남준	kidney	위험 알림 트리거 임계값 검증	AI	칼륨 ≥ 5.5, eGFR < 15 등 임계값 초과 시 alerts 생성 확인
SC-NJ-10	김남준	kidney	저장 모달 동작 검증	Frontend	저장 버튼 클릭 시 확인 모달 표시, 취소 시 저장 미실행 확인
SC-NJ-11	김남준	kidney	임베딩 벡터 생성 검증	AI	embed_query() 호출 시 768차원 list[float] 반환 확인
SC-NJ-12	김남준	kidney	DB 연결 실패 시 예외 처리 검증	Backend	DB URL 오류 시 빈 리스트 반환, 예외 없이 서버 유지 확인
SC-NJ-13	김남준	kidney	RAG 검색 결과 중복 제거 검증	AI	multi_search() 동일 content(앞 80자) 중복 제거 확인

단위 테스트

시나리오

시나리오 ID	담당자	모듈	시나리오명	구분	비고
SC-MS-01	김민수	skin	피부 이미지 정상 진단 흐름 검증	AI	질환명·신뢰도·Top3·Grad-CAM 반환
SC-MS-02	김민수	skin	악성 질환 트리아지 RED 분류 검증	AI	악성흑색종 등 예측 시 RED 설정
SC-MS-03	김민수	skin	저화질 이미지 입력 시 진단 거부 처리	AI	quality_check 노드 동작
SC-MS-04	김민수	skin	파일 형식·크기 클라이언트 유효성 검증	Frontend	JPG/PNG만 허용, 5MB 이하
SC-MS-05	김민수	skin	진단 결과 화면 정상 렌더링	Frontend	뱃지·질환명·Grad-CAM 비교 출력
SC-MS-06	김민수	skin	진단 결과 백엔드 저장	Frontend	diagnoses API 연동
SC-MS-07	김민수	skin	진료 완료 환자 분석 차단	Frontend	DONE 상태 시 업로드 비활성화
SC-MS-08	김민수	skin	AI 서버 헬스체크	Backend	GET /skin/health 응답
SC-SH-01	홍승현	eyes	안저 이미지 정상 진단 흐름 검증	AI	질환명·신뢰도·Grad-CAM 반환
SC-SH-02	홍승현	eyes	실명 위험 질환 트리아지 RED 분류 검증	AI	중증 안질환 예측 시 RED 설정
SC-SH-03	홍승현	eyes	저화질 안저 이미지 입력 시 진단 거부 처리	AI	quality_check 노드 동작
SC-SH-04	홍승현	eyes	파일 형식·크기 클라이언트 유효성 검증	Frontend	JPG/PNG만 허용, 5MB 이하
SC-SH-05	홍승현	eyes	진단 결과 화면 정상 렌더링	Frontend	뱃지·질환명·Grad-CAM 출력
SC-SH-06	홍승현	eyes	진단 결과 백엔드 저장	Frontend	diagnoses API 연동
SC-SH-07	홍승현	eyes	진료 완료 환자 분석 차단	Frontend	DONE 상태 시 업로드 비활성화
SC-SH-08	홍승현	eyes	AI 서버 헬스체크	Backend	GET /eyes/health 응답

단위 테스트

테스트

테스트 No	TC 명	케이스 내용	사전 조건	테스트 데이터	예상 결과	수행 결과	수행자
UT-NJ-01	TabNet 정상 예측	혈액 수치 9개 전체 입력 시 CKD 단계 및 신뢰도 반환	pkl 파일 존재, 서버 기동 완료	sc=2.1, bu=35, egfr=45, pot=4.2, bp=145, hemo=10.5, bgr=115, al=3.6, sod=137	prediction="Stage3", confidence 0.0~1.0 범위 내 반환	○	김남준
UT-NJ-02	Stage5 예측 검증	eGFR 극저값 입력 시 Stage5 예측 확인	pkl 파일 존재	sc=8.5, bu=90, egfr=8, pot=5.8, bp=180, hemo=7.5, bgr=200, al=2.8, sod=130	prediction="Stage5", confidence ≥ 0.8	○	김남준
UT-NJ-03	누락값 imputer 대체	일부 수치 None 입력 시 중앙값 대체 후 정상 예측	Imputer 흡련 완료	sc=None, bu=None, egfr=30, 나머지 정상값	예외 없이 prediction, confidence 반환	○	김남준
UT-NJ-04	신뢰도 범위 검증	다양한 입력에서 신뢰도가 항상 0.0~1.0 범위	pkl 파일 존재	5가지 다른 입력 세트	모든 케이스 confidence ∈ [0.0, 1.0]	○	김남준
UT-NJ-05	pkl 파일 미존재	pkl 파일 삭제 후 predict() 호출 시 예외 처리	pkl 파일 없음	임의 입력값	FileNotFoundError 또는 RuntimeError 발생, 서버 크래시 없음	○	김남준
UT-NJ-06	피처 가중치 적용	가중치 적용 후 egfr 컬럼이 다른 컬럼보다 크게 스케일됨	Scaler 흡련 완료	표준화된 입력 배열	egfr 컬럼 값 × 2.5 적용 확인	○	김남준
UT-NJ-07	eGFR 남성 계산	남성 기준 CKD-EPI 2021 공식 결과 검증	브라우저 실행	sc=1.0, age=50, sex='M'	약 85 mL/min (±5 허용)	○	김남준
UT-NJ-08	eGFR 여성 계산	여성 기준 공식 적용 — 동일 조건 남성보다 높은 값 반환	브라우저 실행	sc=1.0, age=50, sex='F'	남성 결과보다 높은 값, sexFac=1.012 적용 확인	○	김남준
UT-NJ-09	eGFR 이상 입력	크레아티닌 0 또는 음수 입력 시 예외 처리	브라우저 실행	sc=0, age=50, sex='M' / sc=-1.5, age=50, sex='M'	NaN 또는 null 반환, 화면 에러 없음	○	김남준
UT-NJ-10	고칼륨 알림 트리거	칼륨 5.5 이상 입력 시 위험 알림 포함 여부	진단 결과 화면	pot=6.0 포함 진단 결과	alerts 배열에 level="critical" 항목 존재	○	김남준
UT-NJ-11	투석 알림 트리거	Stage5 진단 시 투석 필요 알림 표시	진단 결과 화면	egfr=8 기반 Stage5 결과	dialysis_required=true, 투석 배너 화면 표시	○	김남준
UT-NJ-12	저장 모달 표시	저장 버튼 클릭 시 확인 모달 표시	진단 결과 화면, 저장 전	저장 버튼 클릭 이벤트	showSaveModal=true, 모달 렌더링	○	김남준
UT-NJ-13	저장 모달 취소	모달 취소 버튼 클릭 시 닫힘 확인	저장 모달 표시 중	취소 버튼 클릭	showSaveModal=false, 저장 미실행	○	김남준

단위 테스트

테스트

테스트 No	TC 명	케이스 내용	사전 조건	테스트 데이터	예상 결과	수행 결과	수행자
UT-NJ-14	embed_query 벡터 차원	텍스트 입력 시 1024차원 float 벡터 반환	BGE-M3 또는 Vertex AI 임베딩 모델 로드	"CKD Stage 4 treatment management"	len(result)==1024, 타입 list[float]	○	김남준
UT-NJ-15	DB 연결 실패 처리	DB URL 오류 시 빈 리스트 반환	DB_URL 환경변수를 잘못된 값으로 임시 변경	"CKD management"	[] 반환, 예외 없음, 서버 유지	○	김남준
UT-NJ-16	중복 제거 검증	동일 content 반환 쿼리 병렬 실행 시 중복 제거	DB 정상, 임베딩 모델 로드	동일 결과 반환 유도 쿼리 3개	결과 리스트 내 content 앞 80자 기준 중복 없음	○	김남준
UT-NJ-17	출처 다양성 보장	동일 출처 결과가 최대 2건 이하인지 확인	DB 정상, 임베딩 모델 로드	동일 출처 집중 쿼리	source_count[src] ≤ 2, 결과 ≤ 6건	○	김남준
UT-NJ-18	빈 쿼리 처리	빈 리스트 입력 시 예외 없이 빈 리스트 반환	임베딩 모델 로드	queries=[]	[] 반환, 예외 없음	○	김남준
UT-NJ-19	BGEWrapper 폴백	FlagEmbedding import 실패 시 sentence-transformers 전환	FlagEmbedding 미설치 환경	_BGEWrapper() 인스턴스 생성	self._backend=="sbert", encode() 정상 동작	○	김남준
UT-NJ-20	임베딩 싱글톤 잠금	다중 스레드 동시 호출 시 단 1회만 초기화	임베딩 미초기화 상태	ThreadPoolExecutor 5스레드 동시 _get_embedder() 호출	초기화 1회, 이후 동일 인스턴스 반환	○	김남준
UT-NJ-21	섹션1 수치 포함	입력 수치가 섹션1 문자열에 정확히 포함	없음	prediction="Stage4", confidence=0.92, egfr=18.0, sc=4.5	반환 문자열에 "18.0", "4.5", "4단계" 포함	○	김남준
UT-NJ-22	폴백 섹션 단계 분기	Stage4 폴백 섹션에 투석 관련 내용 포함	없음	prediction="Stage4", contexts=[], confidence=0.85	sec41에 "ACE 억제제" 또는 "투석" 포함, 6개 키 모두 존재	○	김남준
UT-NJ-23	소견서 조립 구조	최종 소견서에 5개 섹션 제목 모두 포함	section1 문자열, sections dict 준비	모든 섹션 내용 포함 dict	"1. 핵심 수치", "2. 위험도", "3. 이상 소견", "4. 치료", "5. 종합" 포함	○	김남준
UT-NJ-24	ReportSections 기본값	sec2만 포함한 dict로 인스턴스 생성 시 나머지 필드 기본값	없음	{"sec2": "위험도 내용"}	sec3~sec5 모두 "", ValidationError 미발생	○	김남준
UT-NJ-25	LLM 분기 — Vertex AI	GCP_PROJECT_ID 실제 값 설정 시 ChatVertexAI 경로 실행	GCP_PROJECT_ID 유효값, Vertex AI 연결 가능	query="Stage3 치료", prediction="Stage3"	_get_gemini_llm() 호출, LangChain 체인 실행	○	김남준
UT-NJ-26	LLM 분기 — 폴백	GCP_PROJECT_ID=placeholder 시 규칙 기반 소견서 반환	GCP_PROJECT_ID=placeholder	query="Stage3 치료", prediction="Stage3"	build_fallback_sections() 실행, LLM 호출 없음	○	김남준
UT-NJ-27	LLM 실패 시 섹션 보완	LLM 파싱 실패 시 빈 섹션을 폴백으로 채움	LLM 불완전 JSON 반환 mock 설정	부분 섹션만 포함된 LLM 응답	빈 섹션 폴백 대체, 최종 소견서 6개 섹션 모두 비어 있지 않음	○	김남준

단위 테스트

테스트

테스트 No	TC 명	케이스 내용	사전 조건	테스트 데이터	예상 결과	수행 결과	수행자
UT-WJ-01	NIfTI 정상 진단	유효한 .nii 파일 업로드 시 종양 판정·신뢰도·하이라이트 슬라이스 정상 반환	모델 가중치 존재, 서버 실행 중	정상 T1 MRI .nii 파일	HTTP 200, classification(Tumor/Normal)·confidence·슬라이스 URL 포함 반환	○	변운조
UT-WJ-02	종양 감지 시 마스크 생성	Tumor 판정 + fileId 제공 시 mask.nii 및 하이라이트 PNG 슬라이스 자동 생성	UT-WJ-01 완료, fileId 존재	Tumor 판정 .nii + fileId	mask_ready=true, highlight_slice_counts 포함 응답	○	변운조
UT-WJ-03	비NIfTI 파일 거부	.jpg/.png 등 NIfTI 외 파일 업로드 시 400 에러 반환	서버 실행 중	test.jpg	HTTP 400, "NIfTI 파일만 허용됩니다" 메시지 반환	○	변운조
UT-WJ-04	빈 파일 거부	크기 0 바이트 .nii 파일 업로드 시 400 에러 반환	서버 실행 중	empty.nii (0 bytes)	HTTP 400, "빈 파일입니다" 메시지 반환	○	변운조
UT-WJ-05	슬라이스 PNG 생성	POST /ai/brain/slices 호출 시 axial/coronal/sagittal 슬라이스 PNG 정상 생성	/upload 선행 완료, fileId 존재	fileId + axis="axial"	sliceCount > 0, baseUrl 포함 응답	○	변운조
UT-WJ-06	진단 결과 화면 렌더링	진단 완료 후 종양 판정·신뢰도·3축 슬라이스 뷰어가 화면에 정상 출력	UT-WJ-01 선행 완료	Tumor 판정 결과	종양 판정 뱃지, 신뢰도 %, 슬라이스 이미지 3축 표시	○	변운조
UT-WJ-07	진단 결과 저장	저장 버튼 클릭 시 POST /api/patients/{uid}/diagnoses 호출 후 성공 토스트 표시	로그인 상태, 환자 선택 완료	UT-WJ-01 결과	HTTP 201, "저장 완료" 토스트, 이력 갱신	○	변운조
UT-WJ-08	AI 서버 헬스체크	GET /ai/brain/health 호출 시 서버 상태 정상 응답 확인	AI 서버 실행 중	-	HTTP 200, {"module": "brain", "status": "ok"} 반환	○	변운조

단위 테스트

테스트

테스트 No	TC 명	케이스 내용	사전 조건	테스트 데이터	예상 결과	수행 결과	수행자
UT-MS-01	피부 이미지 정상 진단	유효한 피부 이미지 업로드 시 질환명·신뢰도·Top3·Grad-CAM 정상 반환	skin_efficientnet_b4.pth 존재, 서버 실행 중	정상 JPG 피부 이미지 (1MB)	HTTP 200, disease_ko·confidence·top3·gradcam_b64 포함 반환	○	김민수
UT-MS-02	악성 질환 트리아지 RED	악성흑색종·기저세포암 예측 시 triage_level=RED 설정 확인	모델이 악성 클래스 최고 확률 반환	class_idx=6 (악성흑색종)	triage_level="RED", 보고서에 RED 등급 명시	○	김민수
UT-MS-03	저화질 이미지 거부	블러·노이즈 심한 이미지 입력 시 quality_warning 포함 응답, 진단 미수행	-	블러 처리 피부 이미지	quality_warning 필드 포함, classify_node 미실행	○	김민수
UT-MS-04	파일 형식·크기 검증	GIF 또는 5MB 초과 파일 업로드 시 클라이언트에서 차단, API 미호출	브라우저 접근	test.gif, large.jpg (6MB)	오류 토스트 표시, POST 요청 미발생	○	김민수
UT-MS-05	진단 결과 화면 렌더링	진단 완료 후 트리아지 뱃지·질환명·신뢰도·Grad-CAM 비교 이미지 정상 출력	UT-MS-01 선행 완료	정상 JPG 피부 이미지	트리아지 뱃지, 질환명, 신뢰도 %, Grad-CAM 원본·히트맵 나란히 표시	○	김민수
UT-MS-06	진단 결과 저장	저장 버튼 클릭 시 POST /api/patients/{uid}/diagnoses 호출 후 성공 토스트 표시	로그인 상태, 환자 선택 완료	UT-MS-01 결과	HTTP 201, "저장 완료" 토스트, 이력 갱신	○	김민수
UT-MS-07	진료 완료 환자 차단	patientStatus=DONE 환자 선택 시 이미지 업로드 영역 비활성화	환자 상태=DONE	DONE 상태 환자	업로드 섹션 미표시, 분석 불가	○	김민수
UT-MS-08	AI 서버 헬스체크	GET /skin/health 호출 시 서버 상태 정상 응답 확인	AI 서버 실행 중	-	HTTP 200, {"status": "ok"} 반환	○	김민수

단위 테스트

테스트

테스트 No	TC 명	케이스 내용	사전 조건	테스트 데이터	예상 결과	수행 결과	수행자
UT-SH-01	안저 이미지 정상 진단	유효한 안저 이미지 업로드 시 질환명·신뢰도·GradCAM·소견서 정상 반환	모델 가중치 존재, 서버 실행 중	정상 안저 JPG + patient_id	HTTP 200, dl_result·gradcam_base64·report 포함 반환	○	홍승현
UT-SH-02	응급 질환 Emergency 알림	중증 안과 질환 예측 시 emergency.is_emergency=true 및 응급 알림 SSE 이벤트 발생	고위험 안저 이미지 입력	중증 당뇨망막병증 안저 이미지	SSE "emergency" 이벤트 발생, is_emergency=true	○	홍승현
UT-SH-03	안저 외 이미지 OOD 거부	안저가 아닌 이미지(피부 등) 입력 시 IMAGE_NOT_FUNDUS 에러 반환	서버 실행 중	피부 이미지 JPG	HTTP 400, error_code=IMAGE_NOT_FUNDUS 반환	○	홍승현
UT-SH-04	저화질 이미지 품질 거부	초점 불량·노이즈 심한 이미지 입력 시 품질 검증 실패 에러 반환	서버 실행 중	블러 처리 안저 이미지	HTTP 400, 품질 검증 실패 에러 메시지 반환	○	홍승현
UT-SH-05	RAG 문헌 Citation 반환	분석 완료 시 AAO 가이드라인 기반 근거 문헌 최소 1건 이상 포함	hybrid_retriever 로드 완료	정상 안저 이미지	citations 리스트 len ≥ 1, source="AAO PPP" 포함	○	홍승현
UT-SH-06	진단 결과 화면 렌더링	분석 완료 후 질환명·신뢰도·GradCAM·소견서·근거 문헌이 화면 정상 출력	UT-SH-01 선행 완료	정상 안저 이미지	질환명, 신뢰도 %, GradCAM 히트맵, 소견서, 문헌 Citation 표시	○	홍승현
UT-SH-07	진단 결과 저장	저장 버튼 클릭 시 POST /api/patients/{uid}/diagnoses 호출 후 성공 토스트 표시	로그인 상태, 환자 선택 완료	UT-SH-01 결과	HTTP 201, "저장 완료" 토스트, 이력 갱신	○	홍승현
UT-SH-08	AI 서버 헬스체크	GET /sh/health 호출 시 서버 상태 및 모델 로드 여부 정상 응답 확인	AI 서버 실행 중	-	HTTP 200, {"status": "ok", "module": "eyes", "model_loaded": true} 반환	○	홍승현

단위 테스트

테스트

테스트 No	TC 명	케이스 내용	사전 조건	테스트 데이터	예상 결과	수행결과	수행자
TC-DH-01	정상 통합 분석 흐름 검증	정상적인 MRI 이미지와 문진표를 전송했을 때 전체 분석 파이프라인이 정상 동작하는지 확인	React, Spring Boot, FastAPI 서버 및 DB 정상 구동 중	척추 MRI 이미지 (JPG) + 일반 문진 데이터	HTTP 200 응답 vision_analysis, medical_note, final_report 객체 정상 반환	○	김담현
TC-DH-02	Red Flag 응급 감지 검증	문진표에 Red Flag(응급 징후)가 포함되었을 때 LLM이 이를 인지하고 경고하는지 확인	LangGraph 및 Gemini API 연동 완료	척추 MRI 이미지 + Red Flag 문진 체크	final_report 내에 '즉각적인 수술 검토' 등 응급 대처 권고 문구 포함	○	김담현
TC-DH-03	중증 판정 소견 반영 확인	Vision AI가 특정 질환을 Severe로 판정 시, Expert 에이전트의 의학 소견에 반영되는지 확인	Vision AI 모델 (best_mri_model.pth) 로드 완료	협착증 MRI 이미지 + 일반 문진 데이터	vision_analysis의 Severe 값이 높게 도출되고, medical_note에 해당 소견 명시	○	김담현
TC-DH-04	이미지 미업로드 시 분석 차단 검증	MRI 이미지를 첨부하지 않은 상태에서 분석 시작 버튼을 클릭할 수 없는지 확인	프론트엔드 대시보드 화면 렌더링 상태	빈 파일 첨부 (Null)	분석 시작 버튼 비활성화(disabled) 상태 유지, API 호출 미발생	○	김담현
TC-DH-05	리포트 화면 렌더링 검증	응답받은 final_report 텍스트가 UI에 정상적으로 렌더링되는지 확인	TC-DH-01 정상 통과 (분석 완료 응답 수신)	분석 완료 응답 JSON	"AI CDSS Final Report" 섹션에 줄바꿈이 유지된 상태로 리포트 텍스트 정상 출	○	김담현
TC-DH-06	프론트엔드 API 연동 검증	파일과 문진 데이터가 묶인 FormData가 백엔드로 정확히 전송되는지 확인	네트워크 환경 정상 연결	MRI 이미지 파일, 문진표 JSON 객체	브라우저 Network 탭에서 /api/analyze/spine 엔드포인트로 multipart/form-data 요청 발생	○	김담현
TC-DH-07	서버 실패 시 Fallback 처리	서버 장애 발생 시 템플릿 데이터로 화면이 멈추지 않고 렌더링되는지 확인	FastAPI 서버 임의 종료 (장애 상황 시뮬레이션)	MRI 이미지 파일, 일반 문진 데이터	API 연결 실패 alert 노출 후, 하드코딩된 detailedResults 템플릿 기반으로 화면 렌더링	○	김담현
TC-DH-08	새 분석 초기화 동작 검증	새로운 영상 분석하기' 버튼 클릭 시 이전 데이터 초기화 및 화면 복귀 확인	분석 결과 화면 렌더링 상태	초기화 버튼 클릭 이벤트 발생	image, selectedFile, result 상태값이 null로 초기화되고 초기 업로드 UI로 복귀	○	김담현
TC-DH-09	문진표 JSON 직렬화 전송 검증	JS 객체인 문진표가 String으로 직렬화 전송 후, Python 서버에서 다시 Dictionary로 파싱되는지 확인	Spring, FastAPI 서버 구동 중	복합 문진표 데이터 (배열 및 문자열 포함)	FastAPI 로그에서 json.loads 실행 시 에러 없이 Dictionary로 정상 변환됨	○	김담현
TC-DH-10	Spring ↔ FastAPI 브릿지 통신 검	Spring Boot의 RestTemplate을 통해 파일/데이터가 FastAPI로 유실 없이 라우팅되는지 확인	Spring, FastAPI 서버 구동 중	React에서 송신한 원본 FormData	FastAPI 로그에 파일명과 문진데이터가 수신됨이 확인되고, Spring이 200 OK를 리턴받음	○	김담현
TC-DH-11	DB 영속화 (JPA Entity 저장)	통합 분석 완료 후 원본 파일명, JSON 비전/임상 결과, 텍스트 소견이 DB에 INSERT 되는지 확인	PostgreSQL 연결 및 spine_diagnosis_history 테이블 존재	통합 분석 파이프라인 완료 데이터	DB 조회 시 새 레코드 생성 및 clinical_symptoms, dl_vision_result 컬럼에 JSONB 형식으로 데이터 저장됨	○	김담현
TC-DH-12	비전 AI 이미지 전처리 검증	업로드된 이미지가 리사이즈, 정규화(mean/std)를 거쳐 PyTorch Tensor로 변환되는지 확인	FastAPI 서버 및 PyTorch 라이브러리 정상	임의의 해상도를 가진 MRI 이미지	preprocess_image() 통과 후 Tensor shape이 [1, 3, 224, 224]이며 FloatTensor 타입 반환	○	김담현
TC-DH-13	LangGraph 상태(State) 갱신 검증	analyzer -> expert -> communicator 노드를 거치 AgentState 딕셔너리에 출력값이 누적되는지 확인	Gemini API 키 정상 설정	Vision AI 출력 JSON + 문진표 JSON	app.stream() 실행 로그에서 노드 통과 시마다 State 객체의 해당 필드에 값이 순차적으로 채워짐	○	김담현

테스트 - 통합 테스트

작성자 : 김남준

통합 테스트

시나리오

시나리오 ID	시나리오 명	상세설명 (흐름도)	검증 포인트	테스트 결과	비고
SC-01	사용자 인증	① 회원가입 API 호출 (POST /api/auth/register) ② 중복·유효성 오류 케이스 검증 ③ 로그인 API 호출 (POST /api/auth/login) → JWT 수신 ④ JWT 포함 헤더로 보호 API 접근 ⑤ 토큰 없이 보호 API 접근 시도 → 차단 확인	JWT 정상 발급 / 유효성 검증 / 인증 통과 및 차단 여부		
SC-02	환자 관리	① 인증 토큰 획득 (SC-01 선행) ② 신규 환자 등록 (POST /api/patients) ③ 전체 환자 목록 조회 (GET /api/patients) ④ 환자 UID로 단건 조회 (GET /api/patients/{uid}) ⑤ 존재하지 않는 UID 조회 → 404 확인	환자 CRUD 정상 동작 / 인증 없이 접근 차단 / 404 처리		
SC-03	신부전 진단 (NJ)	① 환자 선택 및 임상 수치 입력 (25개 피처) ② POST /ai/kidney/diagnose 호출 → TabNet 5단계 예측 ③ 경고 알림 트리거 확인 (eGFR·칼륨·BUN 임계값) ④ RAG 소견 생성 확인 (pgvector + Gemini) ⑤ POST /api/patients/{uid}/diagnoses/kidney 로 결과 저장 ⑥ GET .../kidney/history 로 이력 조회	TabNet 예측 정확도 / 경고 레벨 분류 / RAG 소견 포함 / 저장·이력 정합성		
SC-04	뇌종양 진단 (WJ)	① NIfTI 파일 준비 (.nii / .nii.gz) ② POST /ai/brain/diagnose 에 파일 업로드 ③ DenseNet121 3D 분류 결과 수신 ④ 진단 결과 백엔드 저장 ⑤ 잘못된 파일 형식 업로드 → 오류 처리 확인	NIfTI 파싱 성공 / 종양·정상 확률 반환 / 오류 형식 거부 처리		
SC-05	피부질환 진단 (MS)	① 피부 병변 이미지 파일 준비 ② POST /ai/skin/diagnose 에 이미지 업로드 ③ 품질 검사 통과 여부 확인 (선명도·밝기) ④ EfficientNet 분류 결과 및 Grad-CAM 이미지 수신 ⑤ 저품질 이미지 업로드 → 품질 오류 반환 확인	품질 검사 필터링 / 분류 레이블 정확성 / Grad-CAM base64 반환		
SC-06	허리디스크 진단 (DH)	① GET /ai/spine/health 헬스체크 ② POST /ai/spine/diagnose 기본 진단 요청 ③ 진단 결과 백엔드 저장	AI 서버 정상 응답 / 응답 스키마 준수 / 저장 성공		

통합 테스트

시나리오

시나리오 ID	시나리오 명	상세설명 (흐름도)	검증 포인트	테스트 결과	비고
SC-07	대장암 진단 (GW)	① GET /ai/colon/health 헬스체크 ② POST /ai/colon/diagnose 기본 진단 요청 ③ 진단 결과 백엔드 저장	AI 서버 정상 응답 / 응답 스키마 준수 / 저장 성공		
SC-08	안과질환 진단 (SH)	① GET /ai/eyes/health 헬스체크 ② POST /ai/eyes/diagnose 이미지 업로드 ③ 진단 결과 백엔드 저장	AI 서버 정상 응답 / 안과 질환 레이블 반환 / 저장 성공		
SC-09	AI 워커 헬스체크 및 회로차단기	① GET /api/workers 전체 워커 상태 목록 조회 ② GET /api/workers/{name} 개별 워커 상태 확인 ③ 워커 연속 3회 실패 시 OPEN 전환 확인 ④ OPEN 30초 경과 후 HALF_OPEN → CLOSED 복구 흐름	6개 워커 등록 여부 / 상태값(HEALTHY·UNHEALTHY·UNKNOWN) / 회로차단기 전이		
SC-10	진단 이력 관리	① 동일 환자에 다수 진단 저장 (여러 질환 타입) ② GET /api/patients/{uid}/diagnoses 이력 조회 ③ 최신순 정렬 확인 ④ 신부전 전용 이력 API 확인 (kidney/history)	건수 정합성 / 최신순 정렬 / 질환별 필터링 / 빈 이력 200 반환		
SC-11	RAG 의료 지식 검색	① kidney 모듈 태그로 pgvector 유사도 검색 ② 상위 6개 문서 반환 및 중복 제거 확인 ③ Gemini 2.5 Flash 리포트 생성 ④ 다른 모듈(brain 등) 태그 격리 확인	검색 결과 건수·관련성 / 모듈 태그 격리 / 리포트 JSON 구조 준수		
SC-12	전체 E2E 플로우	① 회원가입 → 로그인 → JWT 획득 ② 환자 등록 또는 기존 환자 조회 ③ 신부전 임상 수치 입력 → AI 진단 호출 ④ 진단 결과 백엔드 저장 ⑤ 진단 이력 조회 및 저장 데이터 일치 확인 ⑥ 프론트엔드 화면 렌더링 정상 여부 육안 확인	전체 흐름 연속 성공 / 저장·조회 데이터 정합성 / UI 렌더링 정상		

통합 테스트

테스트

시나리오	테스트케이스 ID	테스트케이스 (절차)	사전조건	테스트 데이터	예상결과	테스트 결과	비고
SC-01 사용자 인증	TC-01-01	1. POST /api/auth/register 호출 2. 응답 상태코드 및 DB 저장 확인	이메일 중복 없음 DB 정상 연결	name: 홍길동 email: test@medicore.com password: password1234	HTTP 201 Created DB users 테이블에 행 추가		
	TC-01-02	1. TC-01-01 이후 동일 이메일로 재등록 시도 2. 응답 상태코드 확인	TC-01-01 완료 (동일 이메일 이미 존재)	email: test@medicore.com (중복)	HTTP 409 Conflict 오류 메시지 반환		
	TC-01-03	1. 비밀번호 7자로 회원가입 시도 2. 응답 상태코드 확인	없음	password: 1234567 (7자)	HTTP 400 Bad Request 유효성 오류 메시지		
	TC-01-04	1. POST /api/auth/login 호출 2. 응답 바디에서 token 필드 추출 3. 토큰 형식(JWT 3파트) 확인	TC-01-01 가입 완료	email: test@medicore.com password: password1234	HTTP 200 OK {token, name, email} 반환 token: eyJ... (JWT 형식)		
	TC-01-05	1. 올바른 이메일 + 잘못된 비밀번호로 로그인 2. 응답 상태코드 확인	TC-01-01 가입 완료	email: test@medicore.com password: wrongpassword	HTTP 401 Unauthorized		
	TC-01-06	1. 미등록 이메일로 로그인 시도 2. 응답 상태코드 확인	없음	email: ghost@medicore.com password: password1234	HTTP 401 Unauthorized		
	TC-01-07	1. JWT 없이 GET /api/patients 호출 2. 응답 상태코드 확인	없음	Authorization 헤더 없음	HTTP 403 Forbidden		
	TC-01-08	1. TC-01-04 토큰으로 GET /api/patients 호출 2. 응답 상태코드 확인	유효한 JWT 보유	Authorization: Bearer {token}	HTTP 200 OK 환자 목록 JSON 반환		
SC-02 환자 관리	TC-02-01	1. POST /api/patients 호출 2. 응답 바디 uid·name 일치 확인 3. DB patients 테이블 확인	JWT 보유 동일 UID 없음	uid: MED-0001 name: 김환자 age: 45 gender: 남	HTTP 201 Created {uid, name, age, gender} 반환		
	TC-02-02	1. TC-02-01 이후 동일 UID로 재등록 시도 2. 응답 상태코드 확인	TC-02-01 완료	uid: MED-0001 (중복)	HTTP 409 Conflict 또는 500 UID unique 제약 위반		중복 UID 예외처리 확인
	TC-02-03	1. GET /api/patients 호출 2. 배열 반환 및 항목 수 확인	TC-02-01 환자 1명 이상 등록	Authorization: Bearer {token}	HTTP 200 OK JSON 배열 반환, 길이 ≥ 1		
	TC-02-04	1. GET /api/patients/MED-0001 호출 2. uid·name 일치 확인	MED-0001 등록 완료	path: /api/patients/MED-0001	HTTP 200 OK uid: MED-0001		
	TC-02-05	1. GET /api/patients/NOT-EXIST 호출 2. 응답 상태코드 확인	없음	path: /api/patients/NOT-EXIST	HTTP 404 Not Found		

통합 테스트

테스트

시나리오	테스트케이스 ID	테스트케이스 (절차)	사전조건	테스트 데이터	예상결과	테스트 결과	비고
SC-03 신부전 진단 (NJ)	TC-03-01	1. GET /ai/kidney/health 헬스체크 2. POST /ai/kidney/diagnose 에 정상 수치 입력 3. 응답 prediction 확인	AI 서버(포트 8000) 실행 중 TabNet 모델 로드 완료	age:45, bp:80, sc:1.2, bu:35, hemo:13.5, pot:4.2, egfr:55, htn:yes, dm:no	HTTP 200 OK prediction: Stage 1~5 중 하나 confidence: 0.0~1.0 alerts: [] (정상 수치)		
	TC-03-02	1. eGFR 10으로 설정하여 진단 요청 2. alerts 배열에서 DIALYSIS_THRESHOLD 확인	AI 서버 실행 중	egfr: 10 (< 15 투석 임계값)	alerts 중 level:critical code:DIALYSIS_THRESHOLD 포함		투석 기준 경고 필수
	TC-03-03	1. 칼륨(pot) 6.2로 설정하여 진단 요청 2. alerts 배열에서 HYPERKALEMIA 확인	AI 서버 실행 중	pot: 6.2 (≥ 6.0 고칼륨혈증)	alerts 중 level:critical code:HYPERKALEMIA 포함		
	TC-03-04	1. TC-03-01 결과로 POST /api/patients/{uid}/diagnoses/kidney 호출 2. DB 저장 확인	JWT 보유 환자 MED-0001 등록	result, confidence, description, severity, dialysisRequired, egfr, sc 등 임상 수치	HTTP 201 Created diseaseType: kidney resultJson에 임상 수치 포함		
	TC-03-05	1. TC-03-04 저장 후 GET /api/patients/{uid}/diagnoses/kidney/history 호출 2. 결과 건수 및 최신순 확인	TC-03-04 완료	path: /api/patients/MED-0001/diagnoses/kidney/history	HTTP 200 OK 배열 길이 ≥ 1 result, confidence, createdAt 포함		
	TC-03-06	1. TC-03-01 응답에서 rag_answer 필드 확인 2. rag_sources 배열 비어있지 않음 확인	pgvector DB 연결 및 kidney 지식 삽입	AI 서버 응답 바디	rag_answer: 비어있지 않은 문자열 rag_sources: 배열 길이 ≥ 1		RAG DB 미연결 시 rule-based fallback
SC-04 뇌종양 진단 (WJ)	TC-04-01	1. GET /ai/brain/health 헬스체크 2. POST /ai/brain/diagnose 에 NIfTI 파일 업로드 3. 응답 classification 확인	AI 서버 실행 중 테스트용 .nii.gz 파일 준비	파일: test_brain.nii.gz Content-Type: multipart/form-data	HTTP 200 OK classification.prediction: Tumor 또는 Normal confidence: 0.0~1.0		
	TC-04-02	1. jpg 파일을 /ai/brain/diagnose 에 업로드 2. 오류 응답 확인	AI 서버 실행 중	파일: test.jpg (잘못된 형식)	HTTP 400 Bad Request 또는 오류 메시지 반환		파일 형식 검증
	TC-04-03	1. TC-04-01 결과로 백엔드 저장 2. 이력 조회	JWT 보유, 환자 등록	diseaseType: brain resultJson: AI 응답 JSON	HTTP 201 Created 이력 조회 시 brain 타입 포함		

통합 테스트

테스트

시나리오	테스트케이스 ID	테스트케이스 (절차)	사전조건	테스트 데이터	예상결과	테스트 결과	비고
SC-05 피부질환 진단 (MS)	TC-05-01	1. GET /ai/skin/health 헬스체크 2. POST /ai/skin/diagnose 에 정상 이미지 업로드 3. 분류 결과 및 Grad-CAM 확인	AI 서버 실행 중 선명한 피부 이미지 준비	파일: skin_sample.jpg 품질: 선명도 충분	HTTP 200 OK disease_label 반환 gradcam_image: base64 PNG		
	TC-05-02	1. 흐릿한/어두운 이미지 업로드 2. 품질 검사 실패 응답 확인	AI 서버 실행 중	파일: blurry_image.jpg (선명도 임계값 미달)	HTTP 422 또는 quality_failed 상태 반환 오류 사유 포함		
	TC-05-03	1. POST /ai/skin/diagnose/multi 에 이미지 2장 업로드 2. 배열 응답 확인	AI 서버 실행 중	files: [image1.jpg, image2.jpg]	HTTP 200 OK 배열 길이 2 각 항목에 disease_label 포함		배치 진단
SC-06 허리디스크 진단 (DH)	TC-06-01	1. GET /ai/spine/health 호출 2. status 필드 확인	AI 서버 실행 중	없음	HTTP 200 OK status: ok 또는 healthy		
	TC-06-02	1. POST /ai/spine/diagnose 기본 파라미터로 호출 2. 응답 구조 확인	AI 서버 실행 중	모듈별 정의된 진단 파라미터	HTTP 200 OK 응답 스키마 준수		
SC-07 대장암 진단 (GW)	TC-07-01	1. GET /ai/colon/health 호출 2. status 필드 확인	AI 서버 실행 중	없음	HTTP 200 OK status: ok 또는 healthy		
	TC-07-02	1. POST /ai/colon/diagnose 기본 파라미터로 호출 2. 응답 구조 확인	AI 서버 실행 중	모듈별 정의된 진단 파라미터	HTTP 200 OK 응답 스키마 준수		
SC-08 안과질환 진단 (SH)	TC-08-01	1. GET /ai/eyes/health 호출 2. status 필드 확인	AI 서버 실행 중	없음	HTTP 200 OK status: ok 또는 healthy		
	TC-08-02	1. POST /ai/eyes/diagnose 이미지 업로드 2. 응답 구조 확인	AI 서버 실행 중 안저 이미지 준비	파일: fundus_sample.jpg	HTTP 200 OK 안과 질환 레이블 반환		

통합 테스트

테스트

시나리오	테스트케이스 ID	테스트케이스 (절차)	사전조건	테스트 데이터	예상결과	테스트 결과	비고
SC-09 AI 워커 헬스체크	TC-09-01	1. GET /api/workers 호출 (인증 불필요) 2. 목록 크기 및 name 필드 확인	백엔드 서버 실행 중 6개 워커 DB 등록	없음	HTTP 200 OK 배열 길이 6 name: skin/brain/eyes/spine/kidney/colon 포함		permitAll 엔드포인트
	TC-09-02	1. GET /api/workers/kidney 호출 2. status 및 circuitState 확인	백엔드 서버 실행 중	path: /api/workers/kidney	HTTP 200 OK name: kidney status: HEALTHY/UNHEALTHY/UNKNOWN circuitState: CLOSED/OPEN/HALF_OPEN		
	TC-09-03	1. GET /api/workers/unknown 호출 2. 응답 상태코드 확인	없음	path: /api/workers/unknown	HTTP 404 Not Found		
	TC-09-04	1. PATCH /api/workers/kidney/url?url=http://... 호출 2. 업데이트된 URL 확인	kidney 워커 등록	url: http://192.168.0.100:8000	HTTP 200 OK url 필드 변경 반영		
	TC-09-05	1. AI 서버를 중단하여 헬스체크 3회 연속 실패 유도 2. /api/workers/kidney의 circuitState 확인	워커 헬스 스케줄러 실행 중	AI 서버 강제 중단 후 15초 대기	circuitState: OPEN failCount: 3 이상		실제 서버 환경에서만 가능
SC-10 진단 이력 관리	TC-10-01	1. 동일 환자에 kidney·brain·skin 진단 각 1건 저장 2. GET /api/patients/{uid}/diagnoses 호출 3. 건수 확인	JWT 보유 환자 등록	diseaseType: kidney / brain / skin	HTTP 200 OK 배열 길이 3 각 diseaseType 포함		
	TC-10-02	1. TC-10-01 이후 이력 목록 조회 2. createdAt 기준 최신순 정렬 확인	TC-10-01 완료	없음	배열 [0].createdAt >= 배열 [1].createdAt		
	TC-10-03	1. GET /api/patients/{uid}/diagnoses/kidney/history 호출 2. kidney 타입만 반환되는지 확인	kidney 진단 1건 이상 저장	없음	모든 항목의 diseaseType == kidney (또는 result 포함)		
	TC-10-04	1. 진단 없는 환자 UID로 이력 조회 2. 빈 배열 반환 확인	진단 없는 환자 존재	path: /api/patients/NEW-PATIENT-UID/diagnoses	HTTP 200 OK 빈 배열 []		

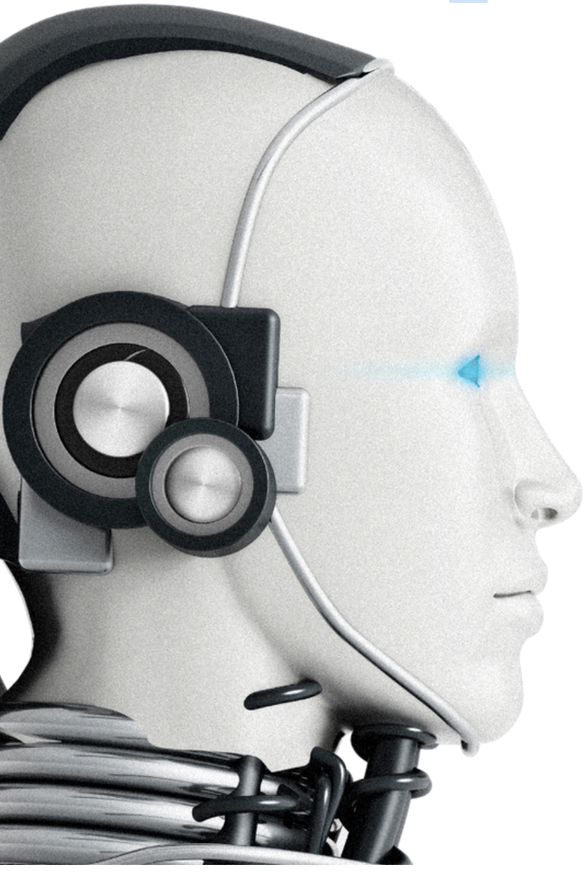
통합 테스트

테스트

시나리오	테스트케이스 ID	테스트케이스 (절차)	사전조건	테스트 데이터	예상결과	테스트 결과	비고
SC-11 RAG 의료 지식 검색	TC-11-01	1. kidney 모듈 진단 요청 2. 응답 rag_sources 배열 확인 3. 소스 출처 문자열 확인	pgvector DB 연결 kidney 태그 문서 삽입	query: CKD Stage 3 관리 방안	rag_sources 배열 길이 ≥ 1 출처(source) 문자열 포함		
	TC-11-02	1. kidney 진단 응답 rag_answer 텍스트 검토 2. JSON 구조 또는 마크다운 형식 확인	TC-11-01 완료	rag_answer 필드	비어있지 않은 의료 소견 문자열 한국어 포함		
	TC-11-03	1. brain 태그로 검색 시 kidney 문서 미포함 확인 (module 필터 격리 검증)	brain·kidney 문서 각각 삽입	module 필터: brain	반환 문서가 모두 brain 모듈 태그 kidney 태그 문서 미포함		AI 환각 방지 핵심
SC-12 전체 E2E 플로우	TC-12-01	1. POST /api/auth/register → 가입 2. POST /api/auth/login → JWT 획득 3. GET /api/patients → 환자 목록 조회 4. POST /ai/kidney/diagnose → AI 진단 5. POST /api/patients/{uid}/diagnoses/kidney → 저장 6. GET /api/patients/{uid}/diagnoses/kidney/history → 이력 확인	백엔드(8080) + AI 서버(8000) 동시 실행 환자 사전 등록	정상 임상 수치 (SC-03-01 데이터)	각 단계 HTTP 2xx 최종 이력에 저장 데이터 포함 전체 소요 시간 < 10초		주요 통합 시나리오
	TC-12-02	1. TC-12-01 완료 상태에서 2. 피부질환 이미지 업로드 진단 추가 3. GET /api/patients/{uid}/diagnoses 에서 skin 타입 포함 확인	TC-12-01 완료 피부 이미지 준비	skin_sample.jpg	이력에 kidney + skin 2가지 타입 모두 포함		다중 질환 저장
	TC-12-03	1. 로그인 후 kidney·brain 진단 동시 요청 (별도 HTTP 클라이언트) 2. 두 응답 모두 200 확인	백엔드 + AI 서버 실행 두 모델 로드 완료	kidney: 임상 수치 데이터 brain: test.nii.gz	두 요청 모두 HTTP 200 응답 지연 < 30초		동시성 검증

회고록

1주차



김민수

여러가지 기능 합치고 기초 설계를 하면서 모르는 부분이 많아서 힘들었지만 조원들의 도움으로 잘 해결해 나간 것 같습니다.

박기완

AI프로젝트가 아직은 생경한데 팀원들 도움으로 잘 진행하게 되었습니다.

김남준

AI 팀 프로젝트가 처음이어서 부족한 부분이 많다는 것을 깨달았습니다. 팀원 분들과 회의를 하면서 많은 것을 배웠고 프로젝트가 원활하게 이루어질 수 있게 노력하겠습니다.

김담현

시스템이 가진 제약 조건과 데이터의 구조를 먼저 파악하는 것이 설계의 핵심임을 몸소 체감했습니다

변운조

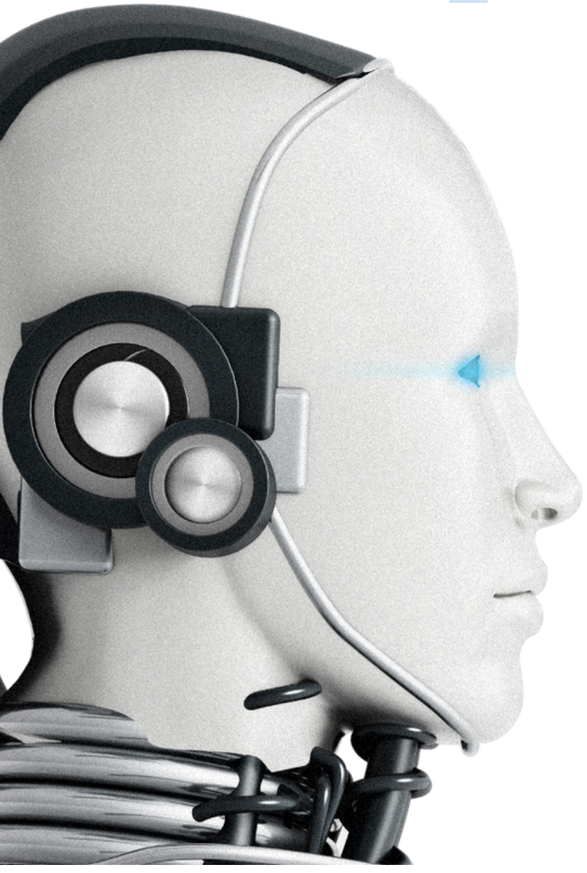
상상했던 기능을 구현하는데 생각 보다 제약 조건이 많아서 설계를 하는 것이 어려웠습니다.

홍승현

데이터 불균형을 해결하려 단계별로 스케일링을 고민했던 과정이 참 재밌었습니다. 덕분에 목적에 맞는 전처리와 학습 전략이 성능을 가른다는 걸 제대로 배웠습니다.

회고록

2주차



김민수

피부질환 분류 모델을 FastAPI 연동하면서 리액트랑 스프링이랑 서로 연결하는 경험이 힘들었지만 재미있었던 경험인 것 같습니다

박기완

모델 개발을 넘어 인프라 전반을 관리하며 시스템 안정성을 유지하는 과정이 생소하고 어려웠으나, 전체 서비스 흐름을 파악함

김남준

신부전 데이터셋에 맞춰 모델을 학습시키는 데에 어려움이 있어서 구현에 어려움을 겪었지만 전처리를 통하여 직접 해결했다는 것이 의미 깊었습니다.

김담현

단순히 기능 구현에 그치지 않고, 협업 환경에서 최적의 개발 효율을 낼 수 있는 구조(MSA 방식)를 고민해 본 뜻깊은 시간이었음.

변윤조

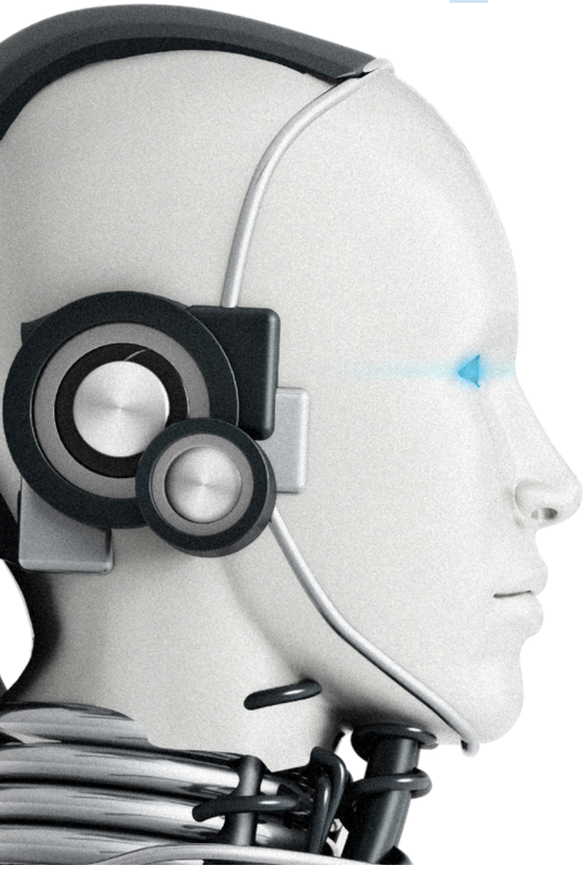
의료 데이터 특유의 복잡성과 불균형으로 인해 학습이 까다로웠으나, 증강 기법과 세밀한 튜닝을 통해 분류 성능을 끌어올리며 딥러닝 최적화의 중요성을 배웠습니다

홍승현

안저 이미지의 노이즈와 모호한 경계를 FOV 크롭과 CLAHE 기법으로 해결하며, 단순 이미지 처리를 넘어 AI가 임상적 특징에 집중할 수 있도록 고품질 의료 데이터를 구축하는 과정을 통해 이미지 전처리에 대한 지식을 배웠습니다.

회고록

3주차



김민수

도커 이미지로 DB 서버를 띄우면서 처음 해봤던 방식으로 진행하는 과정에서 많은 시행착오가 생겼지만, 배움의 기회가 늘어난 것 같아서 즐거웠습니다.

박기완

현업에서 쓰이는 방대한 IT 용어와 개념을 익히는 데 많은 에너지를 쏟았고 기술적 소통의 기반을 다지는 계기가 되었습니다.

김남준

WorkerPC 구성을 위한 Dockerfile 구성 및 의존성 모듈 설정에 있어 다른 환경에서 구축 시 어려움이 많아 시행착오가 많았지만 공부가 되었습니다.

김담현

도커 이미지 빌드 중 에러 로그를 분석하여 근본 원인을 파악하는 '데이터 중심의 문제 해결 능력'을 기를 수 있었음

변운조

파이썬 서버 간 비동기 통신과 데이터 직렬화 과정에서의 시행착오를 거치며, 안정적인 백엔드 아키텍처와 예외 처리의 필요성을 깊이 체감했습니다.

홍승현

데이터 불균형을 해결하려 단계별로 스케일링을 고민했던 과정이 참 재밌었습니다. 덕분에 목적에 맞는 전처리와 학습 전략이 성능을 가른다는 걸 제대로 배웠습니다.

Thanks for listening

Any questions?

참고 자료 변운조, 김민수, 김남준, 김담현, 박기완, 홍승현, 김용